

WEST CAPITAL LENDING

CLR Connection Center

Complete System Manual

The definitive reference for CLR's, Admins, and Operators

Version 1.0 · 2026 Edition

Frontend · Backend · Architecture · Operations · Troubleshooting

AUTHORED BY
Perplexity Computer

BUILT BY
Chris Redoble & Ethan Wood

CLR Connection Center

Complete System Manual

Edition. First Edition, 2026.

Publisher. West Capital Lending.

Author. Perplexity Computer.

Built by. Chris Redoble and Ethan Wood.

Typeset in. Inter (Google Fonts), embedded in this PDF at build time.

Platform URL. <https://web-production-b6285.up.railway.app>

This manual is the single authoritative reference for the CLR Connection Center platform operated by West Capital Lending. It covers every user-visible feature, every administrative workflow, the complete system architecture, the algorithm that powers daily lead assignment, and a deep troubleshooting reference. It is written for an intelligent non-technical person who must understand, operate, troubleshoot, and maintain the platform in day-to-day production.

All information in this manual reflects the state of the CLR Connection Center as of its 2026 release. Features, default algorithm weights, integrated services (such as Resend for email delivery, Railway for hosting, and GoDaddy for domain management), and the roster of active Loan Officers are accurate as of print. When the platform changes, update this manual in parallel.

© 2026 West Capital Lending. All rights reserved. Internal use only.

How to Use This Manual

This manual is long because the CLR Connection Center does a lot. You are not expected to read it cover to cover. Instead, it is organized so that you can enter at the chapter that answers the question in front of you and leave as soon as that question is answered.

The manual is divided into eight parts.

Parts 1 and 2 are the right starting point for a new CLR. They cover the mission of the platform, the user roles, navigation, and every daily workflow — your Calling List, your Appointments, your EOD Report, and Team Chat. A CLR who has read Parts 1 and 2 can run a full working day without help.

Part 3 covers LO management — the roster, adding and editing Loan Officers, availability, archiving and restoring, and LO performance statistics. This is where Admin-CLRs and Admin-only users spend most of their administrative time.

Part 4 is the deep technical explanation of the assignment algorithm. If you have ever wondered why a particular LO was or was not recommended on a given day, or you are considering changing the algorithm weights, this is the part you need.

Part 5 covers administrative operations — user management, algorithm settings, the email system, Bonzo credentials, NMLS compliance checks, the audit log, and the Settings reference. If you have been given admin rights, Part 5 is your reference.

Part 6 is the technical reference: system architecture, database schema, the algorithm in code, authentication, email infrastructure, notifications, and deployment. It is written for a technically curious operator rather than a developer, but it gives a developer everything they need to reason about the system.

Part 7 is the troubleshooting reference. When something is broken, Part 7 is the first thing to open. Each entry is a specific symptom, the cause, and a numbered recovery procedure.

Part 8 is the appendix — a quick reference card, the algorithm weight defaults, the complete permission matrix, a glossary of every term of art used in the platform, and contact information.

Conventions used in this manual

Numbered procedures are stepwise and should be executed in order. Bulleted lists are enumerations without implied ordering. Tables summarize reference data that you will come back to — permission matrices, algorithm weights, environment variables, and the like. Callout boxes with a gold left border contain warnings and tips; read them carefully.

About callouts

A gold callout box like this one is used whenever a single piece of information is important enough that you should not miss it. Warnings use the same treatment with a red border. Tips are informal but worth reading.

Table of Contents

PART 1 · Introduction

Chapter 1 · What is CLR Connection Center?	1
Chapter 2 · Getting Started	4
Chapter 3 · User Roles Explained	8
Chapter 4 · Navigation Guide	12

PART 2 · Daily Operations

Chapter 5 · The Dashboard	16
Chapter 6 · Your Calling List	21
Chapter 7 · Appointments	29
Chapter 8 · EOD Reporting	37
Chapter 9 · Team Stats	45
Chapter 10 · Team Chat	51

PART 3 · LO Management

Chapter 11 · The LO Roster	61
Chapter 12 · Adding and Editing LOs	67
Chapter 13 · Archiving and Restoring LOs	75
Chapter 14 · LO Availability System	80
Chapter 15 · LO Stats and Performance	85

PART 4 · Assignment System in Depth

Chapter 16 · How the Algorithm Works	101
Chapter 17 · Daily Generation	111
Chapter 18 · Admin Override	119

PART 5 · Admin Operations

Chapter 19 · Managing Users	131
Chapter 20 · Algorithm Settings	139
Chapter 21 · Email and Notification Settings	147
Chapter 22 · Bonzo and Lead Mailbox Credentials	153
Chapter 23 · NMLS Compliance Checks	157
Chapter 24 · History and Audit Log	163

Chapter 25 · Settings Reference	168
PART 6 · Technical Reference	
Chapter 26 · System Architecture	176
Chapter 27 · Database Schema	186
Chapter 28 · The Algorithm in Code	196
Chapter 29 · Authentication and Sessions	204
Chapter 30 · Email System	209
Chapter 31 · Notifications System	214
Chapter 32 · Deployment	218
PART 7 · Troubleshooting	
Chapter 33 · Common Issues Reference	221
PART 8 · Appendix	
Appendix A · Quick Reference Card	241
Appendix B · Algorithm Weight Defaults	243
Appendix C · User Permission Matrix	244
Appendix D · Glossary	246
Appendix E · Contact & Support	249

PART ONE

Introduction

The mission, the users, and how to find your way around.

What is CLR Connection Center?

A purpose-built operations platform that replaces spreadsheets, group texts, and memory with a single fair, measurable, auditable system.

The platform in one sentence

The CLR Connection Center is the internal operations platform at West Capital Lending that every Client Liaison Representative uses to receive their daily list of Loan Officers to work, log the outcomes of that work, coordinate with teammates, and close out the day with a standardized end-of-day report to management. It is the operational spine of the CLR team.

What the platform replaces

Before the Connection Center, three problems compounded each other. First, CLRs worked from informal spreadsheets and group messages — which LO to call, what was said last time, and who had already been contacted this week. Second, lead distribution depended on memory and habit, which meant some LOs were reliably over-worked while others were under-worked, independent of actual performance or availability. Third, end-of-day reporting was either skipped entirely or collected as a chat message that could not easily be searched, audited, or aggregated into team-level metrics.

The Connection Center replaces all three. A single algorithm generates the day's Calling List for every CLR — the same way, on the same schedule, using the same rules, every day. Appointments and transfers are logged as structured data that can be counted without anybody being asked. End-of-day reports are pre-populated from that logged data, submitted from the same interface, and delivered to management by email without human forwarding.

The mission

Every design decision in the platform is measured against one sentence: **spend CLR attention on actual conversations with Loan Officers, not on the bookkeeping that surrounds those conversations.** That is why the Calling List is generated for you each day, why transfer and appointment counts feed automatically into the EOD report, why chat and notifications are inside the same screen instead of across three apps, and why the audit log exists so that no one has to remember what happened yesterday.

The one principle that explains most of the product

If a human can mis-remember it, mis-count it, or forget to do it, the platform does it for them. The platform does not replace judgment — CLRs still choose how to run each conversation — but it removes the parts of the job where a human would add error without adding value.

Who the platform is for

The CLR Connection Center has three kinds of users, and every screen and every feature assumes one of them. Knowing which category you are in is the single most important thing for understanding what you will see when you log in.

CLRs (Client Liaison Representatives).

A CLR is the standard user and the primary audience of the platform. A CLR logs in, opens their Calling List, works through the ranked list of Loan Officers for the day, logs appointments as they happen, submits an EOD report at the end of the day, and uses Team Chat to stay coordinated with the rest of the team. A CLR cannot change algorithm weights, cannot archive Loan Officers, cannot add other users, and cannot see administrative history. The platform is deliberately simple for CLRs: the features they don't need are invisible to them.

Admin-CLRs.

An Admin-CLR is a CLR who has also been granted administrative permissions. They see everything a CLR sees — their own Calling List, their own appointments, their own EOD report flow — and they see everything an admin sees: user management, algorithm settings, NMLS check assignment, the audit log, and the ability to archive or edit Loan Officers. Critically, an Admin-CLR has **is_clr = true**, which means they continue to appear in daily Calling List generation just like any other CLR. They are a player and a referee at the same time.

Admin-only users.

An Admin-only user is a manager or operator who needs administrative access to the platform but is not themselves working a Calling List. This is the role assigned to leadership, support staff, and anyone who manages the system without actually calling Loan Officers. Admin-only users have **is_clr = false**, which excludes them from daily generation. They will not see a Calling List tab populated for them, and they will not appear in Team Stats individual breakdowns as a working CLR.

Role summary at a glance

CLR: standard user, appears in generation, no admin powers.

Admin-CLR: appears in generation AND has admin powers (**is_clr = true**).

Admin-only: admin powers, does NOT appear in generation (**is_clr = false**).

Where the platform fits in the West Capital Lending workflow

Loan Officers at West Capital Lending originate and work loan files for clients. CLRs are the relationship layer between the company and its Loan Officers — calling, checking in, handling transfers, and logging appointments. The Connection Center is the CLR-facing half of that relationship. Loan Officers themselves do not log in to the Connection Center; they are the subject of the CLR's work, not users of the platform.

Outside the Connection Center, CLRs also use **Bonzo** (the dialer / CRM that actually places calls and stores lead data) and a shared **lead mailbox** that receives new inbound leads. Credentials for both live inside the Connection Center on the Settings page so CLRs can copy them without leaving the platform. But the

Bonzo tool itself is separate software; the Connection Center does not place calls.

The problem the platform solves, in detail

Before we wrote a line of code, we interviewed CLRs and managers. Four recurring pain points defined what the platform had to fix.

Problem 1: assignment was informal and uneven.

Who worked which LOs on a given day was a function of habit, shift start order, and who grabbed the spreadsheet first. Over weeks, a few LOs received the bulk of CLR attention and the rest received almost none. This was not anyone's fault — it was the predictable outcome of an informal system.

The Connection Center solves this with a weighted algorithm that considers days since last worked, call frequency, availability, priority, and recent transfer volume. Over time, every active LO who should be contacted is contacted, and the distribution is traceable to a specific score rather than to a person's memory.

Problem 2: the end-of-day report was unreliable.

EOD reports arrived as freeform chat messages at inconsistent times. Some days half the team forgot. Managers could not trust the count of transfers and appointments without re-asking. Trend data across weeks was not possible because the reports were not structured.

The Connection Center solves this by (a) generating the transfer and appointment counts automatically from logged activity, (b) showing a popup reminder when a CLR logs in for the day and has not yet submitted their report, and (c) emailing the structured report directly to the managers. The CLR still writes the narrative portion — 'any other notable work done today not mentioned above' — but the numbers are not in their hands.

Problem 3: accountability required memory.

When a lead fell through a week ago, nobody could quickly tell who had been on it, what was said, or whether the LO had been available. Disputes — 'I did call them' / 'no one told me' — took hours to resolve and often stayed unresolved. The audit trail that matters for a compliance-sensitive business did not exist.

The Connection Center solves this with a single structured history: every login, every assignment generation, every admin override, every user change, and every NMLS compliance outcome is recorded with a timestamp and actor. The History page makes this readable by a non-technical admin.

Problem 4: coordination happened in too many places.

CLRs coordinated through group texts, sticky notes, and verbal handoffs across shifts. New CLRs had no way to catch up on context. Off-hours coverage was lost on return.

The Connection Center solves this with Team Chat integrated into the sidebar, notifications that persist until read, and a notification bubble that tells you at a glance whether there is something new to see. Everything that matters for coordination is one click away from the Calling List.

The payoff

Every one of these problems used to cost CLR time every day. The Connection Center was built so that the CLR's day ends with more conversations completed and fewer minutes spent on bookkeeping. Everything else in this manual is in service of that one outcome.

CHAPTER 2

Getting Started

System requirements, first login, interface tour, and installing the app.

System requirements

The CLR Connection Center is a web application. It runs inside any modern desktop or mobile browser without installation. You do not need to be a technical user to run it, and you do not need anything installed on your computer other than a browser.

Requirement	Minimum	Recommended
Browser	Chrome 100, Safari 15, Firefox 100, Edge 100	Latest Chrome or Safari
Operating system	Windows 10, macOS 11, iOS 15, Android 10	Any current OS
Internet	Any broadband connection	Wired or strong Wi-Fi
Screen	1280×720	1920×1080 or retina display
Cookies	Enabled (required for login session)	Enabled
JavaScript	Enabled (required)	Enabled

If something looks broken

Ninety percent of 'the site looks weird' reports are solved by a hard refresh (**Ctrl + Shift + R** on Windows or **Cmd + Shift + R** on macOS) or by clearing the browser cache. Try that before reporting an issue.

Logging in for the first time

The production URL is:

`https://web-production-b6285.up.railway.app`

Bookmark this URL. (West Capital Lending may in the future publish this under a custom subdomain such as `clr.wlc.it.com`; when that happens, the short domain will work as well. Either URL lands on the same platform.)

Your first login follows this sequence:

1. An administrator creates your account in the **Settings → Manage Users** panel. At account creation, the admin sets your role (CLR, Admin-CLR, or Admin-only) and decides whether to send you a welcome email. The welcome email toggle is on by default.

2. You receive a welcome email from **reports@wlc.it.com**. The email contains your login URL, your username (your email address), and a temporary password set by the admin. If you did not receive this email, check your spam folder and then contact the admin who created your account.
3. Open the login URL in any modern browser. Enter your email and the temporary password. Click **Sign In**.
4. On your very first login, the platform displays a welcome introduction that walks through the core features. This intro is governed by a flag on your user record called **hasSeenIntro**, which starts out false. When you finish the intro — or skip it — the flag flips to true and the intro will not appear again.
5. After the intro, you land on the Dashboard. Change your password at your earliest convenience by asking an admin (self-service password reset is a planned feature).

Understanding the interface

The application is organized around a left sidebar and a main content area. The sidebar is always present and contains six menu groups. The content area changes depending on what you have selected.

The sidebar is organized into six groups:

Group	What it contains	Who sees it
Main	Dashboard, LO Management, Your Calling List	All users
Team	Team Stats, Team Chat	All users
Reports	Call Reports, EOD Reports, Upcoming Schedule, LO Stats, LO Breaks	All users
Tools	State Lookup, NMLS Checks, Stats	All users
Admin	Settings, History	Admins only
Help	Watch Intro Video, Install App, Help Videos	All users

Within each group, menu items that are relevant to you are shown in full color. Items that require admin rights appear only if your role has them. If an entire group has no items you can see (for example, a CLR never sees the Admin group), the group header is hidden as well.

Top bar and right-side controls

The top bar contains, from left to right: the sidebar toggle (useful on narrow screens), the page title for whatever you are currently looking at, and a right-side cluster of controls: a notification bell, a dark/light mode toggle, and your user menu (click your avatar or initials to sign out).

Dark mode and light mode

The theme toggle is in the top right. The platform remembers your preference per browser — if you pick dark mode on your laptop, dark mode is what you will see the next time you open the site on that laptop. The setting is stored in **localStorage**, not in your user record, so different devices can have different themes for the same person, and clearing browser data resets the preference.

Dark mode uses the same navy and gold accents at lower luminance. All text and UI elements are designed to meet WCAG AA contrast in both themes. If you find any text difficult to read in either mode, report it — it is a bug.

A note on preference persistence

Your theme choice, your recently visited pages, and a handful of UI hints are stored in your browser's local storage. Logging out does not clear them. Using a new browser or a private window will reset them to defaults.

Installing as a Progressive Web App

The CLR Connection Center is installable as a Progressive Web App (PWA). When installed, it appears on your desktop or home screen with its own icon, opens in its own window without the browser chrome, and feels like a native application. It is still the same platform loaded from the same server; installation just wraps it in an application shell.

To install the app on a desktop browser:

1. Open the CLR Connection Center in Chrome, Edge, or another Chromium-based browser. (Safari on macOS supports installation since macOS Sonoma; Firefox does not support PWA install on desktop.)
2. Look for an install icon in the right of the address bar — it looks like a small monitor with a downward arrow. Alternatively, open the browser menu and select **Install CLR Connection Center**.
3. Confirm the install in the dialog that appears. The app will launch in its own window.
4. The app icon is added to your Applications folder (macOS), Start menu (Windows), or app drawer (Chromebook).

To install on iPhone or iPad:

1. Open the CLR Connection Center in Safari — not Chrome, Edge, or any other browser. On iOS, only Safari can install PWAs.
2. Tap the **Share** icon at the bottom of the screen (a square with an upward arrow).
3. Scroll down in the share sheet and tap **Add to Home Screen**.
4. Edit the name if desired, then tap **Add**.
5. The CLR Connection Center icon appears on your home screen. Tap it to launch.

To install on Android:

1. Open the CLR Connection Center in Chrome.
2. Chrome will show an install prompt at the bottom of the screen; tap **Install**. If you dismissed the prompt, open the browser menu (three dots, top right) and select **Install app**.
3. The app appears in your app drawer and on your home screen.

Why install the app?

Installing the PWA adds one genuine benefit beyond aesthetics: faster launch time because the app shell is cached locally. On mobile, this turns the Connection Center into a one-tap tool instead of 'open browser, find tab, maybe log in again.' On desktop, it moves the app out of the browser so a rogue browser restart doesn't kill your session.

A first tour of the Dashboard

Right after your first login, you land on the Dashboard. The Dashboard is a summary screen — it doesn't do anything by itself, but it shows you at a glance what matters today. The most prominent element is the **Upcoming Appointments** card, which counts every appointment with a future follow-up date across all CLRs.

Chapter 5 covers the Dashboard in detail. For now, it's enough to know that the Dashboard is safe — you cannot break anything by clicking around. The platform never asks for destructive confirmation on the Dashboard because the Dashboard is read-only.

What to do next

If you are a new CLR, the most useful next step is to read Chapter 4 (Navigation Guide), then skip ahead to Chapter 6 (Your Calling List). That pair of chapters will let you run your first real working day.

If you are an Admin-CLR or Admin-only user, read Chapter 3 (User Roles Explained) next to understand exactly what your role grants and then continue into Parts 3 and 5, which are your main administrative reference.

CHAPTER 3

User Roles Explained

Three roles, one toggle (`is_clr`), and everything that flows from them.

The three roles

Every user in the CLR Connection Center has exactly one role. The role is set when the account is created and can be changed only by an administrator. The role determines two things: what menus and actions the user sees, and whether the user participates in daily Calling List generation.

CLR — standard user.

A CLR is the intended primary user of the platform. They log in, work their Calling List, log appointments, submit EOD reports, and chat with the team. They appear in daily generation (`is_clr` = true for every CLR), which means they receive a list of Loan Officers to work every business day. They cannot see the Admin sidebar group, cannot change algorithm weights, cannot add or delete users, and cannot archive or delete Loan Officers. Most of the team is a CLR.

Admin-CLR — CLR with administrative powers.

An Admin-CLR is a CLR who also needs administrative capabilities — typically a senior CLR, a team lead, or a player/coach figure. They have every capability a CLR has plus every capability an admin has. Importantly, **`is_clr` is still true** for an Admin-CLR, which means they continue to be assigned Loan Officers by daily generation just like any other working CLR. They are a full participant in the Calling List. The platform does not treat them specially in that regard — an Admin-CLR who is marked as available on a given morning gets a full list, same as anyone else.

Admin-only — administrator who does not call.

An Admin-only user has every administrative capability but **`is_clr` is false**. This is the role for leadership, operations staff, support, or an external consultant who needs access to manage the system but is not working a Calling List. An Admin-only user will not see their own Calling List populated, will not appear in Team Stats as a working CLR, and will not receive Loan Officer assignments from daily generation. They will, however, see every administrative page and can act on behalf of the system as needed.

The one field that controls everything

Under the hood, a single database column called **`is_clr`** (a boolean) determines whether a user appears in daily generation. A user's role ('CLR', 'admin') is a separate text field. The combination yields the three effective roles. When an admin creates or edits a user, they set both the role and the `is_clr` toggle; the UI exposes this as a friendly dropdown, but it's worth knowing that the underlying mechanism is simple.

Permission matrix

The following table is the definitive reference for what each role can and cannot do. When in doubt, consult this table; the UI is authoritative, but the table is the intent behind the UI.

Capability	CLR	Admin-CLR	Admin-only
View own Dashboard	✓	✓	✓
View own Calling List	✓	✓	—
Log appointments	✓	✓	—
Complete appointments (Transfer / Fell Through)	✓	✓	—
Submit EOD Report	✓	✓	—
Use Team Chat	✓	✓	✓
View Team Stats	✓	✓	✓
View Call Reports (own)	✓	✓	—
View Call Reports (all CLRs)	—	✓	✓
View LO Roster	✓	✓	✓
Add new Loan Officer	—	✓	✓
Edit LO profile	—	✓	✓
Archive or restore LO	—	✓	✓
Delete LO (permanent)	—	✓	✓
Set LO priority tier	—	✓	✓
Manage LO availability / breaks	—	✓	✓
Trigger daily generation	—	✓	✓
Override today's assignments	—	✓	✓
View Algorithm Settings	—	✓	✓
Edit algorithm weights	—	✓	✓
Manage Users (add / edit / delete)	—	✓	✓
Edit Bonzo / lead mailbox credentials	—	✓	✓
Edit NMLS check schedule	✓	✓	✓

Capability	CLR	Admin-CLR	Admin-only
Complete an NMLS check assigned to them	✓	✓	—
View History / Audit Log	—	✓	✓
Appear in daily Calling List generation	✓	✓	—
Receive auto-generated Calling List assignments	✓	✓	—
Receive EOD popup reminder after first daily login	✓	✓	—

How to determine your role

Four independent signals will tell you your role without having to ask anyone.

- 1. Check the sidebar.** If you see the **Admin** group (with Settings and History), you are either an Admin-CLR or an Admin-only user. If you do not see it, you are a CLR.
- 2. Check for Your Calling List.** If you see **Your Calling List** populated with LOs on a typical working day, you are a CLR or Admin-CLR. If you see the menu but it is empty every day, you are an Admin-only user.
- 3. Check Team Stats.** If you see yourself listed as a working CLR in the Team Stats individual dropdown, you appear in generation (CLR or Admin-CLR). If not, you are an Admin-only user.
- 4. Ask an admin.** Any Admin-CLR or Admin-only user can look you up in **Settings → Manage Users** and read your role and is_clr flag directly.

Why three roles instead of two

A two-role system ('user' and 'admin') is common, but it creates a friction the Connection Center specifically wanted to avoid: team leads who are also working a Calling List. In a strict two-role world, you either give a team lead admin rights (and they drop out of generation, which hurts the team's capacity) or you leave them as a regular user (and someone else has to do every admin task for them). The Admin-CLR role resolves this by making admin a layer on top of CLR rather than a mutually exclusive role. It costs one extra database column (is_clr) and saves the operational headache in perpetuity.

Changing a user's role

Only an administrator can change a user's role. The process is covered in detail in Chapter 19 (Managing Users). Briefly: go to **Settings → Manage Users**, find the user, click Edit, change the role dropdown or the is_clr toggle, and save. The change takes effect immediately; the user will see the new menu on their next page load. Any change to role or is_clr is written to the audit log with the acting admin's identity and a timestamp.

A warning about demoting an admin

If you demote an Admin-CLR to a plain CLR while you are logged in as that user, you will lose access to the Admin menus on your next page load. This is correct behavior, but it can be surprising. If you need to test the CLR view, use an incognito window rather than demoting yourself.

Role implications for daily generation

Daily generation — the process that builds each CLR's Calling List every morning — pulls the list of CLRs from all users where **is_clr = true**. The generator does not care whether the user is a plain CLR or an Admin-CLR; both participate equally. Admin-only users are skipped entirely.

This is why the `is_clr` flag matters as much as the role. If you have an admin who used to work a Calling List but no longer does, you do not need to demote them to a plain user — you flip `is_clr` to false and their role stays admin. They keep every admin capability and simply drop out of generation going forward. If they later return to active CLR duty, flip `is_clr` back to true and they rejoin the next day's generation.

Role implications for Team Stats

Team Stats shows per-person performance for every user where `is_clr = true`. An Admin-only user will not appear in the individual CLR dropdown on Team Stats. An Admin-CLR will, and their stats are computed just like any plain CLR's.

Role implications for NMLS checks

The NMLS compliance check system rotates checks randomly among CLRs (`is_clr = true`). Admin-only users are excluded from the rotation. Any user — including Admin-only — can edit the schedule, but only CLRs and Admin-CLRs are eligible to be assigned a check.

Role implications for EOD Reports

The EOD popup appears only for `is_clr = true` users after their first daily login, and only if they have not yet submitted today's report. Admin-only users do not see the popup and are not expected to submit EOD reports (they are not working a Calling List).

Role implications for Team Chat

All three roles see and participate in Team Chat equally. Chat is the one place where the platform makes no distinction by role — everyone is a full participant in conversation.

Summary

Role governs what you can do administratively. `is_clr` governs whether you participate in daily generation, Team Stats, the EOD popup, and NMLS check rotation. These two fields together cover every meaningful distinction between users.

CHAPTER 4

Navigation Guide

A complete tour of every menu, every page, and every control.

Six sidebar groups

The left sidebar is the primary navigation for the entire platform. It is organized into six named groups, each containing a set of related pages. On desktop the sidebar is always visible; on mobile it slides in from the left when you tap the menu button.

The Main group

The Main group is the landing area for every user. It contains three items.

Dashboard. The read-only summary screen. Shows an Upcoming Appointments count, unread notification count, your role and team context, and a handful of at-a-glance tiles that help you decide what to do first. Most CLRs open the Dashboard at the start of each shift to orient themselves. Covered in depth in Chapter 5.

LO Management. The master Loan Officer roster — every LO that is active or archived, with full profile detail, availability settings, states licensed, priority tier, and recent performance. Read-only for CLRs; editable for admins. This is where an admin adds a new LO or flips an LO's availability. Covered in Part 3.

Your Calling List. The list of Loan Officers the platform has assigned to you for today. This is the most-used page in the platform for most CLRs — it is the working surface for the entire day. Covered in Chapter 6.

The Team group

The Team group contains everything related to you-in-context-of-your-teammates.

Team Stats. Performance metrics for the whole team. Individual CLR breakdowns are available via a dropdown. Metrics include transfers, appointments, call volume, and — new in 2026 — the percentage of recommended LOs each CLR actually gets through. Covered in Chapter 9.

Team Chat. A group chat channel for the entire CLR team. Real-time messaging, persistent history, and a notification bubble on the sidebar when there are unread messages. Covered in Chapter 10.

The Reports group

The Reports group is where structured operational data lives. These pages are read-heavy — you open them to answer a specific question.

Page	What it shows	Primary audience
Call Reports	Per-call outcome logs across the team (admins see all; CLRs see own)	All roles
EOD Reports	Your submitted and historical end-of-day reports; admins see all	All roles
Upcoming Schedule	Forthcoming appointments across the team	All roles
LO Stats	Per-LO performance: transfers, appointments, last worked, priority	All roles
LO Breaks	Scheduled LO unavailability (vacation, training, etc.)	All roles

These pages are always available to every signed-in user, but the content they show respects your role. A CLR viewing Call Reports sees only their own calls; an admin sees the whole team.

The Tools group

The Tools group is a catchall for utilities that support the job but are not part of the core daily workflow.

State Lookup. A quick reference that answers 'which LOs are licensed in state X?' Type or pick a state and see the roster of eligible Loan Officers. Useful when an unexpected lead comes in for a state you are not familiar with.

NMLS Checks. The NMLS compliance workflow. Schedule, current assignments, escalation status, and completion. Covered in Chapter 23.

Stats. A general statistics page that rolls up platform-wide metrics — total assignments this month, average daily transfers, and similar aggregate numbers. Read-only for everyone.

The Admin group

The Admin group is visible only to Admin-CLRs and Admin-only users. If you do not see it, your role is CLR.

Settings. The central control panel for the platform. Algorithm weights, user management, email and notification settings, Bonzo and lead mailbox credentials, NMLS check schedule, override controls, and the ability to manually trigger daily generation. Covered chapter by chapter in Part 5 and summarized in Chapter 25.

History. The audit log. Every administrative action that modifies the system is recorded here with a timestamp and an actor. This is the page you open when you need to answer 'who changed that?' Covered in Chapter 24.

The Help group

The Help group is a persistent entry point to assistance material. Every user sees it.

Watch Intro Video. Opens the 2-minute welcome video that new users see on their first login. You can re-watch it any time by using this menu item. Useful when onboarding a peer or refreshing on a feature you haven't used in a while.

Install App. Opens an in-app dialog with step-by-step instructions for installing the Connection Center as a PWA on your current device. The dialog detects your device and browser and shows the relevant steps. If you've already installed, the dialog tells you.

Help Videos. A library of short how-to videos covering common tasks — logging an appointment, completing the EOD report, using Team Chat, and others. These are short and focused; most are under 90 seconds.

Navigating within a page

Within any page, tabs and section headers break content into chunks. The LO Management page, for example, has tabs for Active, Archived, and Stats. The Settings page has tabs for every administrative subsystem. Tabs are preserved in the URL, which means you can bookmark or share a link to a specific tab.

The URL / routing system

The platform uses hash-based routing (URLs like `.../#/calling-list`). This has one practical implication: copying a link to a specific page copies the hash too, and the recipient will land exactly where you were. It is safe to bookmark any page this way. Chapter 26 covers the reasoning in technical detail.

The notification bell

In the top right, a bell icon shows a count badge when you have unread system notifications. Click it to expand the notification panel, which shows recent notifications with a read/unread state. Notifications include things like 'an LO is missing required profile information' (broadcast to all users every 3 days until resolved), 'an NMLS check has been assigned to you', and 'today's assignments have been overridden by an admin'. Clicking a notification marks it as read; you can also clear them en masse.

The user menu

Clicking your name or initials in the top right opens a small menu with two items: **Profile** (basic profile info, read-only for most fields) and **Sign Out**. Sign Out ends your session immediately; the next request to any page will redirect to the login screen.

Session duration

Sessions are long-lived by design — CLRs should not have to log in multiple times per day. A session remains valid as long as the signed cookie is present and unexpired. Clearing your browser cookies, using a private window, or signing out explicitly will invalidate the session.

PART TWO

Daily Operations

The working day of a CLR — Dashboard, Calling List, Appointments, EOD, Stats, Chat.

The Dashboard

Your single-glance view of what matters today, read-only by design.

What the Dashboard is for

The Dashboard is the first page you see after logging in, and it is the page you are likely to open ten times during a working shift as a mental checkpoint. It is a read-only summary, which means nothing you can click on the Dashboard changes any data. It is an orientation screen: you open it, you scan it, and you decide what to open next. This is an intentional design choice. The Dashboard answers 'where does my attention go right now?' — it does not ask you to commit to any action.

The anatomy of the Dashboard

The Dashboard is organized as a grid of cards. Each card owns a single metric or a single list. There are no nested tabs on the Dashboard. The intent is that every card can be read in less than a second.

The Upcoming Appointments card

The Upcoming Appointments card is the most prominent element on the Dashboard. It shows a single number: the count of appointments across the entire team whose follow-up date is today or later. Under the hood, this is a direct count of rows in the appointments table where followUpDate is greater than or equal to today's date and the appointment's status is still open (not completed with a Transfer or Fell Through outcome).

This number is intentionally a team-wide count rather than a per-CLR count. The reasoning: if the team has eighty upcoming appointments and you personally have three, the team total is the better indicator of pipeline health. If it crashes from eighty to six overnight, that is worth noticing — even though none of those sixty lost appointments belonged to you.

Why the number might surprise you

Upcoming Appointments counts every future appointment that has not been completed, not only appointments you created. If the number seems too high, check the Upcoming Schedule page — it is almost always correct, just higher than you intuitively expected. Appointments scheduled far into the future (30-60 days out) accumulate and stay on the counter until they resolve.

Other metrics on the Dashboard

In addition to Upcoming Appointments, the Dashboard surfaces a handful of secondary metrics that are useful to see at a glance. Each is a small card with a single number or a very short list. The exact set of cards may evolve, but at print the following are the standard cards.

Card	What it shows	Updates
Upcoming Appointments	Team-wide count of future appointments (status: open)	Real time
Today's Transfers	Appointments completed as Transfer today, across the team	Real time
Today's Fell-Through	Appointments completed as Fell Through today, across the team	Real time
Notifications	Your unread notification count	Real time
Active LOs	Count of LOs where isActive=true and isArchived=false	On any LO change
LOs Missing Info	Active LOs missing NMLS ID, phone, or email	On any LO change
Last Generation	Timestamp of the most recent daily Calling List generation	On generation
EOD Status (you)	Whether you have submitted today's EOD report	On submit

The EOD Status card is the Dashboard's one personal card: it tells you whether you have submitted today's EOD report. If you have not, and the time is after a late-afternoon threshold, the card will display a reminder with a link that jumps to the EOD submission flow. This is a softer version of the EOD popup (covered in Chapter 8) — the popup is mandatory on your first login of the day; the Dashboard reminder is gentle.

How Admin-only users see the Dashboard

An Admin-only user still lands on the Dashboard after login and sees every card. The EOD Status card, however, is suppressed for Admin-only users because they do not submit EOD reports. In its place, Admin-only users see a card for the most recent admin actions — a compact audit-log preview that links to History.

Starting your day with the Dashboard

A reliable daily routine, in order:

1. Open the Dashboard. Glance at Upcoming Appointments. If the number moved in an unexpected direction overnight, open Upcoming Schedule next to understand why.
2. Check LOs Missing Info. If this is non-zero, any CLR may have the means to fix the issue (updating NMLS ID, phone, or email on an LO's profile). If you are a CLR, you do not have edit rights — but noting the count tells you the team will be receiving a notification nudge until it is resolved.
3. Check Last Generation. If the timestamp is anything other than earlier today, the daily generator has not run yet. On a normal weekday this should resolve by the time you are working; if it has not, an admin should trigger generation manually in Settings.
4. Scan Notifications. If the count is non-zero, open the bell menu (top right) and clear anything relevant.
5. Open **Your Calling List**. This is where your day really begins.

CLRs who have been on the team for a while develop an intuition for the normal ranges of each metric. A team-wide Upcoming Appointments count that typically sits around 65 and suddenly shows 15 is a signal worth investigating. The Dashboard is designed to make those anomalies immediately visible.

What you cannot do from the Dashboard

The Dashboard is read-only. You cannot create appointments here, you cannot complete appointments here, you cannot generate the Calling List here, you cannot edit LOs here, and you cannot submit an EOD report here. Every card either surfaces a number or links to another page where the action lives. This is a deliberate separation: the Dashboard is a map, not a steering wheel.

Rule of thumb

If you open the Dashboard and see something you want to change, the change happens on another page. Click through — the Dashboard's job is to send you there.

Dashboard cards and their source of truth

Every number on the Dashboard is derived from live queries against the database at the time you load the page. There is no caching or scheduled refresh. If two CLRs open the Dashboard at the same moment, they will see identical numbers. If one of them completes an appointment a minute later and both refresh, both will see the new number. This makes the Dashboard trustworthy as a real-time operational picture.

The metrics are not aggregated into a separate statistics table. They are computed on demand from the primary tables: appointments, users, los, notifications, and audit_log. This keeps the data honest — there is no possibility of the Dashboard disagreeing with the underlying pages, because they all read the same rows.

Performance considerations

Because the Dashboard queries live data, it is the page most sensitive to the overall health of the database. Under normal load the queries return in well under a second. If the Dashboard ever takes more than two seconds to load, something is wrong — most commonly a Railway instance restart in progress or a database lock from a long-running operation. Chapter 33 (Troubleshooting) covers diagnostics for slow loads.

A short closing note on the Dashboard

The Dashboard is the simplest page in the platform, and that is a feature, not a limitation. It exists to answer one question: what should I do next? It does not try to answer any other question. Everything else — how to work a list, how to log an appointment, how to submit an EOD — lives on its own page, reached in a single click from here.

CHAPTER 6

Your Calling List

The ranked list of Loan Officers to work today, produced by a weighted algorithm and locked to your identity for the day.

What the Calling List is

Your Calling List is the ordered list of Loan Officers that the platform has assigned to you for today. It is produced once per day by the generation algorithm, and once produced, it is yours for that day. No one else is working the same list; the algorithm makes sure every active LO appears on exactly one CLR's list for the day.

The list is ordered by score — the LO with the highest score appears first, the LO with the lowest score appears last. The score is a number between 0 and 1 that represents how much the platform thinks you should prioritize this LO today. The score is computed from six factors that, together, balance recency, fairness, availability, and performance.

The six factors, at a glance

Factor	Default weight	What it measures
Days Since Last Worked	30%	How long since anyone worked this LO
Call Frequency	25%	How often this LO is typically contacted, inverted
Availability	20%	Whether this LO is available in today's time window
Priority Boost	10%	Short-term admin-applied boost for strategic reasons
Priority Tier	5%	The LO's standing tier (Premium, Standard, Reduced)
90-Day Transfer Volume	10%	Recent transfer performance

Those six weights must sum to 1.0. Admins can change them, but the platform normalizes any configuration to sum to 1.0 on save — more on that in Chapter 20. For now, the important thing is that these six numbers determine the entire behavior of the generator. There is no hidden seventh factor.

Factor 1: Days Since Last Worked (30%)

This is the single most important factor. It measures the number of days since any CLR last worked this LO — not just you. If LO Bill Neessen was last worked six days ago by any CLR, today his Days-Since-Last-Worked is 6. The factor is normalized so that a longer gap produces a higher score.

The rationale is straightforward: Loan Officers who have not been contacted recently are more likely to have accumulated something worth hearing, and they are more likely to feel neglected if the pattern continues. By giving this factor 30% of the total weight, the algorithm ensures that the team does not circle the same handful of familiar LOs while others go weeks without contact.

Factor 2: Call Frequency (25%)

Call frequency is the inverse of how often an LO is typically contacted. An LO who is called every other day has a high intrinsic frequency and therefore a lower score contribution from this factor. An LO who is called once a week has a lower intrinsic frequency and a higher score contribution. The factor is computed from rolling 30-day call history.

This is distinct from Days Since Last Worked. Days Since measures the actual gap since the most recent call; Call Frequency measures the long-run pattern. Both matter. A normally-daily LO who hasn't been called in three days is overdue; a normally-weekly LO who hasn't been called in three days is not.

Factor 3: Availability (20%)

Each LO has an availability setting: morning, afternoon, or both. The current time of day is compared to the LO's availability window, and the factor returns 1.0 if the LO is available in the current window, 0.5 if they are available later in the day, and 0.0 if they are not available at all today (e.g., on break). At 20% of the total, the availability factor significantly but not decisively shapes the list — an LO on break will not appear at the top, but an LO who is available later in the day still might, if their other factors are strong.

Factor 4: Priority Boost (10%)

Priority Boost is an admin-applied temporary nudge. An admin can mark an LO as boosted for a reason — a new LO who needs early attention, a strategic relationship, a recovery from a rough stretch — and that LO will score a little higher in generation until the boost is removed. Priority Boost is binary (on or off) and decays automatically after a configurable period so that forgotten boosts do not compound.

Factor 5: Priority Tier (5%)

Every LO has a priority tier: Premium (tier 1), Standard (tier 2), or Reduced (tier 3). Standard is the default, and at print all twelve active WCL Loan Officers are Standard. Premium tier contributes the most to this factor, Standard is neutral, and Reduced is negative. At only 5% of total weight, Priority Tier is the gentlest of the factors — it shapes the edges of the list rather than the middle.

The tier system exists so that admins can reflect a structural decision about an LO's standing without overriding the rest of the algorithm. A Premium LO will appear earlier in the list on days when all other factors are roughly equal, but will not skip ahead of a Standard LO who has been neglected for a week.

Factor 6: 90-Day Transfer Volume (10%)

This is the performance factor. It counts the number of transfers credited to an LO in the most recent 90 days and normalizes across the roster. Higher transfer volume means higher contribution. This factor lets the algorithm gently reward the LOs who are currently converting — giving them a small edge in attention, since attention to a closing LO often produces more closures.

In code, this factor is not stored in the Drizzle ORM schema — it is computed at score time by a raw SQL query against the appointments table. Chapter 28 covers the implementation in depth.

Weights must sum to 1.0

The algorithm strictly requires that the six weights sum to 1.0. Admins can drag sliders in Algorithm Settings to any value, and the platform normalizes on save. If the stored weights ever sum to more than 1.05 (which can happen after certain migrations), a startup routine renormalizes them automatically and writes an entry to the audit log.

How a score is built

For each active, unarchived LO, the generator computes six raw factor values (each between 0 and 1), multiplies each by its weight, and sums them. The result is the LO's score for today. Here is a worked example using the default weights.

Factor	Raw value	Weight	Contribution
Days Since Last Worked	0.80 (last worked 8 days ago)	0.30	0.240
Call Frequency	0.60 (normally called every 4 days)	0.25	0.150
Availability	1.00 (available morning, now is morning)	0.20	0.200
Priority Boost	0.00 (no boost)	0.10	0.000
Priority Tier	0.50 (Standard)	0.05	0.025
90-Day Transfer Volume	0.70 (above average transfers)	0.10	0.070
Total score		1.00	0.685

A score of 0.685 places this LO firmly in the middle of most daily lists. To rank near the top, an LO typically needs scores around 0.80 or higher, achieved by a combination of a long gap since last worked, high availability right now, and strong recent transfer volume.

Assignment to CLRs

Once every LO has a score, the generator assigns LOs to CLRs. The assignment algorithm takes all active CLRs (`is_clr = true`) and all scored LOs and produces a matching where each LO appears on exactly one CLR's list and each CLR receives a balanced number of LOs. The assignment step takes state licensing into account — an LO is only placed on a CLR's list if the CLR can productively work with that LO (historical working relationship is preferred where possible).

The matching algorithm is designed to produce a fair distribution of scores across CLRs. Two CLRs should never end the day with wildly different average scores on their lists — that would mean the algorithm gave one person a workload of 'easy, recent' LOs and another person a workload of 'stale, unavailable' LOs.

Once per day, locked after generation

The Calling List is generated once per day. Once it has been generated, it is locked: the assignments for that day cannot be regenerated without an admin override (Chapter 18). This lock is fundamental to the platform's promise of fairness. If the list could be regenerated freely, a CLR who did not like their list could simply regenerate until they got one they preferred, and the algorithm's guarantees would be meaningless.

The lock is implemented by a simple flag on the assignments table: when the generator completes, each day's row is marked with `locked = true`. A request to regenerate checks this flag first. If the flag is set, the request is rejected unless it is an admin override request.

Why past dates are blocked

The same lock applies to any past date. You cannot generate a Calling List for yesterday, for last week, or for any date in the past — not even as an admin with override rights. The platform refuses entirely. The reasoning is the same fairness guarantee, backdated: the point of the algorithm is to produce a list and have the team work it. A list generated for a date that has already passed is not a list, it is historical fiction. Blocking past-date generation prevents anyone from 'rewriting' what the list 'was' yesterday.

Past date block summary

Today's list: generated once, locked after, overridable by admin. Yesterday's list: historical fact, not editable, not regenerable, not overridable.

How to read your Calling List

The Calling List page shows each assigned LO as a row. Each row contains the LO's name, NMLS ID (useful for Bonzo lookups), phone number, state(s), availability window, and a visible score. The list is ordered top-to-bottom by score (highest first).

On each row, buttons let you log an appointment, view the LO's profile (which opens in a side panel so you do not lose your place), and mark the LO as 'worked' without creating an appointment (e.g., left a voicemail). The log-appointment flow is covered in Chapter 7.

A small status chip on each row tells you at a glance whether the LO is available right now, available later, or not available today. If an LO is on break, a longer chip explains when they return.

Working through your list during the day

Most CLRs work the list top-to-bottom but that is not required. The platform does not enforce an order. A CLR who has a strong relationship with an LO halfway down the list and wants to open the day with an easy win is free to do so.

Recommended rhythm for a new CLR:

1. Scan the whole list once. Note the LOs whose availability says 'available now' — those are the ones to call first.
2. Work those 'available now' LOs in score order, top to bottom. Log appointments or outcomes as you go. Do not batch the logging for later; log it in the moment.
3. At the midday transition, refresh the list. The availability chips will update: morning LOs will drop to 'not available again today' and afternoon LOs will light up.
4. Continue through the afternoon LOs in the same way.
5. Late in the day, review any LOs that you were not able to reach. These stay on your list — they do not disappear — and can be the first attempts of the next day if you can flag them as overdue (see Chapter 7).
6. Toward end of day, open the EOD Report flow. The transfer and appointment counts will already be populated from your logged activity. Add any narrative notes and submit.

What happens if you do not finish your list

There is no penalty for not finishing. The algorithm accounts for the reality that a CLR will not reach every LO every day. LOs you do not reach today will have their Days-Since-Last-Worked increase by one overnight, which moves them up in tomorrow's generation. The system is self-correcting.

What matters is that you log what you did — the conversations that happened, the voicemails you left, the appointments you booked. Unlogged work is invisible to the algorithm, which can make tomorrow's list less accurate.

The one sin

The one thing that genuinely breaks the system is working an LO and not logging the interaction. If you talked to someone, say so — the algorithm relies on your logs to compute Days Since Last Worked. An unlogged conversation effectively tells the algorithm 'this LO has not been contacted,' which will cause them to float to the top of someone's list unnecessarily tomorrow.

CHAPTER 7

Appointments

What counts, how to log, how to complete, and how appointments roll up into reports and team stats.

What counts as an appointment

An appointment in the CLR Connection Center is any scheduled future interaction between a Loan Officer and a lead that a CLR has arranged or confirmed. It does not matter whether the appointment is a phone call, a video call, a face-to-face meeting, or an email follow-up with a commitment to speak. What matters is that a specific LO has committed to engage a specific lead at a specific future time.

A few things that are not appointments: a voicemail you left (log it as a contact, not an appointment); an LO who said 'call me back next week' without committing to a lead; a generic follow-up reminder you set for yourself. These are real work, and some of them will feed into Team Stats under call volume, but they are not appointments.

Logging a new appointment

From the Calling List, click the **Log Appointment** button on the LO's row. Alternatively, from the LO's profile page, click **New Appointment**. Both flows open the same dialog.

The appointment dialog contains the following fields:

Field	Required?	Notes
Loan Officer	Yes	Pre-filled if opened from an LO row
Lead name	Yes	Name of the lead or client
Scheduled date	Yes	Date of the appointment
Scheduled time	Yes	Time of day (in the LO's local timezone)
Follow-up date	No	When to follow up; defaults to scheduled date
Notes	No	Free-text context for you and teammates
Source	No	How the lead came in (inbound call, referral, web form, etc.)

Appointment statuses

Every appointment has exactly one of three statuses at any time:

Scheduled.

The default status on creation. The appointment is in the future; no outcome has been recorded. Scheduled appointments appear on the Upcoming Schedule page and contribute to the Upcoming Appointments count on the Dashboard.

Transfer.

The appointment happened and produced a transfer — meaning the lead was successfully handed off and will progress with the LO. This is the positive outcome. Transfer counts feed directly into the EOD report and Team Stats.

Fell Through.

The appointment was attempted but did not produce a transfer. The lead did not show, the conversation did not convert, or the timing fell apart. This is the neutral outcome — it is not a mark against the CLR or the LO, it is a data point. Fell-through counts are tracked and reported, but they are not used against anyone in performance reviews.

Completing an appointment

When an appointment has occurred, the CLR who owns it (usually the one who created it) clicks **Complete** on the appointment's row. The Complete dialog opens, and the CLR chooses between Transfer and Fell Through. A brief note can be added. On save, the appointment's status changes, the timestamp is recorded, and the transfer or fell-through count is decremented on the Upcoming count and incremented on the appropriate daily metric.

You must choose an outcome

The Complete dialog does not let you leave the appointment in limbo. You must select Transfer or Fell Through. If neither is true yet — for example, the appointment is rescheduling — edit the appointment's date and leave the status as Scheduled instead of completing it. See 'Rescheduling appointments' below.

Overdue appointments

An appointment is overdue if its scheduled date has passed and it is still in Scheduled status. Overdue appointments are shown at the top of the Upcoming Schedule page with a visually distinct treatment (a red-bordered status chip), and they appear in a dedicated overdue section on the Calling List when they belong to you. They do not silently disappear — the platform surfaces them until they are resolved.

An overdue appointment must be resolved by one of three actions:

1. **Complete as Transfer.** The appointment happened, it went well, log the transfer.
2. **Complete as Fell Through.** The appointment did not produce a transfer. Log it.
3. **Reschedule.** Edit the appointment's scheduled date to a new future date. The status returns to Scheduled and it leaves the overdue section.

The platform does not let you delete an appointment as a way of making it go away. Deleting an appointment requires admin confirmation and is reserved for data entry errors. The normal resolution for any appointment — even one that was never going to happen — is to mark it Fell Through.

Upcoming appointments

Upcoming appointments — all Scheduled appointments with a future date — appear on the Upcoming Schedule page, ordered by scheduled time. CLRs see all their own upcoming appointments here; admins see the team's entire upcoming schedule with a CLR filter dropdown.

This page is the single pane of glass for 'what does the next week look like?' It is especially useful at shift handoffs and end-of-week planning. Filters let you narrow by CLR, by LO, by date range, and by status. The default view is 'my upcoming appointments, next 7 days.'

Rescheduling appointments

To reschedule an appointment, open it from any list (Calling List, Upcoming Schedule, or the LO profile) and click **Edit**. Change the scheduled date and time, optionally update the notes, and save. The appointment remains in the Scheduled state. If it was overdue, it is no longer overdue once the new date is in the future.

The platform records the reschedule as an activity on the appointment's history — every appointment carries an internal change log that shows when it was created, when it was edited, and who edited it. This history is visible on the appointment's detail panel.

How appointments feed into EOD reports

The EOD report for a day contains two auto-tallied numbers: transfers (the count of appointments this CLR completed as Transfer today) and appointments (the count of all appointments this CLR logged today, regardless of outcome). These numbers appear in the EOD form and cannot be edited manually — they come directly from the appointments table.

This is why logging appointments is operationally important. An unlogged appointment is invisible to the EOD tally, which means your report will understate what you actually did. Conversely, a mistakenly-logged appointment that you later delete will not appear in the report — but an incorrect appointment that you mark Fell Through instead of deleting will still count toward the 'appointments logged' total.

How appointments feed into Team Stats

Team Stats rolls up the per-CLR transfer and appointment counts over configurable time windows (today, this week, this month, all time). The data source is the same appointments table. An admin looking at Team Stats is looking at the aggregated reality of what the team logged; there is no separate 'stats' table that might drift out of sync.

Best practices for appointment management

A small amount of discipline around appointments pays off dramatically in the quality of the platform's outputs. The following practices are worth building as habits.

Log immediately, not at end of day.

When an appointment is booked, log it in the moment — ideally while you are still on the call with the LO. Batch-logging at day's end is a recipe for forgetting one or two, and the forgotten ones are the ones that break the EOD tally.

Complete on the day of the appointment.

When an appointment has happened, complete it that same day. Do not leave Scheduled appointments in your past. The overdue section exists as a safety net, not as a normal queue.

Use notes for context, not for the outcome.

The notes field is for context that helps you or a teammate pick the conversation back up next time — 'wife's birthday next week,' 'rate sensitive, quoted at 6.75,' 'prefers text messages after 3pm.' The outcome goes in the status, not the notes.

If in doubt, mark it Fell Through.

An appointment that was probably not going to produce a transfer should be completed as Fell Through, even if the conversation was cordial and there's a chance of future activity. If the lead re-engages later, create a new appointment; don't try to keep the old one alive.

Do not delete appointments except for data entry errors.

Delete is reserved for 'I typed the wrong LO' or 'I double-logged the same appointment.' Everything else — including appointments that were cancelled — should be completed as Fell Through. This preserves the audit trail and the historical record, and it keeps the counts honest.

The Call Reports page

Call Reports is a companion page to Appointments. It shows the per-call log — one row per recorded interaction with an LO, whether or not that interaction produced an appointment. A voicemail is a call report. A brief check-in with no appointment is a call report. A booked appointment is also a call report (and an appointment).

Call Reports is read-only for CLRs and displays your own activity. For admins, it displays the team's entire call log with filter controls. Call Reports is not part of the algorithm directly, but the call frequency factor (Chapter 6) is derived from it.

Admin edits to appointments

An admin can edit or delete any appointment on the platform. This is sometimes necessary for data cleanup — for example, when a departing CLR left a messy trail, or when a lead's name was misspelled across several records. Every admin edit is logged in the audit trail with the admin's identity, a timestamp, and a diff of what changed.

Admins should avoid editing appointments on behalf of a CLR as a routine matter. The CLR who owns the appointment is almost always the right person to complete or reschedule it. Admin edits exist for cases where the CLR is unavailable or the record is demonstrably wrong.

A short note on time zones

All appointments are stored with their time in the Loan Officer's stated local time. The platform does not currently convert across time zones for display — if you are a CLR in Pacific time working an LO in Eastern time, the appointment time shown is the LO's time. This is deliberate: the LO is the party who needs to show up at the right moment, and showing the LO's clock time reduces confusion in the CLR-LO conversation.

CHAPTER 8

EOD Reporting

The end-of-day summary that closes your shift and feeds management's view of the team's output.

What EOD Reporting is

The End-of-Day Report — EOD — is a short, structured summary that every CLR submits at the end of every working day. It is the single most important administrative artifact the platform produces, because it is the point at which the day's work becomes management-visible. Scott Petrie and Chris Redoble read EOD reports every working evening; trends across EODs drive operational decisions for the following week.

Why it matters

Before the Connection Center, EOD reports were freeform messages sent to a manager at inconsistent times. Some days they arrived at 5pm, some days at 11pm, and some days not at all. The numbers in them were inconsistent: one CLR counted 'appointments' as everything scheduled, another counted only appointments that had occurred that day. Managers could not aggregate across the team without chasing down what each number meant.

The Connection Center fixes this by making the EOD a structured form with a predictable schema: an auto-tallied transfer count, an auto-tallied appointment count, and one free-text narrative field. Every CLR submits the same three things every day. The narrative field is where CLRs add color — notes about specific wins, losses, process issues, or observations — but the structured fields are the ones that aggregate across reports.

When to submit

Submit your EOD at the end of every working day, before you sign off. Practically, this means between the last conversation of your shift and the moment you close the browser. The platform does not enforce a deadline, but the expectation is that every working day ends with an EOD submission.

If you did not work today — a day off, a sick day, a holiday — you do not submit an EOD. The platform does not ask for zero-reports. A missing EOD on a day you were expected to work, however, will be visible to management the next morning.

The EOD form fields

Transfers.

The count of appointments you completed as Transfer today. This field is auto-tallied from the appointments table. You cannot edit it directly. If the number is wrong, the fix is in the appointments data — edit or complete the underlying appointments, and the EOD will update automatically when the form reloads.

Appointments.

The count of appointments you logged today — regardless of outcome. Also auto-tallied. A brand-new appointment scheduled for three weeks from now counts toward today's appointments number. A completed appointment also counts. A deleted appointment does not.

"Any other notable work done today not mentioned above."

This is the free-text narrative field. It is the place to note anything that doesn't fit the structured numbers: a process issue you want management to know about, a relationship moment with a specific LO, a technical issue you hit, a lead source observation, a teammate-support event. It is explicitly optional — if your day was ordinary and the numbers speak for themselves, you can leave this field empty.

The narrative field has no character limit. CLRs typically write two or three sentences. Occasionally a CLR will write several paragraphs on a day that warrants it. Both are welcome. Management reads them.

What the narrative is not for

The narrative is not the place to dispute the structured numbers. If the transfer count is 3 and you believe it should be 4, find the missing transfer in the appointments table and complete it — do not write 'should be 4' in the narrative. The numbers are the canonical record of structured output; the narrative is the canonical record of unstructured context.

The EOD popup reminder

After the first login of a given working day, if you have not yet submitted today's EOD, the platform displays a popup modal reminder. The popup includes a direct link to the EOD form. Dismissing the popup does not dismiss the obligation — you can still submit the EOD later via the Reports menu — but the popup is the main mechanism that prevents CLRs from forgetting.

The popup is deliberately configured to appear **after the first login of the day**, not on the very first ever login. When a brand-new CLR logs in for their very first time, they see the welcome intro, not an EOD reminder — they haven't done any work yet, so there is nothing to report. The popup shows up starting the morning of their first real working day, and every morning after.

In pseudocode, the popup logic is:

```
if user.is_clr and user.hasSeenIntro and not submitted_eod_today(user):  
    show_eod_popup()
```

The three conditions together mean: the user is a working CLR, they are past their first-ever-login intro, and they have not yet submitted today's report. If any condition is false, the popup does not appear.

Admin-only users and the EOD popup

Admin-only users (`is_clr = false`) never see the EOD popup and are never expected to submit EOD reports. They are not working a Calling List, so there is nothing to report. The popup logic checks `is_clr` explicitly before appearing.

Submitting the report

To submit an EOD:

1. Open the EOD form. The easiest entry point is the popup's button, which appears on your first login of the day. Otherwise, navigate to **Reports → EOD Reports** and click **Submit Today's Report**.
2. Review the auto-tallied transfer count. If it looks wrong, cancel the form, fix the underlying appointments, and re-open the form. Do not attempt to submit with a number you believe is incorrect.
3. Review the auto-tallied appointment count similarly.
4. In the narrative field, write any notable context. If there is nothing notable, leave it blank.
5. Click **Submit**. The form submits to the server, which stores the report in the EOD reports table and dispatches an email to management.

Email delivery

On submit, the platform emails the EOD report to management. The email is sent from **reports@wlc.it.com** via the Resend API. The primary recipients are:

- **Scott Petrie** — West Capital Lending (primary)
- **Chris Redoble** — West Capital Lending

Additional recipients can be added by an admin in **Settings → Email & Notification Settings**. The sender address is configurable but defaults to **reports@wlc.it.com**, which is set up specifically for this purpose on the **wlc.it.com** domain.

The email contains the CLR's name, the date, the three fields from the report, and a link back to the CLR Connection Center for follow-up. It is formatted as a clean HTML email with a plain-text fallback.

Email is not SMTP

The platform does not send email via SMTP. It uses the Resend API, because Railway (our host) blocks outbound SMTP traffic on all plans. This means there is no 'email server' to configure and no SMTP credentials in the environment — there is a Resend API key, and that is all. Chapters 21 and 30 cover email infrastructure in detail.

Reading your sent EOD reports

Navigate to **Reports → EOD Reports**. The page shows a list of your submitted reports, most recent first. Click any report to see its full contents. Admins see every CLR's reports on the same page with a CLR filter.

Submitted reports cannot be edited. The EOD is intentionally append-only: once submitted, it is the canonical record of your day. If a report was genuinely submitted in error — for example, an accidental double-submit — an admin can delete it on your request.

Common mistakes and how to avoid them

A handful of mistakes come up repeatedly in training. Awareness of them is most of the battle.

Submitting without completing today's appointments.

If you have appointments from today still in Scheduled status, the transfer count will understate what you actually did. Complete the appointments first, then submit.

Trying to edit the auto-tally.

The number is not editable. If it is wrong, the fix is in the data, not the form. This confuses new CLRs because the number looks like a form field. It is not.

Submitting twice on the same day.

The platform prevents this — the submit button is disabled after a successful submission for the current day. If you need to 'resubmit' because the first submission was incomplete, ask an admin to delete the first one. Then the button will re-enable.

Writing a narrative that duplicates the numbers.

'I had 3 transfers and 7 appointments' is not a useful narrative — management can read those numbers directly from the structured fields. The narrative should say something the numbers cannot, like 'Bill Neessen had an unusually strong morning — three of those transfers came from his list in under an hour.'

What happens if you forget

A missed EOD is not a catastrophe. It is visible to management the next morning — they will see 'no report submitted' in their EOD digest. The CLR can still submit a late report for the previous day from the EOD Reports page (the submit button re-enables after the day rolls over). A late submission is better than a missing one.

Habitually missing EODs is a conversation with your manager, not a platform enforcement issue. The platform will not lock you out of the next day because you missed yesterday. The social contract around EOD submission is enforced by the team's culture, not by the software. The software only ensures that when you do submit, the submission is honest, structured, and visible.

The historical EOD archive

Every EOD submitted since the platform went live is retained indefinitely in the database. Admins can review any CLR's EOD history back to day one. This is useful for end-of-quarter reviews, for onboarding new managers, and for trend analysis of narrative content (for example: 'when did CLRs start mentioning Bonzo outages?').

The archive is searchable by CLR, date range, and keyword in the narrative field. Search is implemented as a simple SQLite LIKE query against the narrative column — fast enough for the volume of reports generated by a team of this size.

A closing note on EOD culture

The EOD report is the most visible artifact a CLR produces each day. Management's impression of a CLR's rhythm and contribution is shaped more by the consistency and quality of their EODs than by any single day's numbers. A CLR who submits every day, writes a thoughtful narrative when there is something to say, and leaves it blank when there is not, will be known as a reliable operator. The platform makes this easy — the form takes under sixty seconds to complete on a typical day. The rest is habit.

CHAPTER 9

Team Stats

Performance metrics for individuals and the team, derived from logged work.

What Team Stats is

Team Stats is the metrics hub of the platform. It aggregates the structured data produced by every CLR's daily work — transfers, appointments, call volume, EOD submissions — and presents it as a set of tables, charts, and filterable views. It is the page that tells you, at a glance, how the team is doing.

Team Stats is intentionally read-only. Nothing you do on the page affects any underlying data; the page is a view into the database, not a tool for editing it. This is consistent with the Dashboard's design philosophy: the metric pages tell you what is true, and the action pages let you change what is true. Never the two mixed.

What metrics are tracked

The standard Team Stats view surfaces the following metrics for every active CLR.

Metric	What it measures	Time windows
Transfers	Appointments completed as Transfer	Today / week / month / all
Appointments	Appointments logged, all outcomes	Today / week / month / all
Fell-Through	Appointments completed as Fell Through	Week / month / all
Call volume	Total logged interactions (Call Reports)	Day / week / month
EOD submission rate	Days submitted / working days in window	Week / month
List completion %	Percentage of recommended LOs this CLR worked	Week / month
Average daily transfers	Transfers ÷ days worked	Month / all
Average score worked	Mean score of LOs this CLR reached	Week / month

All of these are derived fields. There is no 'stats' table in the database that could drift out of sync with reality. Every number is re-computed at query time from the canonical appointments, call_reports, and eod_reports tables.

The CLR dropdown

At the top of the Team Stats page, a CLR dropdown lets you focus on a single person. Selecting a CLR from the dropdown filters every metric on the page to that individual's numbers only. Selecting 'All CLRs' (the default) shows team aggregates and a ranked table of every CLR's contribution.

The dropdown includes every user with `is_clr = true` — both plain CLRs and Admin-CLRs. Admin-only users do not appear, because they are not working a Calling List and therefore do not generate Team Stats data.

A CLR viewing Team Stats sees the dropdown and can filter to anyone, not only themselves. Team Stats is deliberately transparent: everyone can see everyone's numbers. The goal is to make the team's work legible to itself, not to create a private dashboard for management. A high-performing CLR's approach is visible to their peers; a CLR who is struggling can see who is doing well and ask how.

List completion percentage — a new 2026 metric

New in the 2026 edition: a metric called **list completion percentage**. This metric measures what fraction of the LOs on your daily Calling List you actually got through to — where 'got through to' means you logged either an appointment or a call report for that LO on that day.

If your list had 20 LOs and you reached 16 of them, your list completion for the day is 80%. The metric is then averaged over a configurable window (week or month) to give a rolling figure. A CLR with a consistently high completion percentage is reaching most of the LOs the algorithm recommended; a CLR with a consistently low completion percentage is leaving a meaningful portion of their list untouched.

A metric for self-coaching

List completion is explicitly designed as a self-coaching metric, not a punitive one. Many things can drive a low completion number — LO unavailability, a string of busy days, a CLR's own working style. The point is to make the pattern visible so it can be discussed. Management does not use this metric in isolation to make decisions.

Transfer counts in depth

Transfers are the most-watched metric on the page, because each transfer represents a conversion — a lead that progressed to the Loan Officer's pipeline. A CLR's transfer count is the primary input to their day-over-day performance view.

Team Stats surfaces transfer counts three ways: per CLR, per LO, and team-total. The per-LO view is especially useful for admins setting priority tiers or identifying LOs whose recent performance has shifted. An LO whose 90-day transfer volume has dropped sharply is worth a conversation about what changed.

Appointment counts

Appointment counts are the volume counterpart to transfer counts. A CLR who books many appointments but closes few transfers has a conversion issue; a CLR who books few appointments but closes a high fraction of them has a volume issue. Neither number is meaningful alone; together they tell a richer story.

The ratio is implicit in the page — Team Stats does not compute a 'conversion rate' as a separate metric, because the meaningful denominator depends on context. A conversion rate over a week of heavy Fell-Through days understates the CLR's skill; a conversion rate over a week of strong inbound pipeline overstates it. Raw numbers with thoughtful reading are more honest than a computed ratio.

Call volume

Call volume counts every interaction logged as a Call Report, whether or not it produced an appointment. A voicemail is a call. A brief check-in is a call. An appointment-producing conversation is a call (and an appointment). For CLRs whose style involves many light-touch check-ins, call volume can be significantly higher than appointment count.

Using Team Stats for self-improvement

Team Stats is most useful to individual CLRs when read as a weekly rhythm check. A recommended cadence:

1. Every Friday afternoon, before your EOD, open Team Stats filtered to yourself, monthly window.
2. Note three numbers: transfers, list completion percentage, and EOD submission rate.
3. Compare to last week. If something moved significantly in either direction, ask why. Write a sentence about it in your EOD narrative if the answer is interesting.
4. Once a month, look at the team ranking for the same three metrics. Not to compete — to notice patterns. If you are consistently at the top of list completion but near the bottom of transfers, what might that mean about how you are working the list?

How managers use Team Stats

Scott Petrie and Chris Redoble open Team Stats weekly. They look for trends, not for single-day fluctuations. A CLR whose numbers moved in an unexpected direction for one day is almost certainly having a bad Tuesday. A CLR whose numbers moved for three consecutive weeks is in a pattern that is worth understanding.

Managers also use Team Stats to identify coaching opportunities and to inform headcount decisions. If the team's average list completion is consistently under 70%, the algorithm may be generating lists that are too long for the current team size, and an admin should consider tuning the assignment matching to produce shorter lists or consider adding capacity.

What Team Stats does not show

Team Stats does not show every possible metric a manager might want. It does not break down transfers by state, by lead source, or by LO priority tier. It does not compute percentile rankings across CLRs. It does not produce downloadable CSVs (though this is planned). For any of those, the underlying data is available; it simply is not surfaced on this specific page. An admin who needs a custom cut can ask for a report to be added, or query the database directly.

Privacy and visibility

Team Stats is visible to every signed-in user. CLRs see each other's numbers, admins see everyone's numbers, and Admin-only users see the whole team. This is intentional: the metric is public within the team. If that visibility ever becomes a concern, the platform supports restricting Team Stats to admins only — but no team that uses the platform has requested this.

CHAPTER 10

Team Chat

The integrated group chat that replaces group texts for shift coordination.

What Team Chat is

Team Chat is the in-platform group messaging channel for the CLR team. It is one channel, shared by everyone, with a persistent message history. Messages are stored in the database, searchable, and visible to every signed-in user — CLRs, Admin-CLRs, and Admin-only users alike.

Team Chat replaces the informal group text thread or Slack channel that most CLR teams default to. The advantage of having it inside the platform is that every user who has access to the platform has access to chat automatically — no separate credentials, no 'is the new hire added to the thread yet?' Chat is part of the login.

Accessing Team Chat

Open the sidebar and click **Team Chat** under the Team group. The page loads with the most recent messages at the bottom, as in any modern chat UI. Scrolling up loads older history on demand.

On narrow screens, the sidebar collapses and Team Chat becomes a full-screen view. On desktop, Team Chat fills the main content area with the sidebar still visible. The input field for new messages is always pinned to the bottom of the chat area.

Sending messages

Type in the input field at the bottom of the chat area and press Enter to send. Shift-Enter inserts a line break without sending. There is no character limit. Basic formatting — bold, italic, links — is supported via standard Markdown shortcuts: wrap text in asterisks for bold, underscores for italic, and paste a URL for a clickable link.

Messages appear to every other signed-in user in near real time. There is no notification sound by default — the platform prefers quiet. Instead, the sidebar shows a small counter bubble next to the Team Chat item, which is the main signal that something new has arrived.

The notification bubble

The notification bubble is the small numeric badge that appears next to **Team Chat** in the sidebar menu when you have unread messages. It counts the number of messages sent to the channel since the last time you opened the chat page.

Opening Team Chat and scrolling to the bottom marks all messages as read; the bubble clears. Sending a message also marks all prior messages as read, on the reasonable assumption that you would not reply to something you had not seen.

The bubble is also visible on the sidebar's collapsed state on mobile — a small gold dot appears on the sidebar toggle icon itself, letting you know there is something worth opening even without expanding the menu.

Notification bubble rules

Count of unread messages since you last opened Team Chat. Opening the page clears the count. Sending a message also clears the count. The bubble is per-user — your bubble does not affect anyone else's.

Chat etiquette

Team Chat is a working channel, not a social one. A few norms keep it useful for everyone.

Use Team Chat for what the whole team needs to know.

'Bill Neessen just flagged that his phone is down for the next hour — use mobile' is a Team Chat message. 'Thanks for yesterday's help on that transfer' is better sent as a direct message (which the platform does not currently offer — send it as an email or a text). Team Chat should be read-worthy to the whole channel, not primarily to a single recipient.

Keep messages short.

A Team Chat message is typically one or two sentences. Longer context — a paragraph of analysis, a detailed question — is usually better as an EOD narrative, a direct email, or a conversation. Chat is for the thing that fits on one screen.

Assume a neutral read.

Tone is hard to convey in short messages. If something reads ambiguously, assume the charitable interpretation. If you wrote something that could be read uncharitably, consider editing or adding a clarifying follow-up.

Who can see chat messages

Every signed-in user can see every message. CLRs, Admin-CLRs, and Admin-only users all see the same channel. There are no private channels, no direct messages, and no role-restricted visibility. Team Chat is genuinely one conversation for the whole team.

Signed-out visitors cannot see anything — chat is behind authentication, like every other page. Former team members whose accounts have been deleted lose access immediately; their past messages remain in the history under their name but their ability to read or post new messages is gone on deletion.

Chat history and retention

Chat messages are retained indefinitely. There is no automatic purge. The messages table in the database is a simple append-only log with a timestamp, a sender `user_id`, and the message text. The retention policy is 'forever unless an admin explicitly decides to archive old messages,' and no admin has done so to date.

This means messages from a team member's first week on the job are still available years later. It is the single source of truth for 'what was that thing we talked about back in the spring' — a question that is surprisingly common.

Editing and deleting messages

At print, Team Chat does not support editing or deleting messages. This is intentional: the platform treats chat history as a record, not a draft. A typo is a typo; a send-to-the-wrong-person is a correction in a follow-up message, not a quiet edit. If a message genuinely needs to be removed — for example, a CLR accidentally pasted a lead's personal information into chat — an admin can delete it directly in the database with a supporting audit entry. This is a rare event.

Real-time updates

Team Chat updates in near real time via short polling — the client asks the server for new messages every few seconds. This is less elegant than a persistent WebSocket connection, but it is dramatically simpler to run on Railway's infrastructure and reliably good enough for the chat volume the team produces.

Practically, a message sent now appears on your teammates' screens within a few seconds. The notification bubble updates on the same cadence. There is no perceptible delay in typical use.

When chat is not the right tool

Chat is good for fast coordination. It is less good for deeper discussion, for questions that need a long reply, and for anything that should become part of a decision record. For those, prefer email (for asynchronous context) or a scheduled conversation (for back-and-forth).

A useful test: if you are about to write a Team Chat message longer than five lines, consider whether email or a meeting would serve the team better. Chat is most useful as short, frequent, low-friction messages — the opposite of a long essay.

A closing thought on chat as a coordination tool

The best teams we have seen use Team Chat sparingly. They send a handful of messages a day — a quick heads-up about an LO's availability, a question someone can answer in a sentence, a celebration of a big transfer. They do not treat it as a replacement for direct conversation or for structured reports. Chat is the thin layer that holds the team together between the heavier rhythms of EOD reports, Team Stats, and the Calling List itself.

PART THREE

LO Management

The roster, profiles, availability, archiving, and performance statistics.

The LO Roster

Every active Loan Officer, their profile fields, and the twelve WCL LOs at print.

Viewing the roster

Navigate to **LO Management** in the sidebar's Main group. The page opens on the Active tab, which lists every Loan Officer currently in rotation. A second tab, Archived, lists every LO who has been removed from rotation but whose history is preserved. A third tab, Stats, shows per-LO performance (covered in Chapter 15).

The Active tab renders as a searchable, sortable table. Each row is one LO. Columns include name, NMLS ID, phone, email, states licensed, availability window, priority tier, and a last-worked-at timestamp. Click any row to open the LO's full profile in a side panel without leaving the table view.

LO profile fields

Every Loan Officer in the roster has the following fields. The required fields are enforced on creation and on edit — they cannot be blank for an active LO.

Field	Required	What it is
Name	Yes	Full name, as the LO prefers it displayed
NMLS ID	Yes	The LO's Nationwide Mortgage Licensing System identifier
Phone	Yes	Primary contact number, any format
Email	Yes	Primary email address
States licensed	Yes	Comma-separated list of state abbreviations
Availability	Yes	Morning, afternoon, or both
Priority tier	Yes	Premium (1), Standard (2), Reduced (3)
Priority boost	No	Admin-applied temporary nudge flag
Break schedule	No	Optional unavailability periods (vacation, training)
Notes	No	Internal notes visible to CLRs and admins
isActive	system	Controlled by archive/restore; not manually set
isArchived	system	Inverse of isActive on the archived side of the tab

Understanding priority tiers

Priority tier is the structural signal a platform administrator uses to reflect an LO's standing. It is intentionally simple — three buckets, not a 1-to-100 number — because finer gradations invite debate without adding useful information.

Premium (tier 1).

Reserved for LOs who warrant consistently elevated attention. A Premium tier contribution to an LO's score nudges them forward in generation. In practice, this tier is used sparingly — for LOs whose relationship with the company is strategically important, or whose closing velocity justifies proactive contact.

Standard (tier 2).

The default tier for nearly every active LO. A Standard tier contribution is neutral — it neither adds nor subtracts from the LO's score relative to other factors. At print, all twelve active WCL Loan Officers are assigned Standard tier. The tier has not been used as a performance lever with this roster.

Reduced (tier 3).

For LOs whose relationship warrants reduced attention for a period. Reasons might include extended unavailability, a transition out of active status, or a specific business decision. A Reduced tier contribution lowers the LO's score relative to other factors. Reduced-tier LOs still appear in generation, just more rarely.

Tier vs. archive

Reduced tier is not the same as archiving. A Reduced-tier LO still appears in generation, at a lower frequency. An archived LO does not appear at all. If an LO is returning in three weeks, Reduced tier during the interim is often better than archiving and restoring, because it preserves continuity in the algorithm's memory.

The twelve active WCL Loan Officers

At print, the West Capital Lending roster contains twelve active Loan Officers. They are listed below with their names. Full profile data — NMLS IDs, phone numbers, states licensed — is maintained in the live platform, not in this manual, so that the printed document does not drift out of sync with operational reality.

#	Loan Officer	Priority Tier
1	Bill Neessen	Standard
2	Khashi Tabrizi	Standard
3	Sean Murphy	Standard
4	Dan Baker	Standard
5	Aaron Salazar	Standard
6	Ian Militello	Standard
7	Cole Fairon	Standard
8	James McGowan	Standard
9	Derek Bullen	Standard
10	Gary Dawson	Standard
11	Kurt Christman	Standard
12	Sean Ripperger	Standard

The twelve together are the population from which every daily Calling List is drawn. Any expansion of the roster — hiring an LO, onboarding a new relationship — adds them to this set. Any contraction — an LO leaving the relationship — archives them, which removes them from rotation while preserving their history.

Sorting and filtering the roster

The Active tab is sortable by any column header. The most common sorts are:

- **By last-worked date ascending** — to see which LOs have been neglected longest.
- **By NMLS ID** — useful when cross-referencing with Bonzo, which sorts by NMLS.
- **By priority tier** — to see Premium and Reduced-tier LOs together.
- **By state** — to see coverage gaps in your state footprint.

The top of the table offers quick filters: show only Premium, show only LOs missing required info, show only LOs with an active break scheduled in the next week, show only LOs with recent transfer activity. Filters compose — you can combine several at once. The filtered URL updates with the filter state, so you can bookmark a specific view.

The 'missing required info' warning

If an active LO has a blank required field — NMLS ID, phone, or email — their row is highlighted in the roster with a gold warning chip. The chip links to the LO's profile with the missing field scrolled into view. This is the platform's passive nudge to keep profiles complete.

Separately, the platform's notification system fires a broadcast notification to every user every three days for each LO with missing required info. The notification continues to fire until the field is populated. Chapter 12 covers the notification mechanism in detail, and Chapter 31 covers the technical implementation.

The three required fields are required for a reason

NMLS ID is required because it is how the LO is identified in Bonzo and in compliance filings. Phone is required so CLRs can actually call the LO. Email is required so the LO can be sent confirmations and communications. An LO missing any of these three is effectively un-workable, even if they have a name and states on their profile.

Searching the roster

A search bar at the top of the Active tab performs a case-insensitive substring match against name, NMLS ID, and email. Typing 'neessen' finds Bill; typing the first three digits of an NMLS ID finds every LO whose ID starts with those digits. The search runs client-side over the already-loaded roster, so it is instantaneous.

Search composes with filters. A search for 'ian' combined with the 'Premium tier' filter would show only Premium-tier LOs whose name, NMLS ID, or email contains 'ian'.

The LO profile side panel

Click any LO row to open their full profile in a side panel. The panel slides in from the right without leaving the roster table — you can close it and open the next LO without losing your place. The profile shows every field, including the ones not surfaced as table columns (break schedule, notes, recent activity).

From the panel, admins see **Edit** and **Archive** buttons. CLRs see the same profile as read-only — they can consume every field but cannot change anything. Both see a **Log Appointment** button, which opens the same appointment dialog as the Calling List.

Quick look at LO activity

The side panel includes a compact 'recent activity' section showing the last ten call reports and appointments for this LO across the team. This is often the fastest way to answer 'did anyone call this LO yesterday?' or 'what was the last thing we heard from them?' — the information is right there on the profile, no need to navigate elsewhere.

The Archived tab

The Archived tab mirrors the Active tab structure but lists LOs with `isArchived = true`. They are not in rotation. The tab exists so that an admin can see who has been archived, when, and why (if a reason was recorded). A **Restore** button on any archived row returns the LO to active status; the full restore workflow is covered in Chapter 13.

CHAPTER 12

Adding and Editing LOs

How admins add a new Loan Officer, what each field does, and the consequences of missing required info.

Adding a new Loan Officer

Adding an LO is an admin action (Admin-CLR or Admin-only). CLRs do not have access to the Add LO dialog. To add an LO:

1. Navigate to **LO Management**.
2. Click **Add LO** in the top right of the Active tab. The Add LO dialog opens.
3. Enter the LO's full name exactly as they prefer it displayed. The name is the primary display identifier across the platform.
4. Enter the NMLS ID. This field is validated as a numeric identifier; alphabetic characters are rejected. Double-check the value — it is the link between the LO in the Connection Center and the same LO in Bonzo.
5. Enter the phone number. Any format is accepted; the platform normalizes to E.164-style display but does not require you to enter it that way.
6. Enter the email address. The field is validated as a well-formed email on submit.
7. Select the states licensed. A multi-select dropdown lists all 50 states plus DC. You can type to search.
8. Select an availability window: morning, afternoon, or both. If the LO has atypical availability (only Tuesday mornings, for example), choose the closest approximation and use the notes field to clarify.
9. Select a priority tier. New LOs typically start at Standard. If there is a specific reason for Premium or Reduced, choose accordingly.
10. Optionally add notes. Notes are visible to all users and are a good place for context (communication preferences, personal details, working-style observations).
11. Click **Save**. The LO appears in the Active tab and becomes eligible for the next daily generation.

Why NMLS ID, phone, and email are mandatory

These three fields are enforced as required because they are each load-bearing for a different part of the platform's operation. Missing any one of them renders the LO partially or fully un-workable, regardless of what else is filled in.

NMLS ID.

The NMLS (Nationwide Mortgage Licensing System) identifier is the regulatory key for every licensed loan officer in the United States. Compliance workflows require it, Bonzo uses it as a primary reference, and every outbound communication that references the LO is expected to include it. An LO without an NMLS ID in the platform is an administrative impossibility.

Phone.

Phone is required because calling the LO is the primary mode of CLR-LO interaction. An LO without a phone number on file cannot be productively placed on a Calling List. Even if you intend to contact the LO by email, the phone field is required as a fallback that is assumed to always exist.

Email.

Email is required because it is how the LO receives platform-originated communications: appointment confirmations, break-schedule reminders, and (in future releases) their own performance summaries. An LO without an email address cannot receive any of these.

The every-3-day notification system

When any active LO is missing one or more of the three required fields, the platform's notification system fires a broadcast notification to every signed-in user every three days, listing the affected LOs and the missing fields. The broadcast continues until every required field is populated.

The mechanism works like this:

1. A nightly cron job runs at 3:00 AM UTC.
2. It queries for active, unarchived LOs with any of (NMLS ID, phone, email) null or blank.
3. If the set is non-empty, it checks the last-notification timestamp stored on each LO. If the most recent notification for this LO was more than 72 hours ago (three days), a new notification is created in the notifications table with `user_id = NULL` (meaning 'broadcast to all users').
4. The notification appears in every user's notification bell the next time they refresh.
5. When any admin updates the LO's profile to populate the missing fields, the next cron pass does not create a new notification for this LO, and the cycle terminates.

Why every three days, not more often

Daily notifications for missing fields would become noise — CLRs would filter them out, and the signal value of the notification would be lost. Every three days is frequent enough to be visible and rare enough to retain attention. If a missing field persists for a week and a half, the system will have broadcast the notification three or four times, by which point the team is certain to be aware.

Editing an existing LO

To edit an LO, open their profile side panel from the Active or Archived tab and click **Edit**. The same dialog used for Add LO opens, with every field pre-populated with the current values.

Any field can be edited. Required-field validation applies on save — you cannot save a change that would leave a required field blank. If you attempt to clear an NMLS ID, phone, or email, the dialog refuses and highlights the field.

On save, the platform writes an entry to the audit log with the admin's identity, the timestamp, and a diff of what changed. If you edited an LO's phone number, the audit log records 'phone: 555-0100 → 555-0200' (with both values visible). This history is accessible in the History page (Chapter 24).

Editing during an active day

Edits take effect immediately but do not regenerate the current day's Calling List. If an LO's availability is changed from morning-only to afternoon-only at 10am, the LO remains on whichever CLR's list they were assigned to that morning; their availability chip updates, but the assignment itself is stable. The change takes effect in the next generation, which is tomorrow's.

This is an intentional consequence of the once-per-day lock. Changing an LO's availability mid-day must not reorder CLRs' lists — that would undermine the fairness guarantee. The change is queued for the next fresh generation.

Setting availability

Availability is a three-state field: morning, afternoon, or both. The platform does not currently support more granular schedules — no 'Tuesday mornings and Thursday afternoons.' This is a deliberate simplification. Field experience with earlier, more flexible scheduling showed that detail at that level was rarely accurate and frequently ignored.

If an LO has a schedule that doesn't fit morning/afternoon/both, choose the closest approximation and use the notes field to describe the actual pattern. CLRs reading the notes will know to check timing before calling.

The platform treats 'morning' as the local time block before approximately noon and 'afternoon' as the block from noon onward. The exact cutoff is configurable in Settings but defaults to noon. The availability factor (Chapter 6) uses this threshold when computing the per-day score.

Setting states licensed

A Loan Officer must be licensed in a state to work leads from that state. The states field on an LO's profile is a comma-separated list of state abbreviations — for example, CA, AZ, NV, TX. The UI renders this as a set of tag chips and offers a multi-select dropdown for editing.

The state lookup tool (accessible from the Tools menu) reads this field to answer 'which LOs are licensed in state X?' Keeping this field accurate directly benefits CLR efficiency when a lead comes in for a less-familiar state.

A note on licensing

The platform does not validate the LO's claimed state licensing against any external registry. It trusts the value set by the admin. If a licensing change occurs — a new state added or dropped — the admin must update the profile. The NMLS-based compliance check (Chapter 23) exists partly as a periodic reminder to re-verify these fields.

Priority tier assignment

At LO creation, the tier defaults to Standard. It is set via a dropdown. Changes to tier are simple admin edits and take effect in the next generation, like other profile edits.

When should an LO's tier change? A short decision framework:

Situation	Recommended tier
New LO, first week	Standard
Strategic partnership; wants consistent proactive contact	Premium
Strong recent performance; worth reinforcing	Premium
Quiet period; wants to reduce contact frequency	Reduced
Extended absence or transition out	Reduced, then Archived if it persists
LO has asked to 'call less'	Reduced; document the request in notes

Priority boost

Priority boost is distinct from priority tier. It is a short-term, binary flag that gives an LO an additional positive nudge in generation. It is toggled by the **Boost** button on the LO profile. When boosted, the LO appears with a visible 'Boost' chip in the roster.

Boost auto-expires after a configurable period (default 14 days) so that forgotten boosts do not compound. The expiration creates an audit log entry. Manual removal is also possible at any time.

A typical use of Boost: a new LO who has just joined the roster and needs a week of elevated contact to establish rapport. Boost them, mark them Standard tier, and let the boost expire naturally at the end of their onboarding window.

Profile notes field

Notes are free-text and visible to every user. Use them for anything that helps a CLR have a more useful conversation with this LO. Common uses:

- Communication preferences: 'prefers text before calling,' 'do not call before 9am local.'
- Personal context: 'wife recently had a baby; may be short on time.'
- Recent history: 'had three fell-through appointments in a row; approach gently.'
- Business context: 'primarily works VA loans; rate-sensitive leads preferred.'
- Process notes: 'has us on CC list for all his appointment confirmations via Bonzo.'

Notes should be actionable. A note like 'nice guy' is not useful; 'responsive on Wednesday afternoons' is. Treat notes as operational context, not personal commentary. If it would embarrass anyone to see it in an audit, it does not belong in notes.

Break schedules

The break schedule field captures planned periods of unavailability — vacations, training weeks, personal leave. It is stored as a list of date-range entries, each with an optional label.

When an LO is on break, the availability factor (Chapter 6) returns 0, meaning the LO does not contribute meaningfully to generation during that period. The LO still appears in the roster with a visible 'on break' chip, and their break end date is shown.

The LO Breaks page under Reports gathers every active break schedule across the roster into one view, useful for planning the team's coverage during a particular week.

What happens when an LO is missing required info

Summarizing the cause-and-effect: the platform shows a gold warning chip on the LO's roster row, the LO's profile scrolls the missing field into view when opened, and every three days the notification system broadcasts a reminder to every user. The LO remains active and eligible for generation throughout — the missing data does not prevent generation, it just prevents the LO from being productively worked.

The fix is always the same: an admin edits the LO's profile and fills in the missing field. The warning chip clears immediately; the three-day broadcast stops on the next cron pass.

CHAPTER 13

Archiving and Restoring LOs

How to remove an LO from rotation without losing their history, and how to bring them back.

The difference between archive and delete

Archive and delete are two very different operations. Confusing them leads to data loss. Understanding the distinction before performing either is essential for any admin.

	Archive	Delete
Appears in the roster	In the Archived tab only	Not at all
Appears in generation	No	No
Appears on old reports	Yes, with historical name	Name replaced with 'Deleted'
Associated appointments	Preserved	Preserved but orphaned
Call history	Preserved	Preserved
Can be restored	Yes, one click	No, permanent
Audit entry created	Yes	Yes
Confirmation required	Single dialog	Triple confirmation

The short version:

- **Archive** when the LO is no longer in active rotation but might ever be again, or when you want to preserve the ability to look them up by name in historical data.
- **Delete** only when the record was created in error — a typo, a duplicate, a test entry — and should not exist at all.

Default to archive

When in doubt, archive. Archiving is reversible with one click. Deletion is permanent. Almost every situation in which you consider deletion is actually an archival situation in disguise. True deletion should happen perhaps once or twice a year, for data-entry mistakes only.

Archiving an LO

To archive a Loan Officer:

1. Open **LO Management** and click the LO's row to open their profile.
2. Click the **Archive** button at the bottom of the side panel. Admins see this button; CLRs do not.
3. A confirmation dialog appears: 'Archive [LO name]? This removes them from the Calling List rotation. They can be restored at any time.'
4. Click **Confirm Archive**. The dialog closes.
5. The LO's `isArchived` flag is set to true and `isActive` is set to false. They disappear from the Active tab and appear in the Archived tab.
6. An audit entry is recorded: 'LO archived: [name] by [admin name] at [timestamp].'

What archiving does (and does not) do

Archiving removes the LO from the Active tab and from all future daily generation. Existing appointments — past, current, and future — remain attached to the LO; their history is not disturbed. The LO's name continues to appear on historical EOD reports, Call Reports, and Team Stats in its historical context.

If an archived LO has future appointments scheduled, those appointments remain in the calendar and CLRs can still complete them. The CLR who owns the appointment can mark it Transfer or Fell Through as usual. The archive status does not block completion of existing work.

What archiving does not do: it does not delete any data, it does not purge the LO from history, it does not prevent anyone from looking up the LO's past activity. The LO becomes quieter — they stop being generated onto new Calling Lists — but they do not disappear.

Restoring an archived LO

To restore an LO, open the **Archived** tab, click the row, and click the **Restore** button that appears where **Archive** was. A brief confirmation dialog appears; confirm it, and the LO returns to the Active tab. They are eligible for the next daily generation.

Restored LOs carry their existing profile data forward — NMLS ID, phone, email, states, availability, tier, notes. Nothing is reset by the archive/restore cycle. What changes is the isArchived flag; everything else is where the LO left it.

One subtle consequence: the LO's Days Since Last Worked value continues to tick during the archive period. If an LO was archived six weeks ago and is now restored, their Days-Since-Last-Worked is 42+. This typically means they appear near the top of the next generation's scores, which is usually desired — the team has not been in touch with them for a while, and an elevated score brings them back to attention promptly.

Restoring without wanting the top-of-list effect

If you are restoring an LO and do not want them to shoot to the top of generation tomorrow, consider setting them to Reduced tier for a few days after restoration. The reduced contribution balances against the high Days-Since-Last-Worked and produces a moderate re-entry. After a week of gentle contact, return them to Standard.

When to archive

Good reasons to archive:

- **LO has left the company.** They are no longer an active LO at WCL. Archive; preserve the history.
- **Extended unavailability.** An LO on a three-month leave is not productively worked during the leave. Archive them for the duration and restore on their return.
- **Relationship pause.** A strategic decision to stop contacting an LO for a defined period. Archive with a note explaining the reason.
- **LO has requested to stop.** An LO who asks not to be contacted via CLRs. Archive with a note.

When not to archive

A few situations where archiving is the wrong tool:

- **LO is temporarily hard to reach.** If it is a matter of days, not weeks, use the break schedule instead.
- **LO is underperforming.** Archiving is not a performance action. Reduced tier or a priority-boost-off is the correct tool.
- **LO's contact info is wrong.** Fix the contact info. Archiving to stop the every-3-day notification is treating the symptom, not the cause.

The delete function

Delete is a separate, rarely-used capability. It permanently removes the LO record from the database. The delete button is hidden by default and exposed only after the LO has been archived. It requires triple confirmation:

1. First confirmation: 'Delete [LO name] permanently? This cannot be undone.'
2. Second confirmation: a typed-word challenge. The admin must type the word DELETE in all capitals to proceed.
3. Third confirmation: a virtual signature. The admin types their own name to confirm they personally accept responsibility for the deletion.

All three are required. On completion, the LO is removed. An audit entry records the deletion with the admin's identity, the signature they typed, and the timestamp. The LO's name in historical reports is replaced with '[Deleted]' since the underlying record no longer exists.

The philosophy of triple confirmation

The triple confirmation exists not to make deletion hard, but to make deletion felt. An admin who is three clicks into a delete is an admin who has had three opportunities to realize they meant to archive instead. By the time all three are passed, the action is demonstrably intentional. No one has ever deleted an LO by accident under this system.

CHAPTER 14

LO Availability System

How availability is represented, how it affects scoring, and how breaks fit in.

Morning and afternoon

Every active LO has an availability setting: morning only, afternoon only, or both. This three-state dropdown is the atomic unit of availability in the platform. It represents the LO's general window — the block of the day in which they are typically willing to receive CLR contact.

'Morning' is defined as the period before a configurable cutoff — by default, noon local time. 'Afternoon' is everything from the cutoff to the end of the working day. 'Both' means the LO is available in either window.

How availability affects the algorithm

The availability factor contributes 20% of an LO's total score. The factor is computed per-LO per-generation as follows:

LO's setting	At generation time...	Factor value
Morning only	Now is morning	1.0
Morning only	Now is afternoon	0.2
Afternoon only	Now is morning	0.2
Afternoon only	Now is afternoon	1.0
Both	Any time	1.0
On break	Any time	0.0

The low-but-nonzero value when an LO is available 'later today' (0.2) reflects that they are not unreachable — just not ideal to call right now. Scoring them to zero would remove them from consideration entirely, which is too strong; a small positive value keeps them on the map.

When generation runs

Daily generation typically runs once per day, in the morning before the team's shift begins. Because the availability factor compares to the current time, generation at 8am sees 'morning' LOs as fully available (factor 1.0) and 'afternoon' LOs as partially available (factor 0.2). Generation at 2pm would see the inverse.

In practice, admins schedule generation for early morning so that the whole team's lists are produced before anyone starts working. This produces a consistent daily rhythm and favors morning-only LOs in the top of lists — which is the intended outcome, since those LOs have the shortest contact window in the day.

Mid-day availability changes

If an admin changes an LO's availability mid-day — for example, an LO who was set to 'both' asks to be contacted only in the afternoon going forward — the change takes effect on the next generation, not immediately. The current day's assignments do not re-sort. This is consistent with the lock-after-generation rule.

The LO's availability chip on the Calling List does update immediately to reflect the new status. CLRs working the list see the updated chip on refresh. But the ordering of LOs in anyone's list is not recomputed.

Setting partial-day availability

The platform does not support finer granularity than morning/afternoon/both. This is a deliberate simplification. In practice:

- An LO who is only available 9-10am and 2-3pm would be set to 'both' with a clarifying note.
- An LO who is unavailable on a specific day should have a break scheduled for that day.
- An LO on a Tuesday-only schedule would need a repeating break for every non-Tuesday, which the platform doesn't currently support as a recurring schedule. In practice, any LO with this kind of schedule is rare enough that admins manage it via ad-hoc break entries or rely on the notes field.

Scheduling LO breaks

An LO break is a dated period of unavailability. While a break is active, the availability factor returns 0.0 for that LO, effectively removing them from generation. Breaks are stored as a list on the LO's profile, each entry having a start date, an end date, and an optional label.

To schedule a break:

1. Open the LO's profile and click **Edit**.
2. Scroll to the Break Schedule section and click **Add Break**.
3. Enter the start date (the first day the LO is unavailable).
4. Enter the end date (the last day of unavailability; they return on the following day).
5. Optionally enter a label: 'Vacation,' 'Training,' 'Personal leave.' The label appears on the LO Breaks page and on the LO's roster chip during the break.
6. Save.

A break can be edited or deleted at any time. Deleting a break that is currently active restores the LO to normal availability immediately (effective in the next generation). Editing the end date of an active break extends or shortens it.

The LO Breaks page

Navigate to **Reports → LO Breaks** to see every active and upcoming break across the roster, sorted by start date. This page is the single view of planned unavailability — useful for planning team coverage and for spotting clashes ('three LOs are out the week of the 15th; can we shift anything?').

A break is a positive signal, not a negative one

A break on an LO's profile means the team knows the LO is unavailable and won't be called during that period. No break on the profile means the team doesn't know, and the LO may end up on a list they cannot respond to. If an LO tells a CLR they will be out next week, that information should end up on the LO's profile as a break — not just in the CLR's memory.

How availability changes affect daily generation

Summarizing the behavior:

1. Changes to morning/afternoon/both take effect in the next generation. The current day's list is not re-sorted.
2. Adding a break that starts today and lasts one day takes effect in the next generation (tomorrow's). Today's list is not re-sorted.
3. Adding a break that starts tomorrow takes effect in tomorrow's generation.
4. Removing an active break restores the LO to full availability in the next generation.
5. The Availability Cutoff Time setting (Chapter 25) determines what 'morning' and 'afternoon' mean. Changing this setting takes effect immediately for display (chips update) but for the algorithm, it takes effect in the next generation.

The availability factor in code

For reference (Chapter 28 covers more), the availability factor is computed by a pure function that takes the LO's setting, the current timestamp, and the break list, and returns a value in the closed interval [0.0, 1.0]. The function does no database writes and has no side effects, which makes it trivially unit-testable.

A closing note on availability

Availability is the factor that ties the algorithm to the wall clock. It is the reason a morning-only LO who has not been called in two weeks will appear near the top of a morning list but nearly invisible on an afternoon list. It is the factor that keeps CLRs from calling a vacationing LO — the platform quietly drops them from the list. Kept accurate, it is one of the features CLRs notice least, because it simply makes the list look like a sensible place to start the day.

CHAPTER 15

LO Stats and Performance

Per-LO performance metrics, how they feed the algorithm, and how to use them.

The LO Stats tab

Within LO Management, the **Stats** tab shows one row per LO with a set of performance-oriented columns. It is the sibling of Team Stats — where Team Stats focuses on CLRs, LO Stats focuses on LOs. The data sources are the same underlying tables (appointments, call_reports), just aggregated by the other dimension.

Columns on the LO Stats tab

Column	What it shows
Name	LO name (links to profile)
Priority tier	Current tier chip
Last worked	Date and CLR of the most recent contact
Days since	Days since last worked — the input to Factor 1
Appointments (30d)	Count of appointments logged in the last 30 days
Transfers (30d)	Count of transfers in the last 30 days
Transfers (90d)	Count of transfers in the last 90 days — input to Factor 6
Fell-through (90d)	Count of fell-through appointments in the last 90 days
Conversion rate (90d)	Transfers / appointments over 90 days
Call count (30d)	Total call reports logged in the last 30 days
Avg call frequency	Rolling mean days between calls — input to Factor 2

Every column is sortable. The default sort is by priority tier (Premium first), then by Days Since (highest first), which surfaces the LOs most in need of attention.

90-day transfer volume tracking

The 90-day transfer volume is the numeric basis for the sixth algorithm factor (Chapter 6, Factor 6). It is computed as: count of appointments where this LO is the target, status is Transfer, and completion timestamp falls within the trailing 90 days ending today.

The window is rolling. An LO who had a strong month four months ago will see those transfers drop out of the 90-day window as they age out. Conversely, a recent burst of transfers will raise the 90-day count quickly. The rolling window is the reason the metric is responsive to current performance rather than historical reputation.

How LO stats feed into the algorithm

Three of the algorithm's six factors derive directly from LO stats:

- **Factor 1 (Days Since Last Worked).** The 'Days since' column is the input.
- **Factor 2 (Call Frequency).** The 'Avg call frequency' column is the input.
- **Factor 6 (90-Day Transfer Volume).** The 'Transfers (90d)' column is the input, normalized across the roster.

The other three factors (availability, priority boost, priority tier) come from the LO's profile fields directly, not from aggregated statistics. Only performance-driven factors depend on the stats tables; structural factors depend on the profile.

An LO whose 90-day transfer volume is in the top quartile of the roster gets a noticeable positive contribution from Factor 6. An LO in the bottom quartile gets a small negative contribution relative to the middle. Because Factor 6 is only 10% of the total, even a strong performer does not dominate a generation — they just get a small, consistent nudge forward.

Reading LO performance data

A few patterns that are worth recognizing at a glance.

Pattern 1: The neglected but productive LO.

High Days Since, high 90-day transfers, low 30-day calls. This LO has converted well recently but the team hasn't been in touch. They should be near the top of the next generation's list — and if they aren't, it's worth asking why (check for an archived state, a break, or an availability issue).

Pattern 2: The quiet LO.

Low calls, low appointments, low transfers — across a sustained window. This LO is on the roster but rarely produces activity. Worth a conversation about whether the relationship is still active or whether Reduced tier is appropriate.

Pattern 3: The high-volume, low-conversion LO.

Many appointments, few transfers. The leads are flowing but not closing. This might be a lead-quality issue, a pricing issue, or an LO-skill issue. The stats don't diagnose it; they flag it for a conversation.

Pattern 4: The suddenly cold LO.

30-day stats much lower than 90-day stats. Something has changed recently — an availability shift, a personal situation, a pricing decision in the market. Worth noticing before it becomes a sustained trend.

Stats describe, they do not prescribe

LO Stats surfaces signal, not verdict. A pattern in the data warrants a conversation — it does not warrant an automatic action. The platform deliberately does not auto-adjust priority tier based on stats, because the appropriate response to any pattern depends on context the platform does not have.

Using stats to make priority tier decisions

Priority tier changes are ideally informed by stats but not driven by them alone. A useful process:

1. Every quarter, open LO Stats and sort by 90-day transfers (descending). Note the top three and the bottom three.
2. For the top three: are any of them currently Standard tier? Would a Premium tier upgrade be appropriate? What is the case for or against?
3. For the bottom three: are any of them currently Premium tier? Is the Premium tier still warranted given the recent volume? Or would Standard be more accurate?
4. Bring a candidate list of tier changes to a team sync rather than applying them unilaterally. Tier changes affect every CLR's list; they deserve a team conversation.
5. Apply the agreed changes. The changes take effect in the next generation.

The relationship between LO Stats and Team Stats

LO Stats and Team Stats look at the same events from opposite angles. A transfer appears in both: as credit to the LO (increasing their 90-day transfer volume) and as credit to the CLR (increasing their transfer count in Team Stats). There is no double-counting; the underlying row is the same appointment record.

This symmetry is useful when investigating a discrepancy. If an admin sees a transfer in Team Stats but cannot find it in LO Stats, or vice versa, the appointment record is almost certainly the place to look — one of the two references (CLR or LO) may be incorrect on the record.

LO stats are available to all users

CLRs can read LO Stats. This is deliberate. A CLR preparing to work a specific LO benefits from seeing that LO's recent activity: 'I see Bill has had four transfers in the last 30 days; the team is clearly in rhythm with him.' LO Stats is a tool for conversation preparation, not only a management dashboard.

Admins have the same view as CLRs, plus the ability to act on it — editing priority tier, adjusting priority boost, scheduling breaks. The read/write split follows the same pattern as the rest of the platform.

Historical LO stats

The stats page shows 30-day and 90-day rolling windows by default. For longer historical views, the underlying tables retain all data indefinitely. An admin who needs 'transfers for this LO in calendar Q2 2025' can obtain that figure, but it is not a default column on this page. Future releases may add a custom date range selector; at print, ad-hoc questions like this are answered via a direct database query.

A closing note on using LO Stats well

The most common mistake with any stats page is reacting to a single number instead of a pattern. An LO with one transfer in the last 30 days might have had zero leads to work; an LO with twelve might have had forty. The numbers are informative only alongside their context. Use LO Stats to find the questions worth asking, then ask them — of the LO, of the CLR working them, of your own intuition about the relationship. The platform cannot replace that judgment; it can only make sure the numbers you are working from are honest.

PART FOUR

Assignment System in Depth

The algorithm, daily generation, and the admin override protocol.

How the Algorithm Works

The complete, non-technical explanation of the scoring, normalization, and assignment pipeline that produces every daily Calling List.

A complete mental model

The algorithm has three sequential phases. Each is simple on its own; together they produce the Calling Lists. Understanding them in order is the fastest way to reason about any specific behavior of the system.

Phase 1 — Score.

Every active, non-archived Loan Officer receives a single numeric score for today between 0.0 and 1.0. The score is a weighted sum of six factors. This phase ignores CLRs entirely — it just asks 'how much should this LO be called today?'

Phase 2 — Match.

The scored LOs are distributed across the CLR workforce. Each active CLR receives a balanced set of LOs to work. This phase considers state licensing, historical working relationships, and overall balance of score across CLRs.

Phase 3 — Lock.

The day's assignments are written to the database with `locked = true`. This makes them durable against accidents and turns tomorrow morning into a deliberate decision rather than a passive recomputation. Overriding the lock requires the explicit protocol described in Chapter 18.

The weighted scoring system

Scoring is the core of the algorithm. Mechanically, for each LO and for each of the six factors, a raw value between 0.0 and 1.0 is computed from database state. Each raw value is multiplied by a factor weight. The weighted contributions are summed. The result is the LO's score.

The six factors and their default weights, repeated here for reference:

Factor	Default Weight	Weighs...
Days Since Last Worked	0.30 (30%)	how recently anyone worked this LO
Call Frequency	0.25 (25%)	how often this LO is typically called, inverted
Availability	0.20 (20%)	whether this LO is reachable right now
Priority Boost	0.10 (10%)	short-term admin nudges
Priority Tier	0.05 (5%)	standing tier: Premium / Standard / Reduced

Factor	Default Weight	Weighs...
90-Day Transfer Volume	0.10 (10%)	recent conversion performance

The weights must sum to 1.0

Whatever values admins choose, the platform normalizes them to sum to exactly 1.0 before they are used in scoring. This guarantees that every LO's score is always in the [0.0, 1.0] range and that the relative influence of each factor is meaningful.

Step-by-step score calculation walkthrough

To make the machinery concrete, here is a complete walk-through of scoring a single hypothetical LO — call him **Dan Baker** — on a specific day. Dan is Standard tier, has no active priority boost, is available both morning and afternoon, was last worked 5 days ago (by Anna, another CLR), is normally called every 4 days on average, and has had 6 transfers in the last 90 days. Generation is running at 8:30 AM.

Factor 1: Days Since Last Worked.

The raw Days-Since value is 5. The algorithm normalizes this against a rolling target: the mean Days-Since across all active LOs is, say, 6.2 days today. Dan's value of 5 is below the mean — he has been called more recently than average. The normalized factor value is computed via a monotonic mapping that returns 0.5 for a value at the mean, 1.0 for a value far above the mean, and near 0 for a value far below. Dan's 5-vs-6.2 produces a value of approximately 0.44.

Weighted contribution: $0.44 \times 0.30 = \mathbf{0.132}$.

Factor 2: Call Frequency.

Dan's rolling 30-day call frequency is one call every 4 days. The algorithm maps infrequent-to-frequent to high-to-low factor values: an LO called every day gets close to 0, an LO called once a month gets close to 1. Dan's 'every 4 days' sits close to the team median and maps to roughly 0.55.

Weighted contribution: $0.55 \times 0.25 = \mathbf{0.138}$.

Factor 3: Availability.

Dan is set to 'both.' Generation is running at 8:30 AM, which is before the noon cutoff, so 'now is morning.' The availability factor for 'both' at any time returns 1.0.

Weighted contribution: $1.00 \times 0.20 = \mathbf{0.200}$.

Factor 4: Priority Boost.

Dan has no active boost. Factor value is 0.0.

Weighted contribution: $0.00 \times 0.10 = \mathbf{0.000}$.

Factor 5: Priority Tier.

Dan is Standard tier. Standard maps to the neutral mid-value, 0.5. Premium would map to 1.0 and Reduced to 0.0.

Weighted contribution: $0.50 \times 0.05 = \mathbf{0.025}$.

Factor 6: 90-Day Transfer Volume.

Dan has 6 transfers in the last 90 days. Across the whole roster, the mean is 4.2 and the max is 12. Dan's 6 maps to approximately 0.58 after normalization.

Weighted contribution: $0.58 \times 0.10 = \mathbf{0.058}$.

Total score: $0.132 + 0.138 + 0.200 + 0.000 + 0.025 + 0.058 = \mathbf{0.553}$.

A score of 0.553 is middle-of-the-pack. On a 20-LO day with the top LO scoring 0.82 and the bottom LO scoring 0.31, Dan would land somewhere around position 11 in the ranked list.

How normalization works

Two kinds of normalization happen in the algorithm. The first is weight normalization: the six admin-configurable weights are scaled to sum to exactly 1.0 before every scoring run. The second is factor normalization: each raw factor input is transformed into a 0.0-to-1.0 value before being multiplied by its weight.

Weight normalization.

If an admin sets the six weights to, say, 30/25/20/10/5/15 — which sums to 1.05 — the platform divides each value by 1.05 before use. The resulting weights are approximately 0.286 / 0.238 / 0.190 / 0.095 / 0.048 / 0.143, summing to 1.000. The admin's intent (relative emphasis) is preserved; the mathematical invariant (weights sum to 1) is enforced.

On save in Algorithm Settings, the platform displays the normalized values next to the raw inputs so admins can see exactly what the system will use. This transparency prevents surprise: the settings page never lies about what is in effect.

Factor normalization.

Each factor has a bespoke mapping from its raw input to a 0-to-1 value. These mappings are intentionally monotonic — more of the underlying thing always produces a higher factor value, never a dip — and are designed to be insensitive to extreme outliers. A single LO with 50 days since last worked does not dominate the scoring; the factor saturates at a maximum value and does not continue to climb indefinitely.

Why each factor was chosen

Six is a deliberate number. Fewer factors made the algorithm feel thin; more factors made it feel like an unreadable black box. Six is the count at which the system can be explained to an intelligent non-technical reader in a single page — the table above — while producing meaningfully different lists day to day.

Days Since Last Worked (30%). The highest weight because neglect is the single largest failure mode of an unassisted CLR system. An LO who hasn't heard from anyone in two weeks is the most likely to quietly disengage. The algorithm's single strongest bias is against that outcome.

Call Frequency (25%). Recognizes that LOs have different natural cadences. An LO normally called daily and not called for three days is overdue; an LO normally called weekly and not called for three days is right on schedule. Treating these cases identically would misallocate attention.

Availability (20%). The factor that ties the list to the wall clock. Calling an LO when they are unreachable wastes CLR time; the availability factor discourages that without forbidding it outright.

Priority Boost (10%). A short-term admin lever for specific situations. Intentionally not a large weight — boost should be noticeable, not overwhelming.

Priority Tier (5%). The smallest weight because it is the most structural. Tier should shape the edges of generation, not dominate the middle.

90-Day Transfer Volume (10%). Recognizes performance without worshipping it. An LO converting well gets modest extra attention; an LO converting poorly does not fall off the map.

How the algorithm promotes fairness over time

The six-factor design produces several fairness properties that emerge automatically rather than being programmed as explicit rules.

Every active LO eventually surfaces.

Because Days Since Last Worked keeps growing until the LO is worked, an LO who has been ignored for a week will score progressively higher each day until they appear near the top of someone's list. No LO can be indefinitely overlooked.

Recent stars don't crowd out newcomers.

The performance factor is only 10%. An LO in the middle of a hot streak gets an advantage, but not enough of one to push a neglected LO off the top of the list. This keeps the algorithm honest — it rewards performance without monopolizing attention.

Structural decisions stay soft.

Priority Tier is only 5%. Even a Premium-tier LO does not automatically rank first; they just get a small consistent boost. An admin cannot accidentally guarantee that a given LO dominates everyone's list by setting them Premium.

The list changes every day.

Because Days-Since and availability are time-varying, even a roster of LOs with identical profiles would produce different rankings on different days. The system does not settle into a fixed hierarchy.

CLRs get comparable lists.

The Phase 2 matching stage explicitly aims for similar average-score loads across CLRs. No one ends up with only the easy LOs; no one ends up with only the difficult ones. The daily challenge is roughly distributed.

No special cases in the algorithm

There are no explicit 'fairness rules' in the code — no 'if an LO hasn't been worked in X days, force them to the top.' The fairness emerges from the factor weights and the matching step. This is a design virtue: fewer special cases mean fewer surprises, and the behavior is easier to predict because it is the outcome of a simple calculation, not a pile of conditional rules.

Edge case 1: a brand-new LO

A Loan Officer added to the roster for the first time has no history. Their Days-Since-Last-Worked is undefined (they have never been worked); their Call Frequency is undefined; their 90-day transfers are zero.

The algorithm handles this by treating new LOs as if their Days-Since were the maximum the saturation curve allows — which produces a high Factor 1 value. Combined with a neutral Factor 2 (no history, treated as 'unknown cadence'), the net effect is that new LOs score relatively high on their first day in rotation. This is the intended behavior: a new LO who has never been called should be called promptly.

Admins who want additional head-start for new LOs can apply Priority Boost, which adds up to another 10% of score. Combined with the new-LO treatment, a boosted new LO is nearly certain to appear on the first generation after being added.

Edge case 2: an LO returning from archive

An archived LO who is restored retains their full history. If they were archived three months ago, their Days-Since-Last-Worked reflects the full elapsed time. On restoration, they almost always appear near the top of the next generation.

This is usually desired — the team has not been in touch with them for a while — but occasionally an admin wants a softer re-entry. The recommended approach is to set the restored LO to Reduced tier for the first week and restore them to Standard after. This moderates the initial high score without requiring any algorithm changes.

Edge case 3: ties

Two LOs can end up with identical scores. Ties are broken deterministically by a secondary sort on LO name (alphabetical). This guarantees that a re-run of scoring on identical inputs produces identical output, which is a useful property for debugging and for admin confidence.

Edge case 4: an LO with zero active CLRs who can work them

If the Phase 2 matching cannot find a suitable CLR for an LO — for example, an LO licensed only in a state where no CLR has relevant context — the LO is logged to a fallback queue. An admin sees this as a notification and can manually assign the LO or adjust the CLR pool. This is rare in practice because the WCL roster and CLR team are structured to avoid such gaps, but the algorithm has a graceful failure mode for the case.

Edge case 5: all CLRs unavailable

If no CLRs are active on a given day — a holiday, a global team outage — generation still runs but produces no assignments. An audit entry is recorded noting the empty run. The next working day, generation runs normally, and the previous day's LOs are treated as 'not worked yesterday' (their Days-Since continues to tick).

Edge case 6: weights sum to something absurd

If an admin somehow sets all six weights to zero, normalization divides by zero, which is mathematically undefined. The platform detects this case and falls back to the defaults. A notification is shown to the admin explaining that the all-zero configuration was rejected.

Similarly, if the stored weights sum to more than 1.05 — which can happen after certain migrations or manual database edits — a startup routine renormalizes the stored values and writes an audit entry. Chapter 28 covers this migration in detail.

Edge case 7: an LO on break during generation

The availability factor returns 0.0 for an LO on an active break, which cuts 20% of their potential score. The other factors still contribute — an on-break LO with a long Days-Since and strong 90-day performance still has a nonzero total score. However, because Phase 2 matching strongly avoids scheduling unreachable LOs to any CLR, a break-bound LO is typically deferred entirely until the break ends.

The three phases together, in pseudocode

For reference:

```
def generate(date, now):
    if past(date): abort("past dates are read-only")
    if already_generated(date) and not override: abort("locked")
    los = active_unarchived_los()
    w = normalize_weights(load_weights())
    scores = {lo.id: compute_score(lo, w, now) for lo in los}
    clrs = active_clrs()
    assignments = match(scores, clrs)
    save_locked(date, assignments)
    audit_log("generated", date, who=system_or_admin)
    return assignments
```

Every line of this pseudocode corresponds to behavior you can observe in the running system. 'abort("locked")' is the explanation for the error an admin sees when they try to re-generate. 'abort("past dates")' is why you cannot re-generate yesterday. 'save_locked' is why today's list is durable after it is produced.

What the algorithm does not do

A list of things the generator intentionally does not attempt, because they would complicate it without making it better.

- **It does not predict which LOs will transfer today.** It is not a forecasting model. It is a prioritizer, not a predictor.
- **It does not learn.** There is no machine learning, no model training, no parameter fitting. The six weights are explicit admin choices, not learned values. This keeps the system auditable — any score can be traced to a known arithmetic on known inputs.
- **It does not rebalance within a day.** Once the list is locked, it does not re-sort itself based on mid-day events. This property is essential to the fairness guarantee.
- **It does not consider CLR preferences.** CLRs cannot mark LOs as 'don't give me this one.' Every CLR is equally eligible to work every LO (subject to state licensing). Individual affinity is not a factor.
- **It does not penalize fell-through appointments.** Fell-through is tracked, but it does not directly lower an LO's score. An LO who fell through three times this month is not punished by the algorithm; they may still rank high tomorrow.

Explainability

Every LO's score can be decomposed into its six components at any time. An admin who wants to understand why a specific LO was or was not near the top of a list can open the LO's profile and see each factor's value for today. This is a planned feature rather than currently-shipping; at print the decomposition is available via a direct query but not in the UI. Chapter 20 covers the planned UI for this.

Summary: the algorithm in one paragraph

Every active LO gets a daily score from a weighted sum of six factors — recency, cadence, availability, boost, tier, and transfer performance. The six weights sum to 1.0 and are admin-configurable with defaults of 30/25/20/10/5/10. The scored LOs are matched to the active CLRs in a fair, score-balanced distribution. The day's assignments are locked to prevent re-rolling. No machine learning, no hidden state — just arithmetic on known inputs, reproducible and auditable.

CHAPTER 17

Daily Generation

How the platform produces the day's Calling Lists, when it happens, and why the result is locked.

The one-per-day rule

Daily generation happens exactly once per day. This rule is the load-bearing simplification that makes the rest of the platform's behavior intelligible. If generation could happen twice, CLRs would not know which list was canonical. If it could happen zero times, no one would know what to work on. Once per day is the answer.

Once generation has completed for a given date, the assignments for that date are locked — they cannot be re-generated without an explicit admin override (Chapter 18). The lock is implemented by a flag on the assignments table, checked by every generation request. If the flag is true and the request does not carry an override token, the request is refused with a clear error message.

How to trigger generation

Generation is an admin-only action. CLRs cannot trigger it. There are two ways it can happen:

Automatic, scheduled.

By default, the platform schedules generation to run once per morning (typically around 6:00 AM Pacific Time — configurable in Settings). The scheduler is a simple cron job inside the Express server. On schedule, it calls the same `generate()` function an admin would manually invoke. The resulting assignments are written to the database; an audit entry records the run; every CLR sees their new list on their first page load of the day.

Manual, by an admin.

Any admin can trigger generation from **Settings** → **Daily Generation**. A single **Generate Today's Lists** button is present. Clicking it invokes the same `generate()` function. If today's lists have already been generated, the button is replaced with a message explaining the lock and offering the override flow as a separate action.

Manual vs. scheduled

Manual generation is useful on days when the scheduled run did not happen for some reason (server restart at the wrong moment, timezone mismatch after a DST change, etc.). It is not a substitute for the scheduled run — it is a fallback. Ninety-nine percent of the time, the scheduled run produces the lists and no one has to think about it.

What happens after generation

A successful generation produces the following state changes, atomically within a database transaction:

1. Every active LO is scored (Chapter 16, Phase 1).
2. Scored LOs are matched to active CLRs (Phase 2).
3. One row per assignment is inserted into the **assignments** table, with today's date, the CLR's ID, the LO's ID, the score, and the rank (position in the CLR's list).
4. A single row is inserted into the **daily_generation_log** table, recording the date, the timestamp, the generating actor (system or admin), and the total count of assignments produced.
5. Every per-assignment row has its **locked** flag set to true.
6. An audit-log entry is written: 'generated Calling Lists for [date] — N assignments across M CLRs.'
7. The transaction commits. If anything above fails, the transaction rolls back and no partial state is left behind.

List locking in detail

The lock is a single boolean column on the assignments table. When the column is true, the assignment row cannot be modified or deleted without explicit override. Admins cannot drag-and-drop LOs between CLRs; CLRs cannot reject an LO off their list. The list is what the generator produced.

The lock applies to every row for a given date, not just one. The generator writes them all with locked=true in a single transaction, and the unlock operation (Override, Chapter 18) must unlock all rows for the date together. This prevents half-locked states in which some CLRs' lists are editable and others are not.

Why the lock matters culturally

The lock is the mechanism that enforces the fairness promise of the algorithm. If lists could be re-rolled freely, CLRs would naturally re-roll until they got a list they preferred — and any guarantee the algorithm made about balance, coverage, or recency would become a suggestion rather than a rule.

The lock also protects against accidents. Without it, a bug anywhere in the system that briefly triggered re-generation would wipe out the current day's assignments in progress. With it, re-generation is a deliberate choice that requires passing through the override protocol.

Past date blocking

The platform does not permit generation for any date in the past. This is distinct from the lock — the lock protects today's list from re-generation; past-date blocking prevents the creation of historical fiction. An admin cannot generate a list for yesterday, for last week, or for any date before today.

If an admin attempts to trigger generation for a past date (which requires specifically passing a date parameter to the underlying API — the UI does not expose past-date generation at all), the request is rejected with the error message 'cannot generate assignments for past dates.' The audit log records the attempt with the admin's identity and the rejected date.

Why past-date blocking is absolute

Unlike the lock on today's list, the past-date block cannot be overridden. Not even with the three-step override protocol. There is no admin action, no emergency procedure, and no configuration toggle that enables past-date generation. The rule is permanent.

The reasoning: the purpose of the algorithm is to produce a list and have the team work it. A list generated for a date that has already passed is not a list — no one is going to work it. It is an after-the-fact rewrite of what the team's day 'should have been.' Allowing that rewrite would corrupt the audit trail, contaminate the historical metrics in Team Stats, and invite revisionism about past work. The platform makes this impossible.

What to do if an admin 'needs' past-date generation

The answer is almost always: they don't. The past's assignments are what happened; the past's reports reflect what was done; changing the past does not improve any future decision. If a genuine data issue exists — a misattributed appointment, a wrong CLR on a completion — the fix is to edit the specific wrong record, not to re-generate the list that contained it.

How the system knows a day has already been generated

The check is simple: the generator queries the **daily_generation_log** table for a row matching today's date. If a row exists, the day has been generated and the lock is in effect. If not, the day is fresh and generation may proceed.

The `daily_generation_log` is distinct from the assignments table. Assignments store per-CLR-per-LO rows; the generation log stores one summary row per date. Separating them allows the generation check to be a fast, indexed lookup on a single column, rather than a potentially slow aggregation over thousands of assignment rows.

The generation log is never deleted. Every day that has ever had a generation run has a row in the log, even days far in the past. This is the canonical record of 'did the system generate lists on date X?' An admin investigating historical assignment problems can query this log directly.

What the generation log row contains

Column	Type	Purpose
date	DATE (unique)	Which date this generation produced lists for
generated_at	TIMESTAMP	When the run completed
generated_by	TEXT	'system' or the admin user's ID
assignment_count	INTEGER	Total assignments produced in this run
clr_count	INTEGER	Number of CLRs who received a list
lo_count	INTEGER	Number of LOs placed on at least one list
overridden_count	INTEGER	How many times this date has been overridden
last_override_at	TIMESTAMP NULL	Most recent override timestamp, or null
last_override_by	TEXT NULL	Admin who most recently overrode, or null

The audit trail entry created on generation

Separate from the generation log, every generation also writes a row to the **audit_log** table. The audit log is the general-purpose history of every administrative action in the platform; generation is one of those actions. Having both a specialized log and a general audit entry is a belt-and-braces design — the specialized log is efficient; the audit log is complete.

A typical generation audit entry reads:

```
2026-04-18 06:00:12 | GENERATE | actor: system | subject: assignments |
  details: { "date": "2026-04-18", "clrs": 9, "los": 12, "assignments": 12, "runtime_ms": 342
}
```

The audit entry is what appears in the History page (Chapter 24) for any user with history access. The details field is human-readable JSON; an admin reviewing yesterday's activity can see at a glance which runs succeeded, how long they took, and how many assignments they produced.

What if generation fails partway?

The transaction-wrapping described earlier means partial failures do not leave partial state. If the database connection drops after writing five assignments but before completing the run, the transaction rolls back and those five rows are removed. The generation_log entry is not written. No audit entry is written. From the platform's perspective, the failed run never happened — the lock is not set, and a retry is possible.

Operationally, a failed generation is rare. It has happened exactly once in the platform's history: during an unscheduled Railway deploy that coincided with the 6:00 AM cron. The failed run was detected because the expected row in daily_generation_log never appeared; an admin manually triggered a retry, which succeeded. The whole incident was resolved inside ten minutes.

Generation performance

For a roster of 12 LOs and a team of 9 CLRs, a generation run completes in well under a second. The runtime is dominated by a small number of SQL queries — one to load the active LO list, one to load the active CLR list, a raw-SQL aggregate query for the 90-day transfer volume, and the write of the assignment rows. Everything else is arithmetic in memory.

The platform is designed to scale comfortably to rosters of a few hundred LOs and CLR teams of a few dozen. SQLite (the current database) remains fast at that scale; the algorithm itself is linear in the product of LOs and CLRs, which is a tiny number for any plausible team.

If the team ever grows beyond the point where a single-machine SQLite instance can serve it, migration to PostgreSQL is a straightforward next step. The algorithm does not change; only the storage engine does. Chapter 26 discusses the upgrade path.

Generation and LO break schedules

An LO on an active break has a zero Availability factor value at generation time, which cuts 20% of their potential score. In practice this is usually enough to keep them off any CLR's list for the day — but not guaranteed, because the other five factors can still sum to a respectable total. Phase 2 matching strongly prefers to not assign unreachable LOs, so in almost all cases a break-bound LO is simply skipped from the day's matching.

Breaks that begin today take effect in today's generation only if the generation run happens after the break's start timestamp. Because generation typically runs at 6:00 AM, any break with a start date of 'today' will be in effect for the day's run.

Closing thoughts on daily generation

Daily generation is the single most important operational event in the platform's day. Every other feature — Calling Lists, Appointments, EOD Reports, Team Stats — depends on it having happened successfully that morning. The one-per-day rule, the lock, and the past-date block are the three mechanisms that keep generation a trustworthy foundation for everything above it.

CHAPTER 18

Admin Override

The three-confirmation protocol that allows an admin to regenerate today's locked Calling Lists, and the philosophy behind it.

When to use override

Admin Override is the mechanism by which today's locked Calling Lists can be regenerated. It exists for one reason: mistakes. The canonical case is a generation that ran with stale inputs — for example, generation fired at 6 AM and then an admin realized at 7 AM that three LOs had been accidentally left archived overnight, excluding them from the day's lists. Override corrects the run.

Override is not for preferences. An admin who does not like how the algorithm distributed a particular LO between two CLRs should not override — that is how the algorithm works, and overriding to 'fix' it undermines the fairness guarantee. Override is for data errors and operational emergencies only.

What override is not for

- **Not for CLR preferences.** A CLR who does not like an LO on their list may work the list in any order, skip LOs they cannot reach, and note the skip in their EOD. They may not request override to remove the LO.
- **Not for LO preferences.** An LO who calls and says 'I don't want to hear from you today' can be added to break for the day, which reduces their future appearance. Today's list is not retroactively amended.
- **Not for post-hoc balancing.** If it turns out that one CLR's list has more high-scoring LOs than another's, that imbalance is a fact about today. Override does not exist to rebalance the luck of a particular morning.
- **Not for performance tuning.** If the weights need adjustment, adjust them (Chapter 20). The change takes effect in tomorrow's generation. Override does not apply new weights to today's already-generated list.

A useful test

Ask: 'Would a future admin reading the audit log six months from now see the override reason and agree it was justified?' If yes, proceed. If the reason is 'didn't like the list,' don't override — the lock is doing its job.

Step-by-step override process

Override requires five explicit steps by an admin. Each step is a deliberate confirmation; no step is a throwaway click.

- 1. Navigate to Settings.** Open the sidebar and click Settings. The Algorithm Settings tab opens by default; switch to the **Daily Generation** tab.
- 2. Click "Override Today's Assignments."** This button is visually distinct from the regular Generate button — it is bordered in gold, labeled in orange, and positioned below the main content rather than in the primary action area. You cannot click it by accident.
- 3. Enter a reason (required).** A text area opens demanding a written explanation. The field is mandatory — the platform does not let you proceed with it blank. The reason must be at least 20 characters long. The reason is stored forever with the override audit entry.
- 4. First confirmation dialog.** 'Are you sure you want to override today's assignments? This will regenerate every CLR's Calling List. CLRs currently working will see their lists change.' Two buttons: Cancel and Confirm.
- 5. Second confirmation dialog.** 'This action is logged permanently in the audit trail. Your name and the reason you entered will be visible to all admins. Continue?' Two buttons: Cancel and I Understand.
- 6. Virtual signature / third confirmation.** A final dialog appears: 'Type your full name below to sign this override.' A text field expects the exact name matching your user profile. On match, the submit button enables. Click **Override and Regenerate**.

On the signature click, the platform performs the following atomically:

- Marks every existing assignment row for today as **overridden = true**, preserving the record of what was generated previously.
- Increments the **overridden_count** field on today's `daily_generation_log` row.
- Records the override in the audit log with the admin's identity, the reason, the signature, and the timestamp.
- Unsets the **locked** state for today.
- Invokes the generator, producing a new set of assignments for today.
- Re-locks the new assignments.
- Sends a broadcast notification to every user explaining that today's lists have been overridden.

What gets logged in the audit trail

The audit log entry for an override is one of the most detailed in the system. It records every element of the transaction, intentionally so that 'who, what, when, and why' is answerable in one query years later.

Audit field	Value for an override
action	OVERRIDE_GENERATION
actor_id	The admin's user ID
actor_name	The admin's display name
signature	The name typed at step 6
timestamp	Exact moment of the override commit
reason	The text the admin entered at step 3
subject_date	The date whose assignments were overridden (today)
prior_assignment_count	How many assignments existed before the override
new_assignment_count	How many assignments exist after the override
prior_generation_id	Reference to the original generation_log row
notification_sent	true — a broadcast notification was sent

All eleven fields are mandatory. The override cannot succeed if any field is empty. This guarantees that an override in the audit log can always be reconstructed completely, no detective work needed.

The philosophy of override

The override protocol is deliberately more painful than it needs to be. Three confirmations is more than almost any other destructive action in the platform. This is intentional, and the reasoning is worth articulating.

Override erases work that CLRs may be actively doing. If an admin overrides at 10 AM, every CLR who opened their list at 9 AM and has been working it for an hour suddenly has a different list. Whatever they had in progress — the context they built about today's top LO, the appointment they were about to log — is partially invalidated. Not fully (the appointment still logs correctly against the LO), but the CLR's cognitive thread is broken. That is a real cost.

The override protocol makes that cost visible. An admin who has just completed three confirmations and typed their own name understands that they have just disrupted the team's day. The protocol is not trying to prevent override — it is trying to make the decision to override felt, so that it is only taken when it is genuinely justified.

A secondary purpose: the protocol creates a reviewable audit trail. Every override produces a high-signal audit entry that any admin can review. If override frequency starts trending upward, that pattern itself is a signal — perhaps the algorithm needs tuning, perhaps the team's workflow has drifted, perhaps a specific admin needs a conversation. The protocol makes those patterns visible.

Override is rare by design

In a well-run WCL deployment, override is invoked perhaps once a month on average, and often goes weeks without use. If it is being used more often, the right response is usually not 'accept the frequency' but 'investigate why the algorithm or the inputs require so much correction.' The friction of the protocol is a feature, not a limitation.

Who can see override history

Override entries appear in the audit log, which is visible to every admin (Admin-CLR or Admin-only) via the History page. CLRs without admin rights do not see the audit log. However, when an override occurs, every user — including every CLR — sees the broadcast notification that explains 'today's assignments have been overridden by [admin name] at [time] for the following reason: [reason].'

This means every CLR knows when an override happens and why, in plain language, at the moment it happens. They do not have to ask. They do not have to read the audit log. The transparency is built into the override itself.

Override frequency metrics

The audit-log query page can be filtered to only OVERRIDE_GENERATION entries, producing a timeline of every override ever issued. This is the canonical report for 'how often are we overriding?' A rough healthy baseline is one override per month or fewer, with a distribution that is not concentrated in any one admin.

If a specific admin is issuing most of the overrides, that is a signal — perhaps their understanding of when to override differs from the team's norm. A team conversation about override criteria often resolves this.

Override and the locked past

Override only unlocks today's assignments. It does not unlock yesterday's, last week's, or any past date. Override cannot bypass past-date blocking; those two rules compose. An admin who needs to 'correct' yesterday's list cannot. The correct response to a past-date problem is to adjust the specific wrong record — edit the misattributed appointment, fix the misspelled name — not to re-generate the list.

A short note on override UX

The override flow is the platform's one intentionally-painful UX. Every other feature aims to reduce friction; override aims to increase it. This inversion is correct. Making the common thing easy and the dangerous thing deliberate is the pattern that keeps operational systems trustworthy over time.

PART FIVE

Admin Operations

Users, algorithm settings, email, credentials, compliance, history, settings.

Managing Users

Creating accounts, granting permissions, welcome emails, and password recovery.

User management is the act of deciding who can log into CLR Connection Center, what they are allowed to do once inside, and how their account is kept safe. Every person who uses the platform — every CLR, every admin, every observer — exists as a row in the **users** table, created and maintained through the Users page of the admin area. This chapter walks through the full lifecycle: adding a new user, choosing the right role, handling the welcome email, editing accounts, resetting passwords, and eventually archiving users who leave the team.

Where to find the Users page

From the main navigation, click **Admin**, then **Users**. The tab is visible to any user whose role is Admin-CLR or Admin-only. CLRs without admin rights do not see the Admin menu at all. If you do not see it, your account does not have admin privileges — ask a current admin to promote you, or to add the user for you.

The Users page layout

The page opens on a table of every active account. Columns include **Name**, **Email**, **Role**, **Is CLR**, **Last Login**, and **Status**. A search bar at the top filters by name or email. A **Show archived** toggle reveals deactivated users. A prominent **+ Add User** button in the top-right opens the creation modal.

Column	Meaning
Name	First and last name as entered at account creation
Email	Login identifier and primary contact; must be unique
Role	CLR, Admin-CLR, or Admin-only — governs what menus appear
Is CLR	Whether this user participates in the daily assignment list
Last Login	Timestamp of the user's most recent successful sign-in
Status	Active or Archived — archived rows are hidden by default

Adding a new user

Click **+ Add User**. A modal opens with fields for first name, last name, email, role, and the Is CLR toggle. Fill each field carefully — the email is especially important because it is both the login and the address where the welcome email is sent.

1. Type the user's **first name** and **last name**. These appear throughout the platform — in the LO roster, the appointment log, team chat, notifications — so enter them as the user wants to be addressed.
2. Enter the **email address**. This must be unique across the entire users table; attempting to re-use an email fails with a clear error message. Use the user's West Capital Lending email whenever possible.
3. Select the **role** from the dropdown: CLR, Admin-CLR, or Admin-only. See the next section for detail on what each role can do.
4. Decide whether **Is CLR** should be on or off. For CLR and Admin-CLR roles this is typically on; for Admin-only roles it is off. See the separate section below.
5. Click **Create User**. The platform generates a temporary password, writes the account to the database, and sends a welcome email via the Resend API.

The welcome email is automatic

As soon as you click Create User, a welcome email is dispatched from reports@wlc.it.com to the address you entered. The email contains the platform URL, the new user's email as the login, and a one-time temporary password. You do not need to separately notify the user — the email does it for you. However, do confirm with the user (by chat or voice) that the email arrived; occasionally a corporate spam filter intercepts it and a manual forward or resend is needed.

Role selection

The **role** field is the single most important choice in account creation. It determines which sections of the platform the user sees and which actions they can take. The three roles are deliberately distinct; a user is exactly one of them at a time. The table below summarizes the differences; Chapter 25 and Appendix C contain the complete permission matrix.

Role	Can call LOs	Sees admin menu	Typical user
CLR	Yes	No	Front-line CLR's who run the daily list
Admin-CLR	Yes	Yes	Team leads who also work the phones
Admin-only	No	Yes	Managers, analysts, or executives

The distinction between CLR and Admin-CLR is that Admin-CLR has all CLR capabilities plus the admin menu. An Admin-CLR can work the daily list exactly like a regular CLR, and they additionally have the ability to manage users, adjust weights, run overrides, and so on. The distinction between Admin-CLR and Admin-only is that Admin-only does not appear in the daily list and does not have a CLR dashboard; they only see administrative pages.

A useful mental model: role controls visibility of menus, while the Is CLR flag controls participation in the list. They are related but independent axes. Almost every real user falls on one of the three natural combinations above, but the two-axis design leaves room for edge cases like a temporarily paused CLR who still needs admin access.

The Is CLR toggle

The **Is CLR** field is a boolean that answers a single question: does this user take daily assignments? When true, the user's account is treated as an active CLR for algorithm purposes — the daily generation job considers them, the team stats include them, and they receive a personal calling list. When false, the user is treated as a purely administrative account; they never appear in assignment generation, never receive a list, and do not influence load-balancing.

Is CLR is NOT the same as the CLR role

The CLR role and the Is CLR flag are two different fields. The role is a string-valued enum that controls menu visibility; Is CLR is a boolean that controls list participation. A user with role Admin-only but Is CLR true is a contradiction the platform will not accept. A user with role Admin-CLR but Is CLR false is perfectly valid and represents a team lead who has temporarily stopped taking calls but still needs admin tools.

Toggling Is CLR mid-career

A user's Is CLR state can change over time, and the platform accommodates this gracefully. The most common transition is CLR-to-Admin-CLR promotion: a strong CLR is promoted to team lead, gets admin menu access, but continues to work the phones. Edit their account, switch the role dropdown to Admin-CLR, save. Nothing else changes — their assignment history, stats, and open appointments are preserved.

A rarer transition is Is CLR off: a CLR moves to a non-calling role (a manager position, for instance). In this case edit the account, set role to Admin-only or Admin-CLR as appropriate, and uncheck Is CLR. Starting the next day, the algorithm will not include them. Their historical data — past assignments, appointments logged, stats — remains intact and is still visible in reports.

Editing an existing user

Click any row in the user table to open the edit modal. The same fields appear as in the create modal, pre-filled with the user's current values. Edit whichever fields need to change and click **Save Changes**. Every edit is recorded in the audit log with the previous and new values, so there is a permanent record of who changed what.

A common early-mistake pattern is editing the email field. The email is the user's login, so changing it silently breaks their ability to sign in. If you must change an email (a marriage, a legal name change, a corporate domain migration), coordinate with the user — notify them, make the change, confirm they can log in with the new address.

Resetting a password

A user who has forgotten their password can use the **Forgot password?** link on the sign-in page; this dispatches a reset email. If that flow fails (spam filter, typo in the email field, account not yet verified), an admin can trigger a reset from the Users page. Open the user's record; click **Reset password**. The platform generates a new temporary password and sends a password-reset email via Resend, exactly like the welcome email but labelled as a reset.

Never set a password directly

The platform does not expose a field where an admin can type a new password on the user's behalf. This is deliberate — knowing another user's password is a security liability, not a convenience. Always use Reset password, which generates a random temporary and emails it to the user. The user then logs in and picks their own.

Archiving and restoring accounts

When a user leaves the company or changes roles such that they no longer need access, archive their account rather than deleting it. Open their record and click **Archive**. The row disappears from the default table view, the user cannot log in, the account no longer appears in the algorithm, and all historical data remains intact. Archival is the platform's equivalent of 'offboarding' for a user.

Archived users can be restored by toggling **Show archived**, clicking the row, and choosing **Restore**. A restored user's password is not automatically reset, so if they had forgotten it during their absence, use Reset password after restoring. Restoration is useful for employees who return from an extended leave, rotate back from another department, or who were archived in error.

Audit trail for user changes

Every user-management action writes to the audit log: create, edit (with before/after values for each changed field), role change, Is CLR toggle, password reset, archive, restore. The audit log is covered in detail in Chapter 24. When investigating 'who gave X admin access,' the audit log has the answer.

Typical user-management cadence

- **Weekly** — skim the users page for anyone who has not logged in for 14+ days; investigate whether their role has effectively ended.
- **On hire** — add the new user within 24 hours of their start date, confirm they received the welcome email, and watch their first-day login in the audit log.
- **On promotion** — update role and Is CLR to reflect the new position; notify the user that their menu options have changed so they know where to look.
- **On offboarding** — archive the same day as the user's last day; do not delete.

Worked examples — user management

Example 1: Onboarding a new CLR

A new hire, Maria García, starts on a Monday as a Connection Lead Representative. The team lead received her corporate email the prior Friday: maria.garcia@westcapitalending.com. She is expected to hit the phones on Tuesday morning. Here is the complete, unhurried onboarding sequence:

1. Friday afternoon, as soon as the email is assigned: open Admin → Users → + Add User. Enter first name Maria, last name García (note the accent — enter it correctly; the platform handles Unicode). Enter the email. Set role to CLR. Leave Is CLR checked. Click Create User.
2. The platform dispatches the welcome email instantly. Verify in the audit log that USER_CREATED fired, and that EMAIL_SEND_SUCCESS immediately followed.
3. Reach out to Maria by her corporate chat channel: 'Welcome email on the way — check your WCL inbox, and ping me here if it doesn't arrive in five minutes.'
4. If she reports the email arrived, she is done for Friday. If it did not arrive, check her spam folder before anything else (the single most common failure). If truly missing after five minutes, reset her password from the admin page — the reset email uses a different Resend template and sometimes threads the spam filter differently.
5. Monday morning, 9 AM: have Maria sign in. Confirm she can see her dashboard and her (initially empty) calling list. If you want her to shadow rather than call on day one, either leave the list visible but unassigned, or temporarily uncheck Is CLR for a day.
6. Tuesday 5 AM: the daily generation job includes Maria. When she signs in, she has a full calling list.

Total admin time for the above: ten minutes. The platform does the heavy lifting; most of the onboarding is logistics outside the system (computer, phone, desk).

Common first-week friction

New CLRs sometimes report that 'the platform feels complicated' in their first two days. This is almost always unfamiliarity, not complexity. Pair them with a second-week CLR for one morning, have them narrate as they work the list, and the complaint evaporates by mid-week.

Example 2: Promoting a CLR to Admin-CLR

Bill Neessen has been a strong CLR for nine months and is being promoted to team lead. He will continue to work the phones — the team is small enough that leads still call — but he also needs the admin menu for user management and weight tuning.

1. Open Admin → Users, click Bill's row.
2. Change role from CLR to Admin-CLR. Leave Is CLR checked.
3. Click Save Changes. The audit log records USER_EDITED with the role-change before/after values.
4. Message Bill: 'Promotion live — refresh your browser and you'll see the Admin menu. Walk me through anything you want a hand with.'
5. Over the next few days, pair with Bill the first time he does each admin task: adding a user, adjusting a weight (even trivially), running an override (at least as a rehearsal against a test scenario — not in production). The first real override is not the time to learn the confirmation flow.

Example 3: Temporary leave of absence

Kurt Christman is taking a two-week leave for surgery. He will be back; we do not want to archive him (which would drop all his historical data from live views) but we also do not want the algorithm to keep assigning him daily lists he cannot work.

1. Open his user record. Uncheck Is CLR. Leave role and all other fields unchanged. Save.
2. Starting tomorrow at 5 AM, the algorithm will not generate a list for Kurt. His historical stats remain intact; appointments he already logged remain in the database; team stats continue to include his historical performance.
3. Add a note in team chat: 'Kurt on leave through the 15th — his list is paused.' This is courteous but not strictly required; the platform will not accidentally route calls his way.
4. On his return date, open his record, re-check Is CLR, save. The next morning's generation includes him again.

This pattern — toggle Is CLR off, then on — is the correct way to handle any temporary absence: PTO longer than a week, medical leave, sabbatical. For shorter absences (a Friday off, a three-day weekend) simply mark the LO unavailable and let the algorithm route around; it is not worth a user edit.

Example 4: Offboarding a departing user

A CLR accepts a job elsewhere. Their last day is Friday. What should the admin workflow look like?

1. Friday during the day: no action yet. The user works their final shift normally.
2. Friday at end of shift, as they log their final EOD report: do not archive them yet. Let the report land.
3. Friday evening after 9 PM (after the EOD email has sent): open their record, click Archive.
4. The user's sessions are immediately invalidated; they cannot log back in Monday morning. Their data — appointments, EOD history, audit entries — remains in the database and in all historical reports.
5. If asked later (for a reference, a stats question, a compliance review), you can restore their account temporarily, pull the needed record, and archive again. This is rare; historical reports usually contain what is needed.

Example 5: Email change for an existing user

An employee gets married and their corporate email changes from jdoe to jsmith. The account must continue working.

1. Coordinate timing with the user. The change will log them out until they sign in with the new email.
2. At the coordinated time, open their record, edit the email field, Save.
3. Message the user: 'Email updated — sign in with the new address. Password is unchanged.'
4. If they report inability to sign in, they are typing the old email. After a brief 'oh right' moment, it works.
5. Verify in the audit log that USER_EDITED captured the old and new email values. This is your receipt that the change happened.

Email changes and the welcome flow

Changing a user's email does not trigger a new welcome email. The welcome is only sent on account creation. If you want to notify the user explicitly that their email was updated, do so manually in chat — the platform does not do it automatically, by design.

CHAPTER 20

Algorithm Settings

Six weight sliders, normalization, preview, and the responsible use of tuning.

The algorithm that produces each CLR's daily list is parameterized by six numerical weights. Those weights live on the **Algorithm Settings** page in the admin area, where any Admin-CLR or Admin-only can view and adjust them. This chapter explains what the page looks like, how the sliders behave, what the Save Weights action really does, how to use the preview, and — more importantly — the philosophy behind why and when to change weights. Chapter 16 covers the math of the algorithm; this chapter is the operational counterpart.

Opening the Algorithm Settings page

From the Admin menu, click **Algorithm Settings**. The page is dominated by six sliders, each labelled with the factor it controls and its current percentage. Above the sliders is a one-line summary: 'Weights must sum to 100%'; below them is a **Preview** panel that shows how today's list would change if the proposed weights were applied. Two buttons sit at the bottom: **Save Weights** (primary, gold) and **Reset to Defaults** (secondary, outlined).

The six sliders

Factor	Default	What it rewards
Days Since Last Call	30%	LOs who have been ignored the longest
Call Frequency	25%	LOs with a track record of consistent engagement
Availability	20%	LOs whose schedule and status allow calls now
Boost	10%	LOs a human has flagged as 'call these next'
Tier	5%	LOs whose business tier merits priority
90-Day Transfers	10%	LOs who have actively transferred business recently

Each slider ranges from 0% to 100% in whole-number increments. As you drag a slider, the others do not auto-adjust; you are free to set them independently and the page simply displays the running total. When the six values do not sum to 100%, the Save Weights button is disabled and a red message appears: 'Total is currently 95%. Weights must sum to 100%.' This forces every saved configuration to be a proper distribution.

How the Save Weights button works

Clicking **Save Weights** writes the six new values to the weights table in the database. It does not immediately regenerate today's list — today's assignments were computed at 5 AM with the old weights, and re-computing them at noon would be exactly the override flow, which is separately protected (see Chapter 18). Instead, the new weights take effect at the next daily generation — tomorrow morning at 5 AM.

Changing weights is a tomorrow-decision

If you change weights at 10 AM today, today's list is unaffected; tomorrow's list will use the new weights. This is almost always the right behavior — it gives the team a clean mental model ('today's list was computed with the weights that were active at 5 AM this morning'). If you genuinely need today's list to reflect new weights, save the weights and then run an admin override. That is the only path by which same-day weight changes propagate to the live list.

The Preview panel

Whenever the slider values have been changed from their saved state but not yet saved, the Preview panel re-computes the top ten LOs as they would rank under the proposed weights. This is a read-only simulation — no data is modified, and the CLRs' live lists are not touched — but it lets you see the likely effect of your change before committing. The preview uses the same scoring code that the real 5 AM job uses, so it is faithful.

The preview is most useful for catching unintended consequences. If you raise Boost from 10% to 30%, the preview will immediately show that today's boosted LO (if any) rockets to #1. If you drop Days Since Last Call to 5%, the preview may show that a fresh-but-inactive LO suddenly dominates. Glance at the preview for thirty seconds before hitting Save Weights; if the new top ten looks wrong, revise the sliders and re-check.

Reset to Defaults

Below Save Weights sits **Reset to Defaults**, an outlined gold button. Clicking it snaps every slider back to the launch values (30/25/20/10/5/10) but does not save. You still have to click Save Weights to commit. This two-step design prevents an accidental double-click from wiping an intentionally tuned configuration. If you change your mind after resetting, simply click Cancel or navigate away and your saved weights remain untouched.

Normalization — why weights are percentages

The six factors that feed the algorithm are measured in completely different units: Days Since Last Call is a whole number of days, Availability is a rolling percentage, Tier is an enum, and so on. To combine them into a single score, each raw value is normalized to a 0–1 range (where 1 means 'as favorable as possible for selection') and then multiplied by its weight. The weights are percentages specifically so that they compose cleanly: summing to 100% means every unit of 'score' corresponds to one percentage point of importance.

You never see the normalization step on the Algorithm Settings page — the platform does it automatically inside the scoring function — but understanding that it exists explains why the weights can be adjusted independently without re-balancing the underlying factor math. The normalization is the reason the sum must be 100%.

When to actually adjust weights

The defaults (30/25/20/10/5/10) were chosen after a month of pilot operation at WCL, and they work well for a team with roughly a dozen CLRs calling a roster of roughly a hundred LOs. Most teams should leave the weights alone. The most common reason to change weights is wrong — 'the algorithm gave me an LO I did not like this morning, so let me adjust' — and that is the exact impulse the preview and the audit log are designed to discourage.

Legitimate reasons to adjust

- The team has grown or shrunk significantly, and load-balancing dynamics have changed enough that the same weights no longer produce an even distribution.
- Business priorities have shifted — for example, a quarter focused specifically on re-engaging top-tier LOs might raise Tier to 15% temporarily.
- A new factor has been added in a platform update, and the defaults need to be re-balanced so the new factor has room without starving the existing ones.
- The team has reached consensus, reviewed in a meeting, that a sustained tilt (e.g., more weight on Boost because the team is using Boost discipline more effectively) is appropriate.

Illegitimate reasons

- 'I disagree with today's #1' — adjust the LO, not the weights.
- 'Boost is not catching my favorite LOs fast enough' — boost them.
- 'The algorithm is too random' — it is not random; it is producing the result its current weights produce. Study the score breakdown (Chapter 16) before tuning.
- 'One CLR keeps getting hard LOs' — that is a load-balancing or availability issue, usually resolved via LO availability schedules, not weight changes.

The audit log of weight changes

Every save of weights produces an audit entry of type **WEIGHTS_UPDATED**. The entry captures the six old values, the six new values, and the admin who made the change. Filtering the audit log to that type produces a timeline of every weight adjustment the platform has ever seen, which is invaluable when trying to understand why the list feels different today than it did last quarter.

Weight changes and team communication

Because weights change the feel of the daily list for every CLR, it is courteous (and usually wise) to announce a weight change in team chat before saving it. 'Heads up — raising Boost from 10% to 15% starting tomorrow to reflect our week's focus on the re-engagement list' gives the team context. The platform does not enforce this announcement, but it is part of the healthy operating rhythm.

Extreme-value safety

Setting a single weight to 100% and the others to 0% is permitted but produces a degenerate algorithm that ranks only by that one factor. This is occasionally useful for diagnostic purposes (for example, temporarily running at 100% Days Since Last Call to verify the staleness logic) but is not a valid steady-state configuration. If the Preview panel shows top-ten outputs that all tie on multiple factors, your weights are too concentrated.

Weight-tuning scenarios

Scenario A: The team has doubled in size

You started at six CLRs and now have twelve. The LO roster is unchanged at roughly a hundred LOs. The symptom you are seeing is that each CLR's daily list contains fewer LOs than before, and the same LOs are being called within days of each other because there are more CLRs competing for the same top-scoring LOs.

Diagnosis: the per-CLR assignment count was sized for a six-person team. The weights are not wrong; the daily-target parameter needs adjustment. But before touching parameters, try the weight route: raising the Days Since Last Call weight from 30% to 40% (compensating by lowering Frequency from 25% to 15%) will push the algorithm to spread calls more evenly across the roster, reducing the 'everyone calls the same LOs' effect.

Action: make the 30→40% shift, save, and watch the preview. If the top ten looks more diverse than before — different tiers, different age buckets — the adjustment is working. Monitor team stats for a week; if the diversity persists and no one complains about 'stale' LOs, the new weights are the new steady state.

Scenario B: Boost is being over-used

You notice from the audit log that Boost is being applied to 30% of the roster every week. At that rate, Boost effectively marks the default rather than the exception. The algorithm still works, but the signal from Boost has degraded — it no longer means 'call this one above the others' because everyone is boosted.

Diagnosis: this is a process problem, not an algorithm problem. The admins who are boosting are using it as a general emphasis tool rather than a scarce priority signal. The fix is a conversation, not a weight change.

If a conversation does not resolve it: lower the Boost weight temporarily from 10% to 3%. This reduces the impact of each boost, which paradoxically recalibrates admin behavior — boosting no longer feels like it 'does anything' at 30%-usage levels, so admins naturally boost less. Once usage drops below 10% of the roster, return the weight to 10%.

Scenario C: Tier should matter more

Leadership decides that Premium and Top tier LOs should get noticeably more calling attention than Standard tier. The current Tier weight of 5% gives only a modest preference — about one extra position in the daily ranking per tier level.

Diagnosis: 5% is indeed light; 15% would produce a visible preference without drowning the other factors. Going higher than 20% risks starving Standard-tier LOs of attention, which violates the 'everyone gets called' implicit contract of the platform.

Action plan: raise Tier from 5% to 15%. Make up the 10% by shaving 5% from Frequency (which is slightly over-weighted at 25%) and 5% from Days Since (slightly over at 30%). New configuration: 25/20/20/10/15/10. Preview shows Top-tier LOs dominating the top of the list; Premium sliding up a few positions; Standard still present but pushed down modestly.

Verify: after a week, pull the LO stats page and group by tier. Top-tier call frequency should be higher than it was; Standard-tier should still show a reasonable call cadence. If Standard-tier LOs are going 2+ weeks without calls, the shift went too far; revert partially.

Scenario D: The Availability signal is stale

CLRs report that the algorithm keeps routing them to LOs who are demonstrably unavailable when they call — phone goes straight to voicemail, email bounces because the LO is on leave, Bonzo shows 'out of office.'

Diagnosis: the LO availability calendar is not being kept current by the LOs themselves, so the Availability factor has low signal. Raising the Availability weight would make the problem worse, not better — it would give more weight to data that is demonstrably wrong.

Action plan: lower the Availability weight from 20% to 10% and reallocate the 10% to Days Since (up to 35%) and Frequency (up to 30%). Simultaneously, have the team lead reach out to LOs with stale availability calendars and either update them or mark the LO as requiring manual schedule checks. Once the data quality improves, return the Availability weight to 20%.

Algorithm tuning is a conversation

Weight changes that are not accompanied by communication with the team — CLRs and admins — feel mysterious and erode trust in the platform. A two-sentence announcement in team chat before every save ('shifting 5% from Frequency to Tier because leadership asked for more Premium focus this quarter') is part of healthy operation, not separate from it.

The 'do nothing' option

The most underrated option in algorithm tuning is to not tune at all. The defaults were chosen after real pilot operation at WCL; they are already tuned to this team's dynamics. Most concerns that appear to demand weight changes actually trace to data problems (stale availability, mis-tagged LOs, absent Boost discipline) that are better addressed at the source than compensated for in the algorithm.

Before changing weights, spend thirty minutes on the LO Roster and the audit log. Confirm that tier tags are accurate, availability is current, Boosts are recent and intentional, and that no archived LOs have snuck back into active rotation. Nine times out of ten the 'algorithm problem' resolves itself once the data is clean, and no weights need to move.

Weight-change checklist

1. State the problem in one sentence.
2. Confirm it is an algorithm problem, not a data problem.
3. Propose a specific weight change (exact percentages).
4. Check that the new weights sum to 100%.
5. Glance at the Preview panel; verify the top-ten shift looks reasonable.
6. Announce the change in team chat with a two-sentence rationale.
7. Save.
8. Mark a calendar reminder for one week out.
9. At one week, review team stats and LO call distribution.
10. Either keep the change, revert, or iterate.

CHAPTER 21

Email & Notification Settings

The Resend API, from-address configuration, EOD recipient lists, and notification policy.

Email is the platform's primary external communication channel. End-of-day reports, welcome emails, password resets, NMLS escalations, and admin alerts all travel over email. This chapter covers where the email configuration lives, how to rotate the Resend API key, how to update the from-address, how to manage the EOD recipient list (Scott Petrie, Chris Redoble, and anyone else), and the distinction between email and in-platform notifications.

Why Resend, not SMTP

CLR Connection Center runs on Railway, which blocks outbound SMTP on ports 25, 465, and 587 for abuse-prevention reasons. Direct SMTP delivery from the production container is therefore impossible. We use **Resend**, an HTTPS-based transactional email API, instead. Resend accepts our messages over a standard API call, queues them, and delivers them through its own infrastructure. For our purposes Resend is effectively a drop-in replacement for SMTP that happens to work on Railway.

Do not attempt to configure SMTP

Several well-meaning attempts to 'fix' email reliability by adding SMTP credentials have failed on this platform. The failure is not a misconfiguration; it is Railway's outbound port block. Resend is the production path. Any change to the email system should be a Resend change, not an SMTP change.

Where email settings live

Open the Admin menu and click **Email Settings**. The page has four sections: **Resend API Key**, **From Address**, **EOD Recipients**, and **Notification Defaults**. Each section has its own Save button so you can update one part without disturbing the others.

The Resend API Key

The Resend API key is the secret that authorizes our platform to send email. It is shown on this page as a masked field (only the last four characters visible). To rotate the key — which you should do at least once per year as a matter of hygiene — generate a new key in the Resend dashboard, paste it here, and click Save. The platform updates the environment variable and future email sends use the new key immediately. Old keys should be revoked from the Resend dashboard within 24 hours of rotation.

If the key is invalid or has been revoked, email delivery silently fails: the platform still queues the send, but Resend rejects it. The failure is visible in the notifications log and in the Resend dashboard. If EOD recipients report that they have stopped receiving reports, the Resend API key is the first thing to check.

The from address

The default from address is **reports@wlc.it.com**. This is a domain we control through GoDaddy, with SPF and DKIM records properly configured so that Resend deliveries authenticate as 'really from WCL.' Changing the from address is possible but requires matching DNS updates; a change without the DNS update produces rejected messages that never arrive. Do not change this field casually.

If you do need to change the from address — a rebrand, a new domain — coordinate with the IT team who owns GoDaddy. Update SPF, DKIM, and any DMARC policy first; verify the new domain in Resend; then change the field here and save. Send a test email to confirm delivery before trusting production traffic to the new address.

EOD Recipients

The EOD (end-of-day) recipients are the people who receive the summary email generated by the nightly job. By default this is **Scott Petrie** and **Chris Redoble**. Adding or removing a recipient is a matter of typing (or deleting) the address in the list and clicking Save Recipients. The next EOD run picks up the new list automatically.

Field	Purpose	Typical value
Name	Display name in the 'to' field	Scott Petrie
Email	Delivery address	scott@westcapitalending.com
Role	CC label used inside the email header	Executive
Include summary	Whether to receive the full table or just totals	Yes

At minimum keep two recipients so that a single person's inbox failure does not obscure the report from leadership entirely. A three-person list is common: two executives plus the operations lead. Resist adding long distribution lists — EOD reports contain per-LO detail that is not meant for general circulation.

In-platform notifications vs. email

The platform sends two distinct categories of communication: **notifications** appear as in-product banners (bell icon, unread count, dismissible cards) and are ephemeral; **emails** are external messages sent via Resend and are permanent. Some events produce both — an admin override, for example, sends a broadcast notification to every user and also emails the EOD recipients. Other events produce only one. The Notification Defaults section on the Email Settings page controls which events trigger which channel.

Notification defaults — the policy table

Event	In-platform	Email
Daily list generated	Yes (each CLR)	No
Admin override run	Broadcast to all	EOD recipients
EOD report completed	No	EOD recipients
NMLS check assigned	Yes (assignee)	Yes (assignee)
NMLS check 7-day escalation	Broadcast to admins	All admins
Weight change saved	No	No
User added	No	Welcome email (new user)
Password reset requested	No	Reset email (user)

Each row of this table is a toggle on the Notification Defaults section, so you can raise or lower the signal level for any event. The defaults shown are the recommended baseline; changes should be made sparingly and with team agreement. Over-notification is the fastest way to train users to ignore notifications entirely.

Testing email delivery

At the bottom of the Email Settings page is a **Send test email** button. Clicking it dispatches a minimal test message to the first EOD recipient. The button reports the Resend message ID and status in a small panel. Use this after rotating the API key, changing the from address, or editing the recipient list. If the button reports a failure, the platform has logged the error in the audit log and the Resend dashboard will have more detail.

Email failure recovery

If email delivery fails — because the API key was revoked, the from-domain DNS has lapsed, or Resend is experiencing an outage — the platform continues to function locally. In-platform notifications still fire. Data is still recorded. The EOD job still runs and produces a report; the report is simply not delivered. When email is restored, you can resend any missed report manually by opening that day's EOD record on the Reports page and clicking Resend.

Email deliverability deep dive

Why deliverability is a separate concern

Sending a message and having it arrive are two different operations, and the gap between them is the subject of an entire professional discipline called deliverability. An email that is sent successfully — meaning Resend's API returned a 200 status — can still fail to reach the recipient's inbox because of filters at the receiving email server, reputation signals against the from-domain, or content heuristics that flag the message as suspicious. This section walks through what we do to keep deliverability high, what we monitor, and what to do when a recipient reports that a message did not arrive.

SPF, DKIM, and DMARC

Three DNS-level mechanisms govern whether a receiving mail server trusts that a message claiming to be from **reports@wlc.it.com** is actually from us:

Mechanism	What it asserts	Our configuration
SPF	Which IPs are allowed to send	include:_spf.resend.com
DKIM	Cryptographic signature on each message	Two CNAMEs to Resend
DMARC	Policy for handling failures	p=none (monitoring mode)

SPF and DKIM are set up and working; we validate them periodically using tools like [dmarcian.com](#) or [mxtoolbox.com](#). DMARC is currently in monitoring mode (p=none), meaning we receive reports about spoofing attempts but do not instruct receivers to reject unauthenticated mail. Moving to p=quarantine or p=reject is a future improvement; we have not made the change yet because we want several months of monitoring data first.

Reputation and volume

Receiving mail servers build a reputation profile of each sending domain over time. High reputation — built through consistent volume, low bounce rates, low complaint rates — gets messages delivered to inboxes. Low reputation — spikes in volume, high bounces, many spam complaints — gets messages filtered to spam or rejected outright.

Our volume from wlc.it.com is modest and steady: a few dozen transactional messages per day. We do not send marketing email from this domain. We do not have a history of spam complaints. This is the sweet spot for transactional deliverability — enough volume to establish reputation, not so much that any one message creates noise.

Handling 'the email did not arrive' reports

1. Confirm the user's email address is typed correctly in their account record. This catches about 30% of reports: the address was wrong, the email went somewhere they do not check.
2. Have the user check Spam and Junk folders on every device they use. Corporate filters sometimes quarantine transactional messages the first time they arrive from a new domain.
3. Pull the message ID from the audit log for the most recent send to this address.
4. Open the Resend dashboard and paste the message ID. Resend shows the delivery status: Delivered, Bounced, Complained, or Deferred.
5. If Delivered: the recipient's email system accepted the message. The problem is downstream — usually a spam filter. Have the user whitelist reports@wlc.it.com.
6. If Bounced: read the bounce reason. 'No such user' means the address is wrong; correct it and retry. 'Mailbox full' means wait and retry.
7. If Complained: the user (or a previous owner of the address) marked us as spam. Resend will refuse to send to this address again. Coordinate with the user to un-mark and re-enable.
8. If Deferred: the receiving server is temporarily rejecting. Resend retries for 72 hours. Watch the status.

The unsubscribe question

Transactional emails — welcome, password reset, EOD report — are not subject to the same unsubscribe requirements as marketing email under CAN-SPAM. We do not include an unsubscribe link in these messages because the user is not a subscriber to a mailing list; they are an authenticated user of our platform receiving operational communication.

That said, a user can always stop receiving EOD reports by removing themselves from the recipient list on the Email Settings page (for EOD) or by archiving their account (for all mail). A user who asks for 'please stop emailing me' is best handled by having them or an admin perform one of these actions; we should not treat the email system itself as the mechanism for preference management.

Monitoring dashboard

The Resend dashboard provides a metrics view of our send volume, bounce rate, complaint rate, and delivery latency. Once a month, glance at the dashboard and confirm that bounce rate is below 2% and complaint rate is below 0.1%. These are industry-standard thresholds. Exceeding them is an early warning that deliverability will degrade if not addressed.

One weird rule

Never send email to an address you do not know for sure is valid and wanted. This sounds obvious but is the single most important deliverability rule. Every email we send is to an authenticated platform user or to a recipient explicitly added to a recipient list by an admin. We do not send to lists of any kind, we do not scrape emails from anywhere, and we do not re-use addresses across purposes. That discipline alone puts us well ahead of most senders.

CHAPTER 22

Bonzo Credentials

Storing, viewing, and rotating the CRM credentials the team shares.

West Capital Lending's CLR's interact with Bonzo, a customer relationship management tool, many times per day. Because access is shared across the team, storing the credentials somewhere convenient — but also secure — is a real operational need. CLR Connection Center provides a small, purpose-built **Bonzo Credentials** section for exactly this purpose. This chapter explains where it is, how to update it, and the responsibilities that come with editing shared credentials.

The Bonzo Credentials page

Open the Admin menu and click **Bonzo Credentials**. The page is intentionally small: three fields — username, password, and login URL — plus a clipboard icon next to each. Any Admin-CLR or Admin-only can view and edit this page. Regular CLR's do not see the menu entry; they retrieve credentials from their team lead on request, which is almost never needed because the team lead shares them via chat.

Field	What it is	How it is stored
Username	The shared Bonzo login email	Plain text in the database
Password	The shared Bonzo password	Plain text in the database
Login URL	The Bonzo sign-in URL	Plain text in the database

Why the password is stored in plain text

Unlike CLR account passwords, which are hashed and salted and irreversible, the shared Bonzo password is stored in plain text. This is deliberate and the reasoning is important. A hashed password cannot be displayed back to the user, because hashing is one-way. But the whole point of the Bonzo Credentials page is to show the password so it can be copied and pasted into the Bonzo login form. The page functions as a shared password vault, not an authentication system.

Security boundary

Plain-text storage is acceptable only because access to the page is gated behind admin login. The login itself uses strong password hashing. An attacker who has not compromised an admin account cannot reach this page. The risk model is: any admin can see the Bonzo password — which is true whether we store it hashed or in plain text — and no non-admin can. That boundary is the real protection.

Copy-to-clipboard

Each of the three fields has a small clipboard icon to the right. Clicking it copies the field's value to the system clipboard and flashes a brief 'Copied' confirmation. The password field displays as masked dots by default; clicking the clipboard icon copies the underlying value, not the dots. An **eye** icon, also to the right of the password field, toggles visibility of the masked value for visual confirmation. Most CLRs use the clipboard icon and never need the eye.

When to rotate the password

- Anyone with access to the Bonzo password leaves the team.
- A password leak elsewhere suggests the shared password may also be exposed.
- Quarterly rotation as standard hygiene, even without a specific trigger.
- Bonzo itself forces a password change during a security update.

How to rotate

1. Log into Bonzo directly (not through the CLR Center) and change the password using Bonzo's own account-settings page. Choose a strong random password, ideally from a password manager.
2. Return to the CLR Connection Center, open Admin → Bonzo Credentials, and paste the new password into the password field.
3. Click Save. The new value overwrites the old one in the database. The audit log records the rotation with the admin's name and timestamp but does not record the password values themselves.
4. Announce the rotation in team chat so CLRs who may have the old password cached in their browsers or password managers know to refresh from the platform.

Audit trail for credential changes

Edits to the Bonzo Credentials page produce an audit entry of type **BONZO_CREDENTIAL_UPDATED**. The entry records which admin edited the page, which fields changed, and the timestamp — but it does not record the new or old values. That is intentional: writing a password into the audit log would defeat the purpose of rotating it. The audit log tells you 'this password was rotated on Tuesday by Chris,' which is sufficient for governance without creating a new secrets-in-logs problem.

The small-feature philosophy

The Bonzo Credentials page is deliberately minimal. It has three fields, three clipboard icons, one eye toggle, one Save button. It resists the temptation to grow into a full password manager, an SSO integration, or a credential-sharing protocol with permission layers. Each of those would be valuable; none of them is needed at our team's scale. The page solves the real problem — 'we need the Bonzo password here, please' — without inventing bigger ones.

Credential lifecycle management

The Bonzo credential page is small, but the discipline around credential management is as important as any other piece of platform hygiene. This section expands on Chapter 22 with specific lifecycle scenarios, audit techniques, and the relationship between platform credentials and the broader team security posture.

The first rotation after launch

When CLR Center was first deployed, the Bonzo password stored in the platform was the same password the team had been using for several months. The first action after deployment was a rotation: generate a new password in Bonzo, update the CLR Center field, invalidate the old password. This rotation reset the security clock on the shared credential and gave us a known-good starting point.

Every subsequent rotation inherits the baseline established at that first one. Our running understanding is: the current password is known to admins via the CLR Center, known to Bonzo itself, and not known to anyone else (including any party who had the old password). This is the invariant we maintain through every rotation.

When a team member leaves

Any admin who leaves the team (voluntary or otherwise) has had access to the Bonzo password at some point. On their last day, rotate. This is a fixed rule, not a judgement call. It does not imply distrust of the departing admin; it closes off a risk vector that compounds across time otherwise.

Password strength guidance

When choosing a new Bonzo password during rotation, generate rather than compose. A random 20-character string from a password manager is strictly better than any human-composed password. The password is never typed by a human; it is always pasted from the CLR Center clipboard button. Length and randomness are free.

Length	Strength	Notes
8–10 chars	Too weak	Guessable in hours with modern hardware
12–14 chars	Acceptable minimum	Guessable in months; marginal for shared
16–20 chars	Strong	Recommended for shared credentials
20–30 chars	Very strong	What we actually use
30+ chars	Excessive but harmless	No practical downside; overkill

Clipboard hygiene

The clipboard is a briefly-shared resource on any computer. Pasting the Bonzo password into Bonzo is normal; leaving the password on the clipboard and then alt-tabbing into a chat application where you accidentally paste it is an incident. Good habits:

- Paste the password immediately after clicking the clipboard button in CLR Center.
- After the paste, copy anything else (a URL, a word) to overwrite the clipboard.
- Enable clipboard history managers with caution; they retain copied values including passwords.
- Never paste credentials into a chat, email, or document. If you need to share, direct the recipient to the Bonzo Credentials page in CLR Center.

Audit for credential access

The audit log does not record every view of the Bonzo Credentials page — that would be extremely noisy and not very useful. It records every change (BONZO_CREDENTIAL_UPDATED). For a view audit (who looked at the page in the last week?), the server's HTTP access log is the source, filtered to GET /admin/bonzo-credentials. This audit is rarely useful in practice but is available if a specific incident demands it.

CHAPTER 23

NMLS Compliance Checks

The two-month cadence, random CLR assignment, and the seven-day escalation path.

Every Loan Officer who works with West Capital Lending is licensed through the Nationwide Multistate Licensing System, known universally as NMLS. The license is a real legal prerequisite for originating loans, and letting it lapse is an expensive compliance incident. CLR Connection Center includes a lightweight but disciplined workflow for checking NMLS status on a regular cadence, distributing the checks across the CLR team, and escalating when a check is missed. This chapter covers the full workflow.

The two-month cadence

Every active LO is checked once every **two months**. That interval was chosen to balance two concerns: frequently enough that a lapsed license is caught within a few weeks, not months; infrequently enough that the total check volume is well within the CLR team's capacity. With ~100 LOs and a 60-day cycle, the platform generates roughly 50 checks per month, or two per business day — a very manageable load when distributed across a dozen CLRs.

The cadence is per-LO, not global. When an LO's check is completed, the next one for that LO is scheduled for 60 days later. New LOs are added to the cycle at the moment they are created. Archived LOs drop out of the cycle immediately. The scheduler always knows exactly which LOs are due today and which ones will be due each of the next 60 days.

Random CLR assignment

When a check comes due, the platform assigns it to a randomly selected active CLR. Randomness is constrained to maintain fairness: over any 30-day window, no CLR receives significantly more checks than the team average. The selection is made at the moment the check is due, not in advance, so CLR's on leave or archived between 'due date computed' and 'due date arrives' are excluded naturally.

The random-assignment design has two virtues. First, it distributes the compliance workload evenly — no one CLR ends up as 'the NMLS person.' Second, it builds team-wide familiarity with each LO. A CLR who performs a quarterly NMLS check on Bill Neessen learns something about Bill they might not otherwise know, and that knowledge becomes useful during calls.

What a CLR sees when assigned

When a check lands on a CLR, two things happen: (1) an in-platform notification appears in their bell menu, titled 'NMLS check due — [LO name],' and (2) a task card appears in their dashboard's tasks area. The task card shows the LO's name, NMLS ID, the due date (today), and a prominent **Start Check** button.

Step	What the CLR does
1. Open the NMLS website	https://www.nmlsconsumeraccess.org
2. Search by NMLS ID	Use the exact ID from the task card
3. Verify status	Active / Inactive / Suspended / Terminated
4. Verify sponsor	Must be West Capital Lending
5. Record the check	Fill the CLR Center NMLS form: status, notes, screenshot
6. Mark Complete	The task closes and the next check is scheduled for +60 days

Recording the check result

The in-platform form captures four pieces of information: the license **status** observed on NMLS (Active, Inactive, Suspended, Terminated), the **sponsor** shown (normally WCL), free-form **notes** for anything unusual, and an optional **screenshot upload** for documentation. Most checks take under two minutes. A typical completed record reads: 'Status: Active. Sponsor: West Capital Lending. Notes: no changes since prior check. Screenshot attached.'

The form intentionally does not pre-fill the status; the CLR must actively select it. This is a small anti-automation-bias measure — if the platform auto-selected Active, a busy CLR might click through without checking. Forcing the selection makes the check an active act.

Anomaly handling

If a CLR finds a non-Active status or a non-WCL sponsor, they select that status, write a brief note, and mark the check Complete. The act of completing with a non-Active status fires an immediate escalation: every admin receives a broadcast notification and an email, the LO is flagged in the roster, and the next check is scheduled for 7 days later instead of 60. The team has one week to resolve the license issue before the next check.

Escalation is immediate for non-Active

A CLR never needs to manually trigger an escalation for a license issue. Simply recording the true status — Inactive, Suspended, Terminated — is enough to alert the whole admin team within seconds. This is the correct default: compliance incidents should propagate instantly, not wait for a weekly review.

The 7-day escalation for missed checks

If a check is assigned but not completed within 7 days, the platform escalates. At midnight on day 7 a cron job scans every open NMLS task. Any task still open is re-assigned to a different randomly-selected CLR, and a broadcast notification goes to all admins: 'NMLS check for [LO] was assigned to [original CLR] on [date] and is still open. Re-assigned to [new CLR].' The original CLR is not penalized — people get busy — but the check does not quietly rot.

If the new assignee also misses the check, the cycle repeats at day 14, 21, and so on. Beyond 21 days, the escalation also emails the EOD recipient list so that leadership is aware. In practice the 7-day cycle catches nearly every delayed check before it becomes a second escalation.

The NMLS tasks page

The admin area has a dedicated **NMLS Tasks** page. It lists every open check, every check completed in the past 60 days, and the next-due date for every LO. Admins can manually re-assign a check, mark one complete on a CLR's behalf, or force-schedule a check for an LO who is overdue. The manual actions are audited. This page is the single source of truth for 'where are we on NMLS?'

The calendar view

A small calendar at the top of the NMLS Tasks page shows the count of checks due on each of the next 30 days. Days with unusually high counts are highlighted; if the platform notices that the 28th is going to have eight checks due, it will nudge two of them one day earlier or later to smooth the load. This load-smoothing is automatic and does not alter the 60-day cycle materially.

NMLS check walk-through

The CLR performing an NMLS check does a specific sequence of steps on the NMLS Consumer Access site. This section walks through the sequence in unusual detail, because the cost of doing it wrong — missing a suspended license — is high, and the step-by-step reinforcement is a small investment against that cost.

Step 1: Open the task card

From the dashboard, the NMLS task appears as a card titled 'NMLS check due — [LO first last].' The card lists the LO's NMLS ID, the due date (today), and the platform's record of their current sponsor (expected: West Capital Lending). Below these fields is a prominent Start Check button.

Before clicking Start Check, memorize or copy the NMLS ID. You will need it in the NMLS search form in a moment. The NMLS ID is a 5- to 8-digit number; LOs sometimes confuse it with their license number (which is state-specific). Use the NMLS ID.

Step 2: Open the NMLS Consumer Access site

In a new tab, navigate to <https://www.nmlsconsumeraccess.org>. The site is free, public, and does not require a login. It is the canonical source for NMLS license status — do not use third-party 'NMLS lookup' sites; they are often stale or misleading.

The landing page presents a search form with two options: search by name or search by NMLS ID. Always use NMLS ID. Names are ambiguous — two different licensed individuals can share a name — and searching by name will occasionally return the wrong person, which is exactly the failure mode we are trying to prevent.

Step 3: Examine the profile page

The NMLS ID search takes you directly to the LO's profile page. On this page you are looking for three specific pieces of information:

- 1. **License status:** should read Active. If Inactive, Suspended, or Terminated, this is an escalation.
- 2. **Employer / sponsor:** should read West Capital Lending (exact wording may vary: 'West Capital Lending,' 'West Capital Lending LLC,' 'WCL' — all acceptable). If the sponsor is a different company or blank, this is an escalation.
- 3. **State licenses:** the LO should be licensed in the states they operate in. A missing state is a minor issue (we note it but it is not an immediate escalation), since an LO's state list can change without affecting the top-level active status.

Step 4: Screenshot the profile

Take a screenshot of the full profile page. On macOS: Command-Shift-4. On Windows: Windows-Shift-S. The screenshot is your evidence that you actually visited the site and saw what you saw. Crop the screenshot to show just the profile (excluding unrelated browser chrome, other tabs, or personal Bookmarks bars) for cleanliness.

Step 5: Fill the NMLS form in CLR Center

Return to the CLR Center tab. Click Start Check. A form opens with four fields:

Field	What to enter
Status	Active / Inactive / Suspended / Terminated
Sponsor	West Capital Lending (or whatever was shown)
Notes	Free-form; note anything unusual
Screenshot	Upload the screenshot from step 4

If everything is normal (Active, WCL sponsor, no anomalies), a typical Notes entry reads 'No changes since previous check. State licensure unchanged.' Short notes are fine; there is no minimum length. Very short notes ('ok') are acceptable but discouraged — a future auditor will appreciate the extra sentence of context.

Step 6: Click Mark Complete

Clicking Mark Complete closes the task. Several things happen automatically in the next few seconds:

- The task card disappears from the dashboard.
- The NMLS tasks page shows the task as Completed with a timestamp.
- If the status was Active and sponsor was WCL, the next check for this LO is scheduled for 60 days out.
- If the status was non-Active or sponsor was non-WCL, the escalation flow fires: every admin receives a broadcast notification and an email, and the next check is scheduled for 7 days out.
- The audit log records NMLS_TASK_COMPLETED with the full form payload.

Anomaly scenarios

Most checks are Active/WCL and close uneventfully. The interesting cases are the anomalies:

Anomaly 1: Inactive status. The LO's license is Inactive, usually because they have not completed continuing education or they are between sponsors. Record accurately, complete the task, and expect the escalation email within seconds. The admin team will reach out to the LO directly.

Anomaly 2: Sponsor is different. The LO is sponsored by a different company. This means they have moved without notifying WCL, or that our record is wrong. Either way, record the sponsor as shown on NMLS, add notes ('Profile shows sponsor XYZ Mortgage as of today'), and complete. Admins will investigate.

Anomaly 3: LO does not appear in search. The NMLS ID returns no results. Double-check the ID (typos happen). If the ID is correct and returns nothing, the license may be fully terminated. Record status as 'Terminated,' notes as 'Profile not found at NMLS by ID,' and complete.

Anomaly 4: Multiple profiles match. This should not happen — NMLS IDs are unique — but on rare occasions the site appears to return a disambiguation list. Click through to the profile that matches the LO's name and continue. Note the anomaly in the form.

CHAPTER 24

History & Audit Log

What gets logged, how to search it, and how long it is kept.

The audit log is the platform's long memory. Every meaningful action that changes state — a user edit, a weight save, an override, an NMLS check, a credential rotation — produces an entry. The entries are immutable, searchable, and retained indefinitely. This chapter explains what is logged, how to find what you're looking for, and the conventions we follow when using the log for investigation.

Opening the History page

From the Admin menu click **History**. The page shows the most recent 100 audit entries, newest first, with columns for timestamp, actor, action, target, and a short summary. Filters at the top of the page allow you to narrow by date range, action type, actor, or target. A search box performs a full-text search across all audit entry summaries.

What gets logged

Action category	Example events
User management	USER_CREATED, USER_EDITED, USER_ARCHIVED, USER_RESTORED, PASSWORD_RESET
LO management	LO_CREATED, LO_EDITED, LO_ARCHIVED, LO_RESTORED, LO_AVAILABILITY_CHANGED
Assignments	DAILY_GENERATED, ASSIGNMENT_BOOSTED, ASSIGNMENT_DEPRIORITIZED
Appointments	APPOINTMENT_LOGGED, APPOINTMENT_EDITED, APPOINTMENT_DELETED
EOD reports	EOD_SUBMITTED, EOD_RESENT_BY_ADMIN
Algorithm	WEIGHTS_UPDATED, OVERRIDE_GENERATION
NMLS	NMLS_TASK_CREATED, NMLS_TASK_COMPLETED, NMLS_TASK_REASSIGNED
Configuration	EMAIL_SETTINGS_UPDATED, BONZO_CREDENTIAL_UPDATED
Authentication	LOGIN_SUCCESS, LOGIN_FAILURE, LOGOUT

Anatomy of an audit entry

Each entry has a consistent shape: a **timestamp** (UTC, precise to the millisecond), an **actor** (the user who performed the action, or 'system' for automated jobs), an **action** (one of the type tags from the table above), a **target** (the entity affected, if any), and a **details** blob (a JSON object with per-action payload). Clicking an entry row expands it to show the full details. Old values and new values for every changed field are present for edit-style actions.

Filtering and searching

The filter controls at the top of the History page are stackable. You can filter to 'all actions by Chris Redoble on April 15 affecting LO Dan Baker,' and the result is precisely that slice. The search box handles partial text — 'baker' matches any entry whose summary contains the string 'baker' case-insensitively — and is most useful when you don't know the exact action type but remember something about the target.

Common useful filters include: `WEIGHTS_UPDATED` to see every algorithm tuning ever performed; `OVERRIDE_GENERATION` to review every admin override; `PASSWORD_RESET` to see who has reset passwords recently; `LOGIN_FAILURE` filtered to a single account to detect brute-force attempts.

The audit log is not the debug log

The audit log records business-level events — actions a human took, or an automated job ran on their behalf. It does not record every HTTP request, every database query, or every minor internal state change. That kind of detail lives in the server logs on Railway, which are a separate system (covered in Chapter 32). Use the audit log for governance questions ('who did this?') and the server logs for debugging questions ('why did it fail?').

Retention

Audit entries are retained indefinitely. There is no automatic pruning, no 90-day rolling window, no archival. We have chosen this permanence deliberately: the log is small enough (thousands of entries per year, not millions), cheap to store, and genuinely useful for questions that arise months later. If the volume ever grows large enough to matter, we will add a tiered retention policy with cold storage for entries older than two years — but at current scale that engineering is not worth the complexity.

Immutability

Audit entries cannot be edited or deleted through the UI. The database does not expose a delete endpoint for the audit table, and no admin role has permission to modify past entries. The only way to alter the log is direct SQL access to the underlying database, which is restricted to the engineering team and is itself audited at the infrastructure level. In practice no one ever edits the audit log.

Investigating an incident

1. Identify the symptom: what is wrong, as reported by the user? Write it down.
2. Identify the affected entity: which LO, which user, which day? Set the target filter.
3. Identify the time window: when did the symptom appear vs. disappear? Set the date filter.
4. Scan the filtered entries for anything unusual — an unexpected actor, an edit no one remembers making, a failed login burst.
5. Expand suspicious entries to review before/after values. Compare to current state.
6. Document the finding — what you found, where, and at what timestamp. This becomes part of the incident post-mortem.

The audit log and privacy

Audit entries can include names, email addresses, and action context, but never include passwords, API keys, or session tokens. Values that could leak secrets are either redacted (for password changes, we log 'PASSWORD_RESET for user X by admin Y' with no password values) or omitted entirely. This policy is codified in the code that writes audit entries and reviewed whenever the schema changes.

Reading the audit log effectively

The audit log is powerful only to the extent that someone knows how to read it. Every admin should develop the habit of occasionally browsing the log — not to investigate a specific incident, but to internalize what normal operation looks like. Once you know what normal looks like, abnormal is obvious. This section expands on Chapter 24 with reading strategies and a few worked investigations.

Reading strategy 1: The daily five-minute scan

Once a day, preferably in the morning before the list is generated, open the History page and scan the last 24 hours of entries. You are looking for anything you did not expect: unexpected actors, unexpected targets, unusual volumes. Most days the scan takes two minutes and turns up nothing. Occasionally you will spot something worth following up on — a CLR who was archived by mistake, a weight change you did not know about, a surge of login failures suggesting a password guess attempt.

Five minutes a day of this habit is worth more than an hour of investigation when a problem surfaces. The goal is to catch drift early, before it compounds.

Reading strategy 2: Filter-driven investigation

When you know what you are looking for, use filters. Common filter patterns:

Goal	Filters
Find today's weight changes	action=WEIGHTS_UPDATED, date=today
Find overrides this month	action=OVERRIDE_GENERATION, date=last 30 days
See what Chris did yesterday	actor=chris@..., date=yesterday
Find all edits to LO 'Dan Baker'	target_type=lo, target_id=
See failed logins on one account	action=LOGIN_FAILURE, actor=
Find all password resets	action=PASSWORD_RESET, date=last 7 days

Worked investigation: the missing appointment

A CLR reports that an appointment they logged yesterday is not showing up today. Here is how an investigation plays out using the audit log:

1. Open History, filter to action=APPOINTMENT_LOGGED, actor=, date=yesterday. If a row appears, the appointment did get logged and the issue is a display bug; move on to browser cache investigation.
2. If no row appears, check action=APPOINTMENT_DELETED for the same actor and date. Sometimes a CLR accidentally deletes and does not remember.
3. If still no row, check all appointment actions for that CLR that day — perhaps the appointment was logged with a different outcome or against a different LO than the CLR remembers.
4. If the audit log truly has nothing, the appointment was never saved. This is rare but happens — a browser crash mid-submit, a network drop before the POST completed. Have the CLR re-log from their memory or Bonzo notes.

Worked investigation: the weight mystery

A CLR says 'the algorithm has been different since last week — everyone keeps getting the same boring LOs.' Here is the investigation path:

1. Open History, filter to action=WEIGHTS_UPDATED, date=last 14 days. This produces a timeline of every weight change.
2. Expand each entry to see before/after values. Look for a change that matches the CLR's perception — 'boring' often maps to a Tier weight increase or a Days Since Last Call decrease.
3. If you find such a change, note which admin made it and when. Either revert (if the change was unintentional) or communicate (if it was intentional but not announced).
4. If no weight change explains it, check action=OVERRIDE_GENERATION in the same window. A frequent override pattern can also shift the feel of the list.
5. If neither, the perception may not match reality. Pull a week of Team Stats and compare to the week before; if the data shows similar distributions, the CLR's impression is not well-supported.

Worked investigation: the role confusion

An admin reports 'I can't find the Admin menu anymore.' This can happen when their role has been changed (intentionally or not):

1. Open History, filter to `action=USER_EDITED`, `target_type=user`, `target_id=`.
2. Scan for any edit that changed the role field. The before/after values are in the details payload.
3. If you find such a change, coordinate: was it intentional? Who made it? Was the user informed? Revert if appropriate.
4. If you find no such change, the missing menu is a cache problem. Have the admin clear cookies and sign in again.

The audit log as accountability

Beyond investigations, the audit log serves as the platform's accountability layer. In a team where everyone has admin powers (or a subset of them), the log is how we know what each person has done. This is not surveillance — nobody is watching the log in real time — but it is governance. If a question ever arises about 'who did this,' the log has the answer. That alone makes people more careful, which keeps the platform in better shape.

Exporting the log

For quarterly reviews or compliance audits, the History page's filter-and-export combination produces a CSV of any slice of the log. Export is a simple button on the page; the CSV includes every field of every entry in the current filter. The CSV can be opened in Excel, Google Sheets, or any tool that handles comma-separated values.

CHAPTER 25

Settings Reference

Every configurable value in the admin area, with recommended defaults.

This chapter is a reference rather than a narrative. It lists every configurable setting exposed in the admin area, what the setting controls, the recommended default value, and the typical range of values teams actually use. If you only want to answer the question 'what should this knob be set to?' this is the chapter to open.

Algorithm weights

Setting	Default	Typical range	Notes
Days Since Last Call	30%	20-40%	Anchor — always the largest or second-largest weight.
Call Frequency	25%	15-30%	Rewards consistency; lower if team is new.
Availability	20%	10-30%	Raise if team reports calling unavailable LOs.
Boost	10%	5-20%	Raise for campaigns; lower if Boost is under-used.
Tier	5%	0-15%	Keep modest unless tier is a real business signal.
90-Day Transfers	10%	5-20%	Higher values focus on actively transferring LOs.

User and role defaults

Setting	Default	Notes
Default new-user role	CLR	Override per-user at creation time.
Default Is CLR for CLR role	true	Effectively never changes at creation.
Session duration	14 days	Signed cookie; covers a long weekend and back.
Password minimum length	10 chars	Stronger is welcome; the floor is 10.
Password complexity	1 number + 1 symbol	Not configurable via UI; see Chapter 29.

Email and notifications

Setting	Default	Notes
Resend API key	(secret)	Rotate annually; store in env var RESEND_API_KEY.
From address	reports@wlc.it.com	Must have SPF and DKIM aligned in DNS.
EOD recipients	Scott, Chris	2–4 recipients is healthy; avoid distribution lists.
EOD send time	21:00 local	Follows America/Los_Angeles timezone by default.
Welcome email subject	'Welcome to CLR Center'	Localizable but rarely changed.
Password reset subject	'Reset your password'	Localizable but rarely changed.

NMLS compliance

Setting	Default	Notes
Check cadence (days)	60	Two-month cycle; do not shorten below 45.
Assignment method	Random (weighted fair)	Single option in current build.
Escalation threshold (days)	7	Re-assigns after one week of inactivity.
Non-Active escalation	Immediate	Fires on check completion with non-Active status.
Executive email at	21 days	Third-level escalation.

Scheduling and cron jobs

Job	Default time	Purpose
Daily list generation	05:00	Compute today's calling list per CLR.
NMLS escalation sweep	00:00	Re-assign stalled checks.
Incomplete-LO reminder	Every 3 days	Nudge LOs who have not updated their roster entry.
EOD email	21:00	Send daily summary.
Audit log integrity check	04:00	Sanity-check the audit table's invariants.

Security-relevant settings

Setting	Default	Notes
SESSION_SECRET	(env var)	Rotate yearly; invalidates all existing sessions.
Failed-login lockout threshold	10	Per-account lock after 10 failures in 15 minutes.
Lockout duration	15 minutes	Account auto-unlocks; admin can clear earlier.
Admin auth expiry	14 days	Same session cookie as CLRs.
Bonzo password display	Masked by default	Eye toggle reveals; clipboard works while masked.

Role permission quick table

Capability	CLR	Admin-CLR	Admin-only
See own daily list	Yes	Yes	No
Log appointments	Yes	Yes	No
Submit EOD	Yes	Yes	No
See team stats	Limited	Full	Full
Manage users	No	Yes	Yes
Edit algorithm weights	No	Yes	Yes
Run admin override	No	Yes	Yes
View audit log	No	Yes	Yes
See Bonzo credentials	No	Yes	Yes
NMLS tasks page	Assigned only	Full	Full

Appendix C contains the authoritative full permission matrix.

Settings by frequency of use

Chapter 25 listed every setting in the platform. This section reorganizes the same settings by how often they are touched in practice, which is a more useful frame when you are trying to decide which knobs to focus on learning first.

Settings you will touch weekly

- **LO roster edits** — availability, phone, email, tier changes happen as LOs update their own situations.
- **Boost** — campaigns and priorities shift; boost follows.
- **NMLS task assignments** — most are automatic, but occasional manual reassignment is normal.

Settings you will touch monthly

- **User edits** — new hires, role changes, archives.
- **EOD recipient list** — occasional additions or removals as leadership changes.
- **Algorithm weights** — should be rare, but a monthly review is healthy.

Settings you will touch quarterly or yearly

- **Bonzo password rotation.**
- **Resend API key rotation.**
- **SESSION_SECRET rotation.**
- **Deep weight re-tuning** — typically in response to team growth or strategic shifts.

Settings you will almost never touch

- **Daily generation time** (5 AM).
- **EOD send time** (9 PM).
- **NMLS cadence** (60 days).
- **Session duration** (14 days).
- **Password minimum length** (10 chars).

Recommended review cadence

Review item	Cadence	Owner
LO roster completeness	Weekly	Chris
Recent audit log	Daily (5 min)	Chris
Email delivery health	Weekly	Chris
Team stats trend	Weekly	Chris
Algorithm weight review	Monthly	Chris + Ethan
Credential rotation (Bonzo)	Quarterly	Chris
Credential rotation (Resend API)	Yearly	Ethan
SESSION_SECRET rotation	Yearly	Ethan
Backup verification (restore test)	Quarterly	Ethan
DR runbook review	Yearly	Ethan

This table is the operating rhythm of running CLR Center. If the rhythm is kept, the platform stays healthy indefinitely with very little effort. If the rhythm slips — a quarter goes by with no weight review, a year with no backup restore test — the system does not fail, but small problems accumulate in the background.

Change management for settings

When making a non-trivial settings change (weights, EOD recipients, email configuration), follow the informal change-management loop:

1. **Propose** the change in team chat with rationale. Invite pushback.
2. **Apply** the change after 24 hours if no objections.
3. **Announce** the change when applied, with an expected effective time.
4. **Review** the change's effect one week later. Decide: keep, adjust, revert.

This loop is not enforced by the platform; it is a social contract. It scales down (trivial changes skip 1-3) and up (major changes add stakeholders). Most changes fit the four-step loop comfortably.

PART SIX

Technical Reference

Architecture, database, algorithm internals, auth, email, notifications, deployment.

System Architecture

The stack, from browser to database, and how the pieces fit together.

This chapter describes CLR Connection Center's technical architecture in plain language. You do not need to be a software engineer to follow it — the goal is to give an operator enough mental model to understand why the platform behaves the way it does, to read a Railway log without panic, and to communicate precisely with engineering when a problem arises. Subsequent chapters zoom into specific pieces.

The stack at a glance

Layer	Technology	Role
Browser	React 18 + TypeScript	The app the user interacts with
Styling	Tailwind CSS + shadcn/ui	Design system and component library
Build	Vite	Fast dev server and production bundler
Server	Express on Node.js 20	API endpoints, auth, cron jobs
ORM	Drizzle	Type-safe database access layer
Database	SQLite	Persistent storage on a Railway volume
Email	Resend API	Outbound transactional email
Hosting	Railway	Container runtime and networking
Domain	GoDaddy	DNS management (subdomain CNAME)

End-to-end request flow

When a CLR clicks 'Log Appointment' in the browser, a sequence of events fires:

1. The React component collects the form data and calls a client-side function that dispatches a **POST /api/appointments** request over HTTPS to the Railway deployment.
2. Railway's load balancer terminates TLS and forwards the request to the Express server inside our container.
3. Express runs the request through a middleware chain: cookie parsing, session validation (the signed **clr_session** cookie), CSRF checks, and request logging.
4. The appointment route handler validates the payload against a Zod schema, then uses a Drizzle query builder to insert a row into the SQLite database.
5. A follow-up audit-log insert is dispatched in the same transaction.
6. If the appointment triggers a downstream action (notification, email), that action is queued to the post-response hook so the HTTP response is not delayed.
7. Express responds with JSON: the newly-created appointment row. The React client updates its local cache and the UI re-renders.

The entire round trip typically takes 80–150 milliseconds from click to confirmation toast. The most expensive step is the TLS termination at the load balancer — the actual database write is sub-millisecond because SQLite is on local disk.

Why Express + Vite + React + TypeScript

Each piece of the stack was chosen for a specific reason that we still stand behind.

- **Express** is the simplest, most battle-tested HTTP framework in the Node ecosystem. It does not invent its own ideas; it is a thin layer over Node's HTTP module. This keeps the server code readable by anyone who knows JavaScript, and it makes debugging a log-tail exercise rather than a framework archaeology project.
- **Vite** gives us sub-second dev rebuilds and optimized production bundles. It is the default choice for modern React apps and the ecosystem is mature.
- **React 18** with strict mode is our view layer. Concurrent rendering and automatic batching keep the UI responsive even when the CLR is typing in one panel while data is loading in another.
- **TypeScript** catches an enormous class of errors at compile time. In a small team where engineering time is the scarcest resource, types pay for themselves by preventing bugs from ever reaching production.
- **Tailwind CSS** lets us style directly in JSX without context-switching into CSS files. Combined with shadcn/ui's unstyled-but-accessible primitives, we get a consistent design system without a custom component library to maintain.
- **Drizzle** is a lightweight ORM that writes SQL you can read. It is type-safe without being so clever that the generated SQL is mysterious.
- **SQLite** is fast, reliable, and trivially operable. For our data volume (thousands of rows, not millions) it outperforms managed Postgres at a fraction of the complexity. The database is a single file on disk.

Railway as infrastructure

Railway is a platform-as-a-service that gives us a container, a volume, and a public URL. It deploys from our GitHub repository on push, runs our Node server, and surfaces logs and environment variables through a clean web dashboard. We pay for compute by the minute and for storage by the gigabyte; at current scale our monthly bill is in the low tens of dollars.

The Railway-specific constraint that has influenced our architecture is the SMTP port block — covered in Chapter 21 and the reason we use Resend. A second Railway specific consideration is the **volume**: the SQLite database file lives on a Railway persistent volume that survives container restarts but is single-availability-zone. Backing up the volume is covered in Chapter 32.

The Resend integration

Resend is accessed through a thin wrapper in **server/email.ts**. The wrapper exposes three functions: `sendWelcomeEmail`, `sendPasswordReset`, and `sendEodReport`. Each one renders a minimal HTML template, attaches a plain-text alternative, and calls the Resend SDK's `emails.send` method. Failures are logged to the audit log with a tag of `EMAIL_SEND_FAILED` and retried once automatically. Persistent failures produce an admin broadcast notification.

No third-party analytics

CLR Connection Center does not load Google Analytics, Mixpanel, Segment, or any other third-party telemetry. This is deliberate: internal tools should not leak operational data to outside vendors. The only external network call we make from the browser is to our own API on the same domain. The only external network call we make from the server is to Resend. Everything else is local.

Frontend architecture in depth

Routing

The React frontend uses client-side routing with react-router. The top-level routes correspond to the main pages of the platform:

Path	Component	Access
/	Dashboard	CLR and Admin-CLR
/list	CallingList	CLR and Admin-CLR
/appointments	Appointments	CLR and Admin-CLR
/eod	EodReport	CLR and Admin-CLR
/team	TeamStats	All roles
/chat	TeamChat	All roles
/los	LoRoster	All roles
/admin/users	UserAdmin	Admin-CLR and Admin-only
/admin/algorithm	AlgorithmAdmin	Admin-CLR and Admin-only
/admin/email	EmailAdmin	Admin-CLR and Admin-only
/admin/bonzo	BonzoAdmin	Admin-CLR and Admin-only
/admin/nmls	NmlsAdmin	Admin-CLR and Admin-only
/admin/history	HistoryAdmin	Admin-CLR and Admin-only
/login	Login	Public

State management

We do not use Redux, MobX, or any global state library. React's built-in `useState` and `useReducer` hooks handle component-level state; server state (data fetched from `/api` endpoints) is managed with TanStack Query (formerly React Query). This split works well for our scale — global state libraries solve problems we do not have.

TanStack Query

Every server-fetched piece of data flows through a TanStack Query hook. The library provides caching, background refetching, stale-while-revalidate, and optimistic updates out of the box. A typical hook looks like:

```
const { data, isLoading } = useQuery({
  queryKey: ['calling-list', date],
  queryFn: () => fetchCallingList(date),
});
```

The `queryKey` is the cache key. Two components that fetch the same data (same key) share the underlying cache entry, so we do not make redundant network requests. Cache invalidation after mutations is explicit: `queryClient.invalidateQueries(['calling-list'])` after logging an appointment, for instance.

shadcn/ui

Our component library is `shadcn/ui` — a collection of copy-paste React components built on Radix UI primitives and styled with Tailwind. 'Copy-paste' means we do not install `shadcn` as an npm dependency; we run a CLI that adds the component's source directly to our repository, where we can modify it freely. This gives us full control without giving up the benefits of a well-designed component system.

Components we use extensively: Button, Card, Dialog, DropdownMenu, Input, Label, Select, Table, Toast, Tooltip. Each is accessible by default (focus management, keyboard navigation, ARIA attributes) via Radix. We rarely have to think about accessibility as a separate concern because the primitives already handle it.

Tailwind CSS

Every class in every component is a Tailwind utility class. The design tokens — colors (Navy, Gold), spacing scale, typography — live in **`tailwind.config.ts`**. When we want a new shade of navy, we edit the config; that propagates to every component that references it. This replaces what would be a large bespoke CSS codebase in a more traditional stack.

CHAPTER 27

Database Schema

Every table, column, and relationship in the SQLite database.

The CLR Connection Center database is a single SQLite file with roughly a dozen tables. This chapter documents each table, the columns it contains, the types, and the relationships. Understanding the schema makes the rest of the platform easier to reason about — most features are straightforward reads and writes against the tables below.

Table: users

Column	Type	Meaning
id	INTEGER PK	Autoincremented surrogate key
email	TEXT UNIQUE	Login identifier
password_hash	TEXT	Bcrypt hash — never the plain password
first_name	TEXT	Given name
last_name	TEXT	Surname
role	TEXT	CLR Admin-CLR Admin-only
is_clr	INTEGER (0/1)	Whether user participates in the daily list
archived	INTEGER (0/1)	Soft-delete flag
created_at	TEXT (ISO)	Account creation timestamp
last_login_at	TEXT (ISO)	Most recent successful login

Table: los (loan officers)

Column	Type	Meaning
id	INTEGER PK	Autoincremented surrogate key
nmls_id	TEXT	NMLS license identifier
first_name	TEXT	Given name
last_name	TEXT	Surname
tier	TEXT	Standard Premium Top
active	INTEGER (0/1)	Participates in the daily list
archived	INTEGER (0/1)	Soft-delete flag
phone	TEXT	Primary contact number
email	TEXT	Primary contact email
notes	TEXT	Free-form team notes
created_at	TEXT (ISO)	Roster creation timestamp

Table: assignments

Column	Type	Meaning
id	INTEGER PK	Autoincremented surrogate key
date	TEXT (YYYY-MM-DD)	Assignment day, local TZ
clr_id	INTEGER FK	References users.id
lo_id	INTEGER FK	References los.id
rank	INTEGER	Position in the daily list (1–N)
score	REAL	Composite algorithm score
boosted	INTEGER (0/1)	Whether admin flagged this assignment
created_at	TEXT (ISO)	Generation timestamp

Table: appointments

Column	Type	Meaning
id	INTEGER PK	Autoincremented surrogate key
clr_id	INTEGER FK	References users.id
lo_id	INTEGER FK	References los.id
scheduled_at	TEXT (ISO)	When the call/meeting is scheduled
outcome	TEXT	Booked Transferred No-show Other
notes	TEXT	CLR's notes about the appointment
created_at	TEXT (ISO)	Log timestamp
edited_at	TEXT (ISO)	Most recent edit timestamp

Table: eod_reports

Column	Type	Meaning
id	INTEGER PK	Autoincremented surrogate key
clr_id	INTEGER FK	References users.id
date	TEXT (YYYY-MM-DD)	Report date
calls_made	INTEGER	Total dials for the day
connections	INTEGER	Calls that reached a human
appointments	INTEGER	Appointments booked
transfers	INTEGER	Warm transfers completed
summary	TEXT	Free-form narrative
submitted_at	TEXT (ISO)	When the CLR submitted

Table: notifications

Column	Type	Meaning
id	INTEGER PK	Autoincremented surrogate key
user_id	INTEGER FK NULL	Target user (NULL = broadcast)
kind	TEXT	Notification type tag
title	TEXT	Short banner title
body	TEXT	Longer explanation
read_at	TEXT (ISO) NULL	When the user dismissed it
created_at	TEXT (ISO)	Creation timestamp

The **user_id NULL** trick is how broadcasts work. A notification with NULL user_id is shown to every logged-in user until each one individually dismisses it (each dismissal writes a row to a separate notification_reads table). This is how 'admin override issued' reaches every CLR without the platform having to fan-out-insert one row per user.

Table: team_chat_messages

Column	Type	Meaning
id	INTEGER PK	Autoincremented surrogate key
user_id	INTEGER FK	Sender
body	TEXT	Message content, max 4000 chars
created_at	TEXT (ISO)	Send timestamp
edited_at	TEXT (ISO) NULL	Most recent edit

Table: nmls_tasks

Column	Type	Meaning
id	INTEGER PK	Autoincremented surrogate key
lo_id	INTEGER FK	LO to be checked
assigned_clr_id	INTEGER FK	CLR responsible for the check
due_date	TEXT	Target completion date
completed_at	TEXT (ISO) NULL	When check was marked done
status_found	TEXT NULL	Active Inactive Suspended Terminated
sponsor_found	TEXT NULL	Sponsor name as seen on NMLS
notes	TEXT	CLR's free-form notes
reassigned_count	INTEGER	Number of escalations

Table: audit_log

Column	Type	Meaning
id	INTEGER PK	Autoincremented surrogate key
ts	TEXT (ISO)	Event timestamp
actor_id	INTEGER FK NULL	User who acted (NULL = system)
action	TEXT	Action tag, e.g., USER_EDITED
target_type	TEXT NULL	Affected entity kind
target_id	INTEGER NULL	Affected entity id
details	TEXT (JSON)	Payload: before/after, context

Table: algorithm_weights

A single-row table holding the current six weights. On save, the old row is moved to `algorithm_weights_history` and a new row is written. This preserves full historical weight state without joining against the audit log.

Indexes, constraints, and query patterns

Indexes

SQLite creates an implicit index on every PRIMARY KEY. Beyond that, the schema defines explicit indexes on the foreign-key and frequently-queried columns:

Index	Table	Columns	Query pattern
idx_assign_date_clr	assignments	(date, clr_id)	Today's list for a CLR
idx_assign_lo	assignments	(lo_id)	Historical assignments for an LO
idx_appts_clr_date	appointments	(clr_id, created_at)	A CLR's day
idx_appts_lo	appointments	(lo_id)	An LO's call history
idx_eod_date	eod_reports	(date)	EOD for a given day
idx_audit_ts	audit_log	(ts DESC)	Recent entries
idx_audit_actor	audit_log	(actor_id)	Entries by a user
idx_notif_user	notifications	(user_id)	A user's inbox
idx_nmls_due	nmls_tasks	(due_date)	Tasks due soon

At our current data scale (tens of thousands of rows across all tables) the indexes are almost excess; SQLite's full-table scan is fast enough that queries complete in a few milliseconds even without indexes. The indexes exist for forward-compatibility — they will earn their keep when the database grows to millions of rows.

Foreign keys

SQLite supports foreign-key enforcement but does not enable it by default. The server runs `PRAGMA foreign_keys = ON;` on connection open, so every query in production respects FK constraints. Attempting to delete a user with active appointments, or an LO with active assignments, will raise a constraint error and the operation will roll back. This is intentional — the platform does not do hard deletes outside of test fixtures.

Common query patterns

A handful of query shapes account for the majority of the server's SQL traffic:

Today's list for a CLR:

```
SELECT a.*, lo.first_name, lo.last_name FROM assignments a
JOIN los lo ON lo.id = a.lo_id
WHERE a.date = ? AND a.clr_id = ?
ORDER BY a.rank ASC
```

A CLR's recent appointments:

```
SELECT * FROM appointments
WHERE clr_id = ? AND created_at >= datetime('now', '-7 days')
ORDER BY created_at DESC
```

Team stats for a week:

```
SELECT clr_id, COUNT(*) AS appts, SUM(outcome='Transferred') AS transfers
FROM appointments
WHERE created_at >= ? AND created_at < ?
GROUP BY clr_id
```

Audit trail for an entity:

```
SELECT * FROM audit_log
WHERE target_type = ? AND target_id = ?
ORDER BY ts DESC
```

Each of these queries uses the indexes above and completes in sub-millisecond time against the current database.

Migrations

Schema changes are expressed as Drizzle migration scripts in **server/migrations/**. Each script has a numeric prefix (0001, 0002, ...) and is applied exactly once by the server on startup. A migrations table records which scripts have run; applied scripts are never re-applied.

Writing a new migration follows the standard Drizzle flow: define the desired schema changes in TypeScript, run `drizzle-kit generate`, review the generated SQL, commit. The server picks up the new migration on next restart and applies it within seconds. Down-migrations are not automatic; we write them manually when the change is risky enough to warrant a rollback path.

Backups and the schema

A backup of `data.db` includes the schema, the migrations history, and all rows. Restoring the backup onto a fresh server produces an exact clone of the original state — no schema mismatch is possible because the schema travels with the data.

Integrity checks

The nightly audit log integrity check (04:00 cron) runs `PRAGMA integrity_check`; and validates that the audit log has no sequential-id gaps. Any failure produces an admin broadcast and an email to Ethan. In the platform's history the check has never failed.

CHAPTER 28

Algorithm in Code

How the scoring function, normalization, and recent-transfers SQL actually run.

Chapter 16 described the algorithm in operational terms; this chapter walks the actual code paths that compute each CLR's daily list. If you are a developer investigating a scoring edge case, this is the chapter to start with. If you are an admin trying to understand why a particular LO ranked where it did, Chapter 16 is a better entry point — but reading this chapter will solidify your mental model.

High-level structure

The algorithm is implemented in **server/algorithm.ts**. The entry point is `generateDailyAssignments(date)`, invoked by the 5 AM cron job and also by the admin-override endpoint. It performs four steps:

1. Fetch every active LO and every active CLR from the database.
2. For each (CLR, LO) pair, compute a raw factor vector — six numbers corresponding to the six weights.
3. Normalize each factor to the 0–1 range using factor-specific logic.
4. Weight, sum, rank, and assign the top LOs to each CLR in round-robin fashion that respects the availability constraints.

The scoring function

The core of step 2/3 is the `scoreLo` function. Given an LO and the team's current context, it returns a scalar score between 0 and 1 that represents 'how valuable is calling this LO today.' Pseudo-code:

```
score = w.daysSince * norm.daysSince(lo.lastCallAt)
+ w.frequency * norm.frequency(lo.recentCallCount)
+ w.availability * norm.availability(lo.availableSlots)
+ w.boost * (lo.boosted ? 1 : 0)
+ w.tier * norm.tier(lo.tier)
+ w.transfers * norm.transfers(lo.transfers90d)
```

Normalization details

- **daysSince**: clamp to 0–30 days, then divide by 30. An LO last called 15 days ago yields 0.5; an LO last called 30+ days ago yields 1.0.
- **frequency**: count of calls in the last 30 days, divided by the 90th-percentile count across all LOs. Clamped to 0–1. Rewards consistent engagement without letting a single very-active LO dominate.
- **availability**: ratio of available hourly slots to total slots over the next 3 business days, from the LO's own availability calendar.
- **boost**: binary 0 or 1. A boosted LO contributes fully; otherwise nothing.
- **tier**: Standard 0.33, Premium 0.67, Top 1.0.
- **transfers**: the recent-transfers SQL described in the next section.

The recent-transfers raw SQL

The 90-day transfers factor is the one part of the algorithm that drops below the Drizzle ORM to raw SQL, for performance reasons. The query counts, for each LO, the number of appointments with outcome 'Transferred' in the last 90 days:

```
SELECT lo_id, COUNT(*) AS c
FROM appointments
WHERE outcome = 'Transferred'
AND scheduled_at >= datetime('now', '-90 days')
GROUP BY lo_id
```

The result is joined back into the scoring loop as a lookup keyed by lo_id. The normalization step divides each count by the team-wide max, producing a 0–1 factor. If no LO has transferred anything in 90 days, every LO contributes 0 to this factor — the weight effectively goes unused, which is acceptable.

Round-robin assignment

Having a score per (CLR, LO) pair is not enough — we still have to decide which LO goes to which CLR, with no duplicates. The platform uses a round-robin scheme based on stable sorting:

1. Sort CLRs into a consistent order (by id).
2. For each CLR in order, pull the top-scoring unassigned LO (scoring breaks ties by `lo_id` to ensure determinism).
3. Mark that (CLR, LO) pair as assigned and the LO as taken for today.
4. Advance to the next CLR and repeat until each CLR has `daily_target` assignments.

This naive approach produces acceptable load-balancing at our scale. More sophisticated matching (Hungarian algorithm, auction-based assignment) is possible but was unnecessary; the round-robin result is within 5% of optimal in our measurements.

The normalization migration

Early builds of the platform used un-normalized factors, and the Days Since factor effectively drowned out everything else because its raw range was larger. The normalization migration — applied in Q1 2026 — refactored each factor to its current 0–1 form. The migration was transparent to users but required a one-time re-computation of all historical assignments; that re-computation lives in the **`server/migrations/normalize_factors.ts`** script and is idempotent.

Determinism

Given the same database state, the algorithm produces the exact same assignments. There is no randomness anywhere — not even a tie-breaker random draw. This makes the algorithm auditable and reproducible: a disagreement about today's list can always be resolved by re-running the scoring function against a snapshot of the database.

Algorithm edge cases and testing

Edge case: new CLR, no history

When a brand-new CLR joins the team and their first list is about to be generated, several algorithm inputs have no value for them yet. They have no appointment history, no transfer history, no Frequency data. The algorithm must handle this gracefully — giving the new CLR a reasonable first day without unfairly penalizing them (by, say, giving them the least-attractive LOs because they 'have no track record').

The implementation handles this by computing per-LO scores that do not depend on the CLR. Each LO's score is a function of that LO's history, not the CLR's. When the algorithm assigns LOs to CLRs in the round-robin loop, the new CLR picks up their share of high-scoring LOs naturally. The new CLR's first list is structurally identical to a veteran CLR's list of the same size.

Edge case: zero-active-LO state

If every active LO is simultaneously unavailable (a holiday week, a mass PTO event), the algorithm has no LOs to assign. The handler for this case is straightforward: generate empty assignment lists for each CLR and write a warning row to the audit log noting 'no active LOs available on date X.' The team sees empty lists in the morning and knows why.

This is a legitimate corner case; the algorithm does not panic. It simply reports what is true (nothing to assign) and lets the team take whatever manual action is appropriate — reaching out directly, doing admin work, catching up on documentation.

Edge case: tie-breaking

Two LOs can end up with identical composite scores, particularly when the inputs are sparse. The algorithm breaks ties deterministically by LO id (lower id wins). This is arbitrary but stable; running the algorithm twice on the same inputs always produces the same ranking. Users see a consistent ordering, and reproducibility is preserved.

Why lower id and not higher? LO ids are assigned in creation order, so lower id means the LO has been in the roster longer. Tie-breaking in favor of longer tenure feels natural when a real business judgement has to be made arbitrary.

Testing the algorithm

The algorithm is covered by a test suite at **server/algorithm.test.ts**. The tests exercise each factor in isolation (e.g., 'with all other factors at zero, Days Since Last Call monotonically increases the score'), each edge case (new CLR, zero active LOs), and several end-to-end scenarios constructed to produce known rankings.

New tests are added whenever a non-obvious bug is found and fixed. The 'missing transfer in normalization' bug from Q1 2026, for example, produced a test that now lives permanently in the suite and prevents a regression.

Running tests locally

npm test runs the full suite. **npm test -- algorithm** runs only the algorithm tests. **npm test -- --watch** runs in watch mode, re-running on file change. The suite completes in about 8 seconds on a developer laptop.

CHAPTER 29

Authentication

Signed cookies, the `SESSION_SECRET`, `/api/auth/login`, and middleware.

Authentication in CLR Connection Center is deliberately simple: email + password login, a signed session cookie, and server-side middleware that validates the cookie on every request. We do not have federated login, SSO, passkeys, or any of the more advanced mechanisms that modern SaaS products ship. At our scale the simpler system is more reliable and easier to reason about.

Login flow

1. The user visits `/login` and submits email + password via HTML form.
2. The client POSTs to `/api/auth/login` with the credentials.
3. The server looks up the user by email (case-insensitive).
4. If the user exists and is not archived, the server compares the submitted password against the stored bcrypt hash using `bcrypt.compare`.
5. On match, the server issues a cookie named `clr_session` containing a signed JSON payload: `{userId, issuedAt}`.
6. On mismatch, the server returns 401 with a generic 'Invalid email or password' message (we do not reveal whether the email existed).

The `clr_session` cookie

The session cookie is a standard signed cookie. Its value is a base64 of the JSON payload plus an HMAC-SHA256 signature computed with the server's `SESSION_SECRET`. The cookie is marked `HttpOnly` (so client JavaScript cannot read it), `Secure` (HTTPS only), and `SameSite=Lax` (prevents most CSRF). The cookie's `Max-Age` is 14 days.

The `SESSION_SECRET`

`SESSION_SECRET` is a 64-character random hex string stored as a Railway environment variable. It is the key that signs every session cookie. Rotating `SESSION_SECRET` invalidates every session immediately — all users get logged out and must sign in again. We rotate it annually as hygiene, or immediately if we suspect the secret has leaked.

Middleware chain

Every API request passes through an Express middleware chain in this order:

1. **cookieParser** — extracts raw cookies into `req.cookies`.
2. **sessionValidator** — verifies the signature of `clr_session`, parses the payload, and attaches `{userId}` to `req.session`.
3. **userLoader** — fetches the full user row from the database if `req.session` is present; attaches it as `req.user`. Archived users are treated as unauthenticated.
4. **routeHandler** — the per-endpoint code. Admin-only routes include a final `requireAdmin` middleware check that returns 403 if `req.user.role` is plain CLR.

Routes that don't require authentication — `/login`, `/api/auth/login`, `/api/health`, `/favicon` — opt out of the middleware chain by registering before the `sessionValidator` middleware. Everything else requires a valid session.

Password storage

Passwords are hashed with `bcrypt` at cost factor 12 (approximately 250ms per hash on Railway's hardware). The original password is never stored anywhere. When a user sets or resets their password, the server hashes it immediately and stores only the hash. When the user signs in, `bcrypt.compare` re-derives the hash from the submitted password and compares it to the stored hash — a slow operation by design, because slowness is what defeats offline brute-force attacks on leaked hashes.

Lockout

Ten consecutive failed login attempts against the same account within 15 minutes triggers a 15-minute lockout. During the lockout, the server returns 429 Too Many Requests for any login attempt on that email, with a message that does not reveal whether the email exists. Lockouts are recorded in the audit log. An admin can clear a lockout early by resetting the user's password.

Logout

Logging out is a POST to `/api/auth/logout`, which returns a Set-Cookie header that expires `clr_session` immediately. The server also writes a LOGOUT entry to the audit log. Closing the browser without logging out leaves the cookie valid until its Max-Age expires (14 days), at which point the session becomes unrecoverable.

Authentication in depth

What the cookie actually looks like

A decoded **clr_session** cookie, on the wire, looks something like:

```
eyJ1c2VySWQiOiJyLCJpYXQiOiE3MTA4ODg0MDB9.hmac-sha256-signature
```

The left portion is base64-encoded JSON: {userId: 12, iat: 1710888400}. The right portion is an HMAC-SHA256 of the left portion, computed with SESSION_SECRET. The dot separator matches the JSON Web Token (JWT) format, though we are not using JWT proper — we are using a simpler pattern with the same visual layout.

To verify a cookie, the server takes the left portion, computes the HMAC with its SESSION_SECRET, and compares against the signature on the right. If they match, the cookie is trusted; the server then parses userId and looks up the user. If they do not match — because the payload was tampered with, or a different SESSION_SECRET was used — the cookie is rejected.

Why not JWT proper?

JWT has a rich history of security missteps — the 'alg:none' vulnerability, algorithm confusion attacks, the various issuer/audience edge cases. A simple signed cookie has a smaller attack surface. We do not need the federation features that JWT provides; a bespoke signature scheme is faster and safer for our use case. This is an instance of deliberate under-engineering.

CSRF protection

Cross-Site Request Forgery is the attack where a malicious website causes a user's browser to send an authenticated request to our platform without the user's knowledge. Our first line of defense is the cookie's `SameSite=Lax` attribute: modern browsers refuse to send the cookie on cross-origin POST requests initiated from other sites.

Our second line of defense is a CSRF token for state-changing endpoints. The server issues a per-session random token, embedded in a meta tag on every HTML page. Client JavaScript reads the token and includes it as a header on every POST/PUT/PATCH/DELETE. The server validates the header matches the session's token. An attacker with no access to the rendered page cannot obtain the token.

Failed login telemetry

Every login attempt — success or failure — writes to the audit log. Failures include the attempted email but not the attempted password (writing the password to the log would be its own security incident). Filtering `LOGIN_FAILURE` for a single email over a short window reveals brute-force attempts early.

A burst of `LOGIN_FAILURE` entries from different source IPs targeting the same email is a password-spraying pattern; the 10-failures-in-15-minutes lockout stops this cold but is worth investigating regardless. A burst from a single source IP targeting many emails is a credential stuffing pattern and should trigger a wider review — probably a forced password rotation for the team.

Session hijacking

A valid `clr_session` cookie, if stolen, lets the thief impersonate the user for up to 14 days. Defenses:

- `HttpOnly` flag prevents JavaScript in the browser from reading the cookie.
- `Secure` flag prevents transmission over plain HTTP.
- `SameSite=Lax` limits cross-origin use.
- HMAC signature prevents cookie tampering.
- Rotating `SESSION_SECRET` invalidates all cookies instantly.

If we ever suspect a cookie has been stolen, the response is to rotate `SESSION_SECRET` immediately. Every user is logged out; the thief no longer has a valid cookie either.

CHAPTER 30

Email System

Resend SDK, templates, and the `reports@wlc.it.com` from-address.

This chapter is the developer-facing counterpart to Chapter 21. Chapter 21 covered configuration; this chapter covers the code that actually sends mail, the templates it renders, and the diagnostics available when something goes wrong.

`server/email.ts`

All outbound email routes through a single module, **`server/email.ts`**. The module exports three public functions:

- **`sendWelcomeEmail(user, temporaryPassword)`** — called from the user-creation endpoint; subject 'Welcome to CLR Connection Center.'
- **`sendPasswordResetEmail(user, temporaryPassword)`** — called from the admin reset endpoint and the self-service forgot-password flow; subject 'Reset your password.'
- **`sendEodReport(recipients, reportData)`** — called from the 21:00 cron job; subject 'CLR Center EOD Report — [date].'

The Resend SDK call

Each function builds a message object with `from`, `to`, `subject`, `html`, and `text` fields, then calls `resend.emails.send(message)`. The SDK returns a Promise that resolves to a message id on success or throws on failure. We wrap the call in `try/catch`; on success we audit-log the message id, on failure we audit-log the error and fire an admin notification if three consecutive messages of the same type have failed.

Templates

Email templates are plain TypeScript template literals in **server/email-templates.ts**. They are deliberately simple HTML with inline styles — no external CSS, no image assets, no JavaScript. This keeps them reliable across the wildly variable rendering environments of corporate email clients (Outlook, Gmail, Apple Mail, Yahoo). Each template also has a plain-text alternative that mirrors the HTML content for text-only clients and accessibility.

A typical template is 40–80 lines of HTML. The welcome email, for instance, contains the platform URL, the user's email as their login, their temporary password in a monospace font, and a single sentence directing them to log in and choose a personal password. The EOD report template is more complex: it renders a table of each CLR's metrics and a team summary row at the bottom.

From address and DNS

The from address **reports@wlc.it.com** is verified with Resend through DNS records that prove we control the domain: an SPF record listing Resend's sending IPs, and two DKIM CNAME records pointing to Resend's key infrastructure. The records are in GoDaddy under the wlc.it.com zone and do not need to be touched during normal operation. When Resend rotates DKIM keys (rarely), they notify us and we update the CNAME targets.

Delivery diagnostics

Every send produces a Resend message id that is written to the audit log. The Resend dashboard lets us paste a message id and see delivery status, bounces, and open/click tracking (we have opens disabled for privacy reasons). For a user reporting that they did not receive an email, the triage path is: look up their address in the audit log, find the most recent send, grab the message id, paste into Resend, check status. Nine times out of ten the message was accepted and delivered; the user needs to check their spam folder.

Template design philosophy

The three email templates we send — welcome, password reset, EOD report — are each tuned to their purpose. The design decisions are small but deliberate. This section walks through each template and the thinking behind its layout.

The welcome email

The welcome email is sent exactly once per user, at the moment of account creation. Its job is to communicate three things: 'welcome,' 'here is your login,' 'here is your temporary password.' Everything else is noise.

The template is about 70 lines of HTML. It opens with a gold-on-navy header containing the platform name. The body greets the user by first name, states their email as their login identifier, shows the temporary password in a monospaced box, and provides a single prominent button labelled 'Sign in now.' The button links to the platform URL.

A short footer lists the admin who created the account ('Account created by Chris Redoble') and invites the user to reach out with questions. The footer also reminds them to choose a personal password on first login.

What the welcome email does NOT do

- It does not include a full product tour.
- It does not include marketing language.
- It does not include links to social media or other external sites.
- It does not include tracking pixels or open-tracking.
- It does not include an unsubscribe link (transactional email).

Each of these omissions is deliberate. The email is a short, trusted, transactional message; padding it with marketing would undermine that trust.

The password reset email

Structurally similar to the welcome email but with a different headline and a different tone. The headline reads 'Reset your password.' The body states the new temporary password in a monospaced box and directs the user to sign in and change it to something personal.

An important piece of content: the email notes who triggered the reset. If the user triggered the self-service forgot-password flow, the email says 'You requested a password reset.' If an admin reset on the user's behalf, the email says 'An admin has reset your password on your behalf.' This distinction matters — if the user sees a reset email they did not request, they should be alerted that an admin acted on their account.

The EOD report email

Longer and more structured than the other two. The subject line includes the date. The body opens with a one-line summary ('April 18 — 12 CLRs, 287 calls, 41 appointments, 9 transfers'). Then comes a table of each CLR's metrics: calls, connections, appointments, transfers. The table ends with a team-total row.

Below the table is a brief narrative block with highlights: the top-performing CLR of the day, any unusual events (overrides, NMLS escalations, outages), and anything the CLRs flagged in their individual EOD summaries. The narrative is generated from a simple template with conditional sections — no machine learning, just string substitution.

Footer: a link to the platform URL for deeper analysis, and the standard contact info for Chris and Ethan in case the recipient has questions.

Email rendering in Outlook

Outlook (specifically the Windows desktop client) uses a Microsoft Word-based rendering engine that historically struggles with modern HTML. We test every template change in Outlook before release, and we keep our HTML conservative — tables for layout, inline styles only, no flexbox, no CSS grid, no media queries. This constrains what we can do visually, but it guarantees the email renders consistently for every recipient.

CHAPTER 31

Notifications

The notifications table, broadcasts, and the 3-day cron for incomplete LOs.

Notifications are the in-product equivalent of a quick tap on the shoulder. They appear as a small badge on the bell icon in the main nav, expand to a dropdown of dismissible cards, and fade away once read. This chapter describes how they are produced, how broadcasts differ from per-user notifications, and the cron job that periodically nudges the team about incomplete LO records.

Per-user notifications

A per-user notification is a row in the **notifications** table with a concrete **user_id**. The row is created by server code when an event relevant to that user occurs — 'your NMLS check is due,' 'your appointment was edited by an admin,' 'your EOD report is overdue.' The client polls the notifications endpoint every 30 seconds while the app is open, and unread rows are rendered as dismissible cards in the dropdown.

Broadcasts (user_id = NULL)

A broadcast notification is a row where **user_id** is NULL. The server treats such rows as 'visible to every logged-in user,' and each user has a per-user read state tracked in a separate **notification_reads** table. This design keeps the notifications table tiny even when we broadcast to a large team: one row per broadcast event, not one per user.

Broadcasts are used for team-wide announcements that the platform itself generates: admin overrides, NMLS escalations that have missed a second deadline, weight changes that materially affect the daily list, planned downtime. They are deliberately rare; sending too many trains users to dismiss without reading.

The 3-day incomplete-LO cron

One notable automated notification is the **incomplete LO reminder**. The platform's LO roster supports a large number of optional fields — phone, email, NMLS ID, availability, notes. When a new LO is added quickly without filling these in, the roster entry is 'incomplete' and the algorithm cannot score that LO correctly. To catch this, a cron job runs every 3 days and scans the LO roster; for each LO missing one or more key fields, a notification is sent to every admin (broadcast scoped to admin users only).

The notification body lists the specific missing fields, e.g., 'Dan Baker — missing NMLS ID, phone.' Admins can click the notification to jump straight to the LO's edit page, fill in the missing data, and save. The next run of the cron skips LOs that have been completed, preventing notification spam.

Delivery channel mapping

Some notifications also fire an email (see Chapter 21's notification-defaults table). The decision of which channels to use is made when the notification is created — the creating code explicitly calls `notify(user, title, body)` for an in-product-only message, or `notifyAndEmail(user, title, body, emailArgs)` for both. There is no automatic bridge from 'notification created' to 'email sent'; that explicitness is intentional to avoid accidental mass emails.

Cleanup

Notifications older than 90 days are archived to a compressed table once a week. Users never see archived notifications in the UI. Admins investigating 'what did the platform tell the team six months ago?' can query the archive directly.

Notification design principles

The platform produces a few dozen notifications per week across all users. Each one competes for attention with the user's other tools — email, Slack, Bonzo alerts, browser notifications. The temptation to over-notify is real, and the cost of giving in is high: notifications that are ignored are worse than no notifications, because they train users to ignore future ones. This section captures the principles we use when deciding whether a given event deserves a notification at all.

Principle 1: Only things the user might act on

A notification should prompt a response. 'Your NMLS check is due' prompts action (do the check). 'Your appointment was edited by an admin' prompts awareness (know that it was edited; review if concerned). 'The daily list was generated' prompts nothing — the user would have opened the list anyway. The third example does not deserve a notification, even though it is a real event.

Principle 2: Urgency matches channel

An urgent event warrants an urgent channel: a broadcast banner, a popup, an email. A low-urgency event warrants a low-urgency channel: a passive card in the bell menu. Mismatches are obvious — a 'hello, here is a friendly reminder' that pops up as an urgent banner is instantly dismissed; a 'production is down' that only appears as a bell card goes unread for hours.

Principle 3: Dismissible and non-sticky

Every notification can be dismissed. Dismissed notifications do not come back. If the user dismisses a notification about an NMLS check, the platform assumes they have noted it. The platform does not re-notify unless the situation changes (a new day, a new escalation level). This respect for the user's attention builds the trust that makes future notifications worth reading.

Principle 4: One place to see everything

The bell icon in the main nav is the single aggregation point for notifications. Users can glance there and know whether they have anything pending. We resist the urge to scatter notifications across multiple locations (dashboard cards, page banners, toast messages) because that multiplies the places users have to look.

Exception: transient feedback from user actions (the 'Saved' toast, the 'Error' toast) is displayed inline near the action, not in the bell menu. Those messages are not 'notifications' in our sense — they are operation receipts.

Principle 5: Broadcasts are rare

A broadcast reaches every logged-in user. The platform currently uses broadcasts for fewer than one event per month on average. Each broadcast is a deliberate choice: 'every user in the team should see this right now.' If we started broadcasting routine events, users would learn to dismiss broadcasts without reading. That would be an expensive capability to recover.

What a healthy notification distribution looks like

Channel	Target rate	Our current rate
Email per user per week	<3	~2
In-platform notifications per user per week	<7	~4
Broadcasts per month	<2	~1
Toasts per session	1-3	~2

Our current rates are comfortably below the target thresholds, which is the right side of the threshold to be on. If the rates crept up, we would push back on the next notification that was being proposed.

CHAPTER 32

Deployment

Railway project, volume, environment variables, and the GoDaddy CNAME.

This chapter covers everything about running CLR Connection Center in production: the Railway project structure, the persistent volume for SQLite, the environment variables that must be set, the domain configuration, and the deploy workflow that takes a git push to a live update.

The Railway project

Resource	Identifier	Purpose
Project	06e30810	Container for all environments
Service	53411674	The web server deployment
Volume	23ce2e17	Persistent storage for data.db
Environment	production	Default env; we do not run staging
Region	us-west1	GCP-backed region Railway provides

Service configuration

- **Build command:** `npm ci && npm run build`
- **Start command:** `node dist/server.js`
- **Exposed port:** 3000 (set via the PORT env var Railway injects)
- **Health check:** GET `/api/health`, expected 200 with JSON `{status:'ok'}`
- **Restart policy:** always, with 5-minute backoff on repeated crashes

The persistent volume

Railway provides a persistent volume that survives container restarts and re-deploys. We mount it at **/data** inside the container. SQLite's data file lives at **/data/data.db**, with its WAL and SHM sidecar files in the same directory. Backing up the database is a matter of copying these three files together; see the backup procedure at the end of this chapter.

Environment variables

Variable	Purpose
PORT	Port the server listens on (Railway injects 3000)
NODE_ENV	'production' in live deploys
DATABASE_URL	file:/data/data.db
SESSION_SECRET	64-char random hex for cookie signing
RESEND_API_KEY	Resend API key for outbound email
EOD_RECIPIENTS	Comma-separated emails for EOD reports
EOD_SEND_TIME	Local time for the 21:00 cron (default 21:00)
TIMEZONE	America/Los_Angeles

Deploy workflow

1. A commit lands on the main branch of the GitHub repository.
2. Railway's GitHub integration detects the push and starts a new build.
3. The build container runs `npm ci && npm run build`.
4. On success, Railway provisions a new container, starts the server, and waits for the `/api/health` endpoint to return 200.
5. When the new container is healthy, Railway atomically swaps traffic to it and shuts down the previous container.
6. If health checks fail for 5 minutes, Railway aborts the deploy and keeps the previous container running. The failing build is flagged in the dashboard.

The domain

Railway assigns our service a default URL of the form **web-production-b6285.up.railway.app**. This is the URL we use internally and that appears throughout this manual. If we ever point a subdomain of `wlc.it.com` at the service, the setup is a CNAME record in GoDaddy from (for example) `clr.wlc.it.com` to the Railway URL.

CNAME must be a subdomain, not the root

GoDaddy and most DNS providers do not allow CNAME records on the root domain (`wlc.it.com` itself, written as `'@'`); DNS standards reserve the root for A records. Route root-domain requests through a different mechanism (an ALIAS record or a redirect service). For our purposes, a subdomain like `clr.wlc.it.com` pointing to the Railway URL is the correct and simple path.

Backup procedure

1. SSH into the Railway container via the dashboard's shell feature.
2. Run `sqlite3 /data/data.db ".backup /data/backup-$(date +%F).db"`. SQLite's online backup command produces a consistent snapshot even while the server is reading and writing.
3. Download the snapshot via Railway's file-download UI or pipe it out with `cat` and redirect the SSH session's output locally.
4. Store the snapshot in the team's cloud storage (not in git — it contains personal data).

We back up weekly, on Sunday afternoons, and retain the last 12 snapshots. The backup itself is small — current database is ~20 MB — so storage is not a concern.

Rollback

If a deploy causes a regression, the Railway dashboard offers a one-click rollback to the previous build. Clicking Rollback starts a fresh container from the previous build's image and swaps traffic to it; total time from click to stable is about 90 seconds. For database-schema issues, rollback alone is insufficient — restoring a recent backup is also required.

Deployment operations in detail

Reading Railway logs

The Railway dashboard exposes a real-time log stream for the running container. Every line the server writes to stdout or stderr appears here, interleaved with Railway's own platform messages. Reading the log effectively is a skill worth cultivating because it is the fastest path to understanding what the production server is actually doing.

The server uses a simple log format: ISO timestamp, log level, component, message. Example:

```
2026-04-18T12:03:41.217Z INFO server started on port 3000
2026-04-18T12:03:41.230Z INFO db connected: /data/data.db
2026-04-18T12:03:45.811Z INFO algorithm: generating assignments for 2026-04-18
2026-04-18T12:03:45.964Z INFO algorithm: 12 CLRs × 10 slots = 120 assignments
```

Filter the log by piping through `grep` in the Railway shell, or use the dashboard's built-in search. Common searches: 'ERROR' (all error lines), 'algorithm' (generation events), 'email' (Resend calls), 'audit' (audit writes).

Restart

The Railway dashboard has a Restart button. Clicking it terminates the running container (graceful SIGTERM with 30-second drain window) and starts a fresh one from the same build. Restart is useful for:

- Picking up a new environment variable (env changes require a restart to take effect).
- Clearing an in-memory leak — extremely rare, but has happened once at an earlier stage of the product.
- Recovering from a wedged state that no amount of waiting seems to resolve.

The Railway shell

The dashboard offers a browser-based shell into the running container. This is the escape hatch for rare but real operational needs: running a manual backup, inspecting a misbehaving file, running a one-off diagnostic script, or executing a schema migration that was not picked up automatically.

The shell runs as the container's user (not root) in the container's working directory (**/app**). The volume is mounted at **/data** and is writable. Network access is outbound-HTTPS; the same constraints that apply to the server apply here — no SMTP, for instance.

Common shell commands

Purpose	Command
Check database size	<code>ls -la /data/data.db</code>
Run an online backup	<code>sqlite3 /data/data.db ".backup /data/backup.db"</code>
Inspect a table's row count	<code>sqlite3 /data/data.db 'SELECT COUNT(*) FROM los;'</code>
Check disk usage	<code>du -sh /data/</code>
Tail the application log	(logs stream in the dashboard, not the shell)
Check env var	<code>echo \$SESSION_SECRET head -c 10 # first 10 chars only</code>
Inspect running processes	<code>ps aux</code>
Check uptime	<code>uptime</code>

Environment variable rotation

1. In the Railway dashboard, navigate to the service's Variables tab.
2. Edit the variable in place and click Save.
3. Railway automatically restarts the container with the new value.
4. Verify the change by checking behavior (e.g., try a login to verify `SESSION_SECRET` is in force; send a test email to verify `RESEND_API_KEY`).
5. Update team documentation if the change is noteworthy (rare — most rotations are routine and do not need documentation).

Scaling considerations

At current scale — roughly a dozen CLRs, a hundred LOs, a few hundred requests per minute at peak — a single Railway container on the lowest pricing tier is plenty. CPU utilization hovers below 5%. Memory fits comfortably in 512 MB.

If the team grows to 50+ CLRs or the LO roster expands to 1000+, the main scaling pressure will be on the algorithm's daily generation job, which computes scores for every (CLR, LO) pair. At 50×1000 that is 50,000 score computations, still well within a few seconds of compute. At 200×5000 (a million pairs) the generation job might take 30 seconds or more, at which point we would want to parallelize or move to a heavier runtime.

SQLite can comfortably handle the read and write volume at that scale as well. If we ever needed true concurrent write scaling (rare for our workload), a migration to Postgres would be the next step; the Drizzle ORM supports both and the migration would be largely mechanical.

Disaster recovery

A full disaster-recovery scenario — Railway loses our project entirely — is recoverable from three artifacts: the GitHub repository, the environment-variable dump stored in our password manager, and the most recent database backup in cloud storage. The recovery procedure:

1. Create a new Railway project.
2. Connect it to the GitHub repository.
3. Paste the environment variables from the password manager.
4. Provision a new volume and upload the most recent data.db backup.
5. Deploy. Verify `/api/health` returns 200.
6. Update the DNS CNAME if a permanent URL change is needed.
7. Announce to the team; the platform is live again.

Worst-case recovery time is under an hour, driven mostly by the Railway build step. In the platform's history this has never been needed.

PART SEVEN

Troubleshooting

When something breaks, this is the first part to open.

Common Issues Reference

Eighteen real problems, their likely causes, and the exact steps to fix them.

This chapter is the field guide. Each entry is a specific problem a real user has reported, with its most likely causes listed in order of probability and the exact steps to resolve each one. The entries are numbered so that support conversations can reference them directly: 'follow issue 07 in chapter 33.' Start at the top of each entry and work down; in most cases the first listed cause is the one you are hitting.

Issue 01 — I cannot log in

Symptom: the login page rejects your email and password with 'Invalid email or password,' even though you are sure they are correct.

- **Likely cause A — typo.** Email is case-insensitive but passwords are case-sensitive. Check Caps Lock. Copy/paste the password from your password manager rather than re-typing it.
- **Likely cause B — lockout.** After ten failed attempts in 15 minutes, the account is locked for 15 minutes. Wait, then try again, or ask an admin to reset.
- **Likely cause C — account archived.** If you were recently offboarded and unarchived, you might need a password reset; old credentials no longer apply.
- **Likely cause D — session cookie cached.** Hard-refresh the login page with Ctrl-F5 or clear cookies for the domain, then retry.

Fix path: try the password manager, wait out any lockout, then ask an admin to reset if still failing.

Issue 02 — My daily list is empty

Symptom: you open the platform in the morning and your calling list contains zero LOs instead of the usual daily set.

- **Likely cause A — 5 AM job has not run yet.** Check the time. If it is before 5 AM local time, generation has not happened. Wait.
- **Likely cause B — Is CLR flag is off.** Ask an admin to verify your user record; if Is CLR is false, the algorithm skipped you.
- **Likely cause C — admin override in progress.** A pending override temporarily clears lists. Check the notifications bell for an override broadcast.
- **Likely cause D — no active LOs.** If every LO in the roster is archived or marked unavailable, there is nothing to assign. Unusual, but possible.

Fix path: verify the time, check your user record with an admin, look for an override notification.

Issue 03 — I cannot see the Admin menu

Symptom: a teammate says 'go to Admin → Users' but you do not see the Admin menu at all in your navigation.

- **Likely cause A — role is plain CLR.** The Admin menu only appears for Admin-CLR and Admin-only roles. Ask a current admin to update your role if that is appropriate.
- **Likely cause B — you logged in as a different account.** Check the top-right user badge; if the name is wrong, log out and sign in with the right email.
- **Likely cause C — cache.** After a role change, hard-refresh to pick up the new menu state; the role is read at login.

Issue 04 — An appointment I logged is missing

Symptom: you logged an appointment yesterday, but today it is not in your list.

- **Likely cause A — wrong date filter.** The Appointments page defaults to today; change the date filter to the day you logged it.
- **Likely cause B — it was deleted by an admin.** Check the audit log: filter to APPOINTMENT_DELETED, search your name. If an admin deleted in error, have them re-create from the audit entry's 'details' payload.
- **Likely cause C — it was attributed to a different CLR.** If the form was submitted while the wrong CLR was logged in, the row is under their name. Admin can edit the clr_id field to correct.

Issue 05 — EOD report did not send

Symptom: the scheduled 9 PM EOD email did not arrive at Scott's or Chris's inbox.

- **Likely cause A — Resend API key revoked or expired.** Check the audit log for EMAIL_SEND_FAILED and the Resend dashboard for rejected sends.
- **Likely cause B — from-domain DNS issue.** SPF or DKIM can lapse. Check wlc.it.com DNS records in GoDaddy.
- **Likely cause C — spam filter.** The message arrived but got filtered. Ask the recipient to check Spam and whitelist reports@wlc.it.com.
- **Likely cause D — recipients list empty.** If every recipient was removed, the job still runs but sends to no one. Restore recipients on the Email Settings page.

Issue 06 — NMLS task keeps re-assigning

Symptom: an NMLS task has been reassigned several times in two weeks; no one seems to be able to close it.

- **Likely cause A — CLRs keep marking it 'in progress' but not Complete.** The task only closes when marked Complete with a status.
- **Likely cause B — NMLS website was down during each check.** This is rare but happens; the CLR then delays and the task escalates. Wait for the site to be up and try.
- **Likely cause C — LO's NMLS ID on the roster is wrong.** The CLR cannot find the LO in the NMLS directory. Correct the NMLS ID on the LO record and re-assign.

Issue 07 — Algorithm weight change did not take effect

Symptom: you changed weights on the Algorithm Settings page, but today's list looks the same as yesterday.

- **Likely cause A — same-day change.** Weight changes apply to the next generation. Today's list was computed at 5 AM with the old weights. Wait until tomorrow or run an admin override to force today's regeneration.
- **Likely cause B — weights did not save.** The sum must equal 100% for Save to enable. If the Save button was disabled, the change was not committed.
- **Likely cause C — change was small.** A 2% shift in one weight rarely moves the ranking visibly. Use the Preview panel to see the predicted effect before committing.

Issue 08 — User cannot receive welcome email

Symptom: you created a new user but they never received the welcome email.

- **Likely cause A — corporate spam filter.** Most common. Have the user check Spam and Junk; the message usually arrives but is filtered.
- **Likely cause B — typo in the email field.** Edit the user record, correct the email, then trigger a password reset (which re-sends the welcome info).
- **Likely cause C — Resend transient failure.** Rare. Check the audit log for EMAIL_SEND_FAILED on that user; if present, click Reset password to re-send.

Issue 09 — Team stats look wrong

Symptom: the Team Stats page shows a CLR with zero appointments, but you know they booked several this week.

- **Likely cause A — wrong date filter.** Team Stats default to the current week; switch to a wider window.
- **Likely cause B — appointments logged under wrong CLR.** Filter the appointments table by LO and inspect the `clr_id`. If an admin logged on someone's behalf without switching accounts, the attribution is wrong.
- **Likely cause C — CLR is archived.** Archived users are excluded from Team Stats by default. Toggle 'Show archived' to include them.

Issue 10 — Team chat messages not arriving

Symptom: you send a chat message but other team members do not see it until they refresh.

- **Likely cause A — polling is paused in background tabs.** Browsers throttle background-tab polling. The message is queued and arrives on focus.
- **Likely cause B — client cache stale.** Hard-refresh once; the WebSocket-like polling resumes.
- **Likely cause C — network dropped.** If you see 'offline' in the bottom status bar, reconnect to Wi-Fi and wait for the reconnect banner.

Issue 11 — Override was confirmed but list did not change

Symptom: you completed the triple-confirmation override, but the lists look unchanged.

- **Likely cause A — weights were not changed before the override.** Override simply re-runs the algorithm with current weights. If nothing changed about the weights or the LO data, the output is (correctly) the same.
- **Likely cause B — you overrode for a date other than today.** The override only affects today. Past dates cannot be regenerated.
- **Likely cause C — CLR browsers not refreshed.** Each CLR's browser caches their list. They need to refresh to see the new assignments.

Issue 12 — LO is missing from the roster

Symptom: an LO you expect to see in the roster is not there.

- **Likely cause A — LO is archived.** Toggle 'Show archived' on the LO Roster page; archived LOs are hidden by default.
- **Likely cause B — name has a typo.** Search by NMLS ID or partial last name.
- **Likely cause C — LO was never added.** Confirm with the admin who was expected to add the LO. If they added the LO under a different account, check the audit log for LO_CREATED with the LO's name.

Issue 13 — LO availability schedule not honored

Symptom: the algorithm assigned an LO who you know is on PTO.

- **Likely cause A — PTO was not entered in the availability calendar.** The algorithm only knows what is on the calendar. Enter the PTO dates and the algorithm will skip the LO tomorrow.
- **Likely cause B — PTO entered after 5 AM.** Today's list was already computed. Either wait until tomorrow or run an admin override.
- **Likely cause C — availability weight is low.** If the Availability weight is near 0, other factors outweigh unavailability. Raise the weight if this is happening frequently.

Issue 14 — Boost did not move the LO to #1

Symptom: you boosted an LO but they are still ranked 3rd in the list.

- **Likely cause A — Boost weight is only 10%.** Even with boost, other factors still contribute. Two LOs with very high Days Since and strong Frequency can legitimately outrank a boosted LO.
- **Likely cause B — Boost applied after 5 AM.** Boost only affects the next generation. Today's list is already computed.
- **Likely cause C — Boost was applied to the wrong LO.** Check the LO Roster; if you search for 'John' you may have boosted a different John than intended.

Issue 15 — Audit log is missing an expected entry

Symptom: an admin swears they edited a user yesterday, but the audit log does not show the edit.

- **Likely cause A — filter excluded it.** Clear all filters on the History page and re-search by actor name.
- **Likely cause B — edit did not save.** A crashed browser tab can leave a form visibly 'saved' without actually committing. The audit log is the truth.
- **Likely cause C — action was performed by a different admin.** Memory is unreliable. Expand adjacent entries to see which admin actually did the edit.

Issue 16 — Bonzo password does not work after rotation

Symptom: a CLR pastes the password from CLR Center into Bonzo and it is rejected.

- **Likely cause A — rotation was done in Bonzo but CLR Center was not updated.** Re-enter the new password on the Bonzo Credentials page and Save.
- **Likely cause B — rotation was done in CLR Center but Bonzo was not updated.** The opposite mistake. Log into Bonzo directly and change the password there to match.
- **Likely cause C — leading or trailing space.** Copy-paste can include whitespace. Trim and retry.

Issue 17 — Railway deploy stuck

Symptom: the Railway dashboard shows a deploy stuck in 'Building' or 'Deploying' for more than 10 minutes.

- **Likely cause A — build failure masked as stuck.** Open the build log; failed npm installs can appear stuck because the dashboard reports last-log-line time, not progress.
- **Likely cause B — health check failing silently.** The server starts but `/api/health` returns non-200. Check the server log for errors.
- **Likely cause C — port mismatch.** `PORT` env var is ignored; the server listens on a hardcoded port. Verify `PORT=3000` is honored in `server/index.ts`.

Issue 18 — 'Something went wrong' generic error

Symptom: any page load or save produces a red toast that says only 'Something went wrong — please try again.'

- **Likely cause A — server is restarting.** Wait 30 seconds and retry. Restarts happen during deploys and on rare crashes.
- **Likely cause B — database is locked.** SQLite briefly locks during backups or long transactions. Retry after a few seconds.
- **Likely cause C — session expired mid-action.** The cookie is 14 days; if the user has been idle longer, they are logged out. Refresh and sign in.
- **Likely cause D — actual server error.** Check the Railway server log for a stack trace at the time of the toast. The error will identify the failing endpoint.

Issue 19 — PWA install prompt does not appear

Symptom: on a supported browser (Chrome, Edge, Safari iOS 16.4+), the 'Install app' prompt does not appear.

- **Likely cause A — already installed.** Check the OS app list; if installed, the prompt is suppressed.
- **Likely cause B — PWA criteria not met.** The app must be served over HTTPS, have a valid manifest, and a service worker. All three are satisfied in production; if you are testing locally they may not be.
- **Likely cause C — user dismissed the prompt.** Browsers remember dismissals for weeks. You can usually re-trigger from the browser's menu.

Issue 20 — 'CLR Center is slow' generic complaint

Symptom: a user says the platform 'feels slow today' with no specific page.

- **Likely cause A — user's network.** Ask them to try a different network. 90% of 'the platform is slow' reports trace to local Wi-Fi.
- **Likely cause B — server under load.** Rare at our scale. Check Railway's metrics dashboard for CPU and memory.
- **Likely cause C — large audit log query.** If the user opened History with no filters, loading thousands of entries can take a few seconds. Apply filters.

When none of these match

If the issue does not match any of the entries above, escalate through the support path in Appendix E. Capture these details before opening the ticket: the exact URL, the exact time, the signed-in user's email, the browser and version, and a screenshot of the error. With those five inputs, most issues resolve within an hour.

Keep this chapter current

Every genuinely new issue that takes more than thirty minutes to resolve deserves a new entry here. The chapter is a living document; adding to it is one of the highest-leverage forms of documentation, because the next person who hits the same problem will solve it in minutes instead of hours.

Diagnostic techniques

Beyond the specific issues catalogued in Chapter 33 proper, there are general diagnostic techniques that apply across many unknown problems. Knowing them turns 'I have no idea what is happening' into 'let me check a few things.' The techniques below are listed roughly in the order you should try them.

Technique 1: Reproduce the issue

Before investigating, confirm the issue reproduces. Ask the reporter to re-perform the action and describe what they see. Many 'issues' evaporate on the second attempt (transient network glitch, one-off server hiccup). If it does not reproduce, note the original report and move on; if it does, you have a real target to investigate.

Technique 2: Narrow the scope

Does the issue happen for every user or just one? Every browser or just one? Every page or just one? Every time of day or just at 5 AM? The narrower the scope, the more information you have. A bug that only affects Chris's account on Chrome after 3 PM tells you a lot; a bug that affects 'some users sometimes' tells you almost nothing.

Technique 3: Check the recent changes

Most bugs are introduced by recent changes. Git log the last five commits, the last deploy, the last weight change, the last schema migration. If a bug appeared around the same time as one of these, the cause is probably related to that change. Narrow your investigation there first.

Technique 4: Read the audit log

For any state-related issue, the audit log is your first stop. Did someone edit the affected record? Was there a weight change? A role change? An archive and restore? Filter by the relevant target and actor, scan the entries, look for anomalies. This step solves a surprising fraction of mysteries.

Technique 5: Read the server log

For any technical issue (5xx errors, slow endpoints, inexplicable behavior), the server log on Railway has the answer. Filter to ERROR lines around the time of the report. A stack trace is usually the fastest path from 'what is wrong' to 'what to fix.'

Technique 6: Check the database directly

For any data integrity question, bypass the UI and query the database directly from the Railway shell. 'Is the row there? Is the field what the user says it is? Are there duplicates?' SQLite makes this easy with one-line queries.

Technique 7: Eliminate the browser

Browser-specific issues are common. Have the user try a different browser. Have them clear cache (or hard refresh with Ctrl-F5). Have them try an incognito window (disables extensions). If the issue disappears in one of these conditions, the browser state is the cause and can be isolated from there.

Technique 8: Ask for details

Users report symptoms, not causes. 'It is broken' gives you nothing; 'the Log Appointment button returns an error when I click it with an LO who is archived' gives you everything. Do not be shy about asking for specifics. A minute of clarifying questions saves an hour of investigation.

Technique 9: Small reproducible test

If an issue is complex, reduce it to the smallest reproducible case. 'This complicated 12-step workflow breaks' becomes 'these two API calls, in this order, fail.' A small repro is what makes a bug fixable; a large repro is what makes it seem impossible.

Technique 10: When stuck, write it up

After 30 minutes of being stuck, stop and write a description of what you are investigating: what the symptom is, what you have checked, what hypotheses you have eliminated, what remains. Often the act of writing the description surfaces the answer. If it does not, the description is the start of a handoff to the next person and is not wasted effort.

The principle of least surprise

When investigating, assume that the system does what it says it does. The algorithm does exactly what Chapter 16 describes. The database contains exactly what Chapter 27 documents. The email system works exactly as Chapter 30 explains. Most 'surprising' behaviors are actually the system behaving as designed, interpreted through a misunderstanding of what the design is. Double-check the expected behavior before assuming a bug.

Escalation criteria

You should escalate to engineering (Ethan) when any of the following is true:

- The issue involves data loss or corruption.
- The issue affects authentication or authorization (a user can see something they should not, or cannot see something they should).
- The issue affects production availability (the platform is down or intermittently unresponsive).
- You have spent an hour on investigation without progress.
- The root cause appears to be a code defect, not a data or config issue.

Operational issues (weight tuning, user management, LO roster questions) should go to Chris first; code and platform issues should go to Ethan first. When unsure, ask in team chat and the right person will pick it up.

Operational Runbooks

A runbook is a pre-written procedure for a recurring or predictable situation. When the situation arises, you do not think about the steps — you execute them. The runbooks below cover the situations that occur most often at CLR Connection Center. Keep this section handy; for the first six months of owning the platform, you will reach for it regularly.

Runbook A: The weekly Monday start-of-week review

At the start of each Monday, before the team's day ramps up, spend 15 minutes on this routine. It catches anything that drifted over the weekend and sets the week up cleanly.

1. Open Admin → History. Scan Saturday-and-Sunday entries. Anything unexpected?
2. Open Admin → Users. Scan the Last Login column. Any CLR who has not logged in since before Friday? Follow up.
3. Open Admin → Algorithm Settings. Confirm the six weights are as you left them. If any were changed, note who and why.
4. Open Admin → NMLS Tasks. Review the calendar for the week. Any day with 4+ checks due? Plan distribution.
5. Open Admin → Email Settings, click Send test email. Confirm it arrives within 2 minutes.
6. Open Team Stats with a one-week window. Any CLR trending down? Schedule a check-in.
7. Open LO Roster. Toggle 'Show incomplete.' Any roster entries missing key fields? Add them to this week's to-do.

Runbook B: The Friday-afternoon EOD review

Friday afternoons are the right time to set up the following week. Thirty minutes, roughly between 3 and 4 PM.

1. Review all pending NMLS tasks. Any due in the next seven days that are still open? Nudge the assignees.
2. Pull the week's EOD reports. Calculate team totals. Note any CLR with notably high or low numbers.
3. Open Admin → History, filter to BOOST or DEPRIORITIZE events this week. Review whether the boosts and deprioritizations reflected the week's business focus.
4. Glance at the Resend dashboard. Bounce rate below 2%? Complaint rate zero or near-zero? No alerting from Resend? Good.
5. Check Railway's metrics dashboard. Any CPU or memory anomaly? Any failed deploys? Any errors in the log summary?
6. Prepare for Monday: draft any weight changes you plan to announce, any user additions, any strategic boosts. Do NOT apply them on Friday evening — wait for Monday morning so Monday's list reflects them from the start.

Runbook C: Handling a complete platform outage

Rare — has not happened in the platform's deployed history — but the procedure should exist for when it does.

1. Confirm the outage by attempting to load the platform URL from two different networks (e.g., office Wi-Fi and cellular). If both fail, the outage is real and not local.
2. Check status.railway.app for platform-wide issues. If Railway is degraded, the outage is outside your control; post an update to team chat and wait.
3. If Railway is healthy, open the Railway dashboard for our project. Look at the service status: is it running? When was the last deploy? Any crash loops?
4. Open the server log. Look for errors or startup failures. If the container is crashing on startup, the most recent deploy likely introduced a regression — click Rollback in the dashboard.
5. If rollback succeeds and the service stabilizes, post an update to team chat: 'Platform was down from X to Y; rolled back to previous build. Investigation ongoing.'
6. If rollback fails or the service does not stabilize, escalate to Ethan. This is the threshold for waking him up on a weekend.
7. Once recovered, write a short post-mortem: what happened, timeline, root cause, preventative measures. Share with Chris and Ethan.

Runbook D: Onboarding a brand-new admin

When a new team lead is promoted, they need to learn the admin side of the platform. A structured onboarding takes a week but creates a genuinely confident admin at the end.

1. Day 1: change their role to Admin-CLR in the Users page. Walk them through the Admin menu appearing in their nav. Give them this manual; ask them to read Part 5 end-to-end before day 2.
2. Day 2: pair with them on a real user creation. Have them type the fields while you watch. Answer questions. Verify the welcome email fires and the audit entry lands correctly.
3. Day 3: pair on an algorithm-weights inspection. Do not change anything; just look. Have them describe what each weight does in their own words. Correct any misunderstandings.
4. Day 4: pair on a test NMLS check, start to finish. Have them narrate the steps. Review their notes and their completed form.
5. Day 5: pair on an audit-log investigation. Present them with a small artificial mystery ('a user was edited yesterday — who did it and what did they change?') and have them solve it using the History page.
6. Week 2: have them shadow you, or take the lead with you shadowing, on any real admin task that comes up. Offer feedback.
7. End of week 2: remove the training wheels. They are now a full admin. You remain available for questions but they no longer need shadowing.

This schedule is aggressive; a slower pace is fine if the new admin needs it. What matters is that by the end, they have performed each of the main admin tasks at least once with a peer watching.

Runbook E: Weekly data integrity spot-check

Every few weeks, spend 20 minutes spot-checking data integrity. This catches issues that the platform's automated checks might miss.

1. Open the LO roster. Sort by last_call date. Any LO with no last_call for more than 60 days? Either they are genuinely stale (investigate why) or their record is wrong.
2. Open Team Stats. Calculate sum-of-individual-CLRs versus team-total. They should match exactly. If they differ, a data anomaly exists.
3. Open Audit Log, filter to action=APPOINTMENT_DELETED for the past month. Any unusually large number? Any deletions by a user who is not admin?
4. Open Algorithm Settings. Manually add the six weights. They should sum to 100%. If they do not, the save logic has a bug.
5. Query the database (via Railway shell): `SELECT COUNT(*) FROM users WHERE is_clr AND NOT archived;`. This count should match the team's actual number of active CLRs.
6. Open NMLS Tasks. Any task with due_date more than 7 days in the past that is still open? That is a bug — the escalation cron should have caught it.

The value of spot-checks

Automated monitoring catches most problems. Spot-checks catch the problems that automation missed — often because they are new kinds of problems that the automation was not designed for. Twenty minutes every two weeks is the budget that keeps the platform at a level of cleanliness the team feels in its daily work, even if they cannot articulate why.

Runbook F: Handling a confused user request

Occasionally a user — CLR, admin, or executive — asks for something that seems confused: 'can you make the algorithm prefer LOs in California?' or 'can we delete all the audit logs older than 30 days?' or 'can we let CLRs see Bonzo credentials?' These requests often trace to a valid underlying goal poorly expressed. Respond with curiosity, not dismissal.

1. Ask what problem they are trying to solve. 'What is driving the California question — is there a state-specific campaign coming up?'
2. Understand the real goal. 'We want to focus on California this quarter because of the housing market dynamics there.'
3. Consider whether the platform's existing tools solve the goal without the asked-for change. 'We already have per-LO boost; we could boost every California LO for the quarter.'
4. Propose the better path. 'Let's boost the California LOs. You will see them rise in everyone's lists starting tomorrow. At the end of the quarter, remove the boost.'
5. If the existing tools genuinely do not solve the goal, document the request as a feature idea and thank the person for raising it.

This approach — understand, translate, propose, document — turns friction into collaboration. Over time, users learn to bring their real goals instead of pre-packaged solutions, and the quality of requests improves.

Complete CLR Onboarding Guide

A new CLR's first two weeks on the platform set the tone for their next two years. A disciplined onboarding produces a CLR who trusts the platform, understands the algorithm, and contributes to the team's rhythm from day one. A hasty or ad-hoc onboarding produces a CLR who treats the platform as a necessary evil and whose habits take months to recalibrate. This guide walks through the two-week curriculum that the WCL team has converged on over time.

Week 1, Day 1 (Monday)

- Meet the team lead. Brief introduction to WCL's Connection Lead Representative role in plain terms: 'you call loan officers, listen for their pipeline, book appointments or warm-transfer to our LOs when they're ready.'
- Receive the welcome email. Sign in for the first time. Choose a personal password.
- Tour the platform with the team lead: Dashboard, Calling List, Appointments, EOD, Team Stats, LO Roster. No calls today; just orientation.
- Read Chapters 5-8 of this manual (Dashboard, Calling List, Appointments, EOD).

Week 1, Day 2 (Tuesday)

- First calling day. Your list is populated (the 5 AM job includes you).
- Shadow a veteran CLR for the first hour. Watch how they use the list, how they open Bonzo, how they log appointments.
- Make your own first call. The team lead is on hand for questions.
- End-of-day: submit your first EOD report. The template walks you through every field.
- Read Chapters 9-10 (Team Stats, Team Chat).

Week 1, Day 3 (Wednesday)

- Full day of calling on your own. Team lead available for questions but not hovering.
- Start building a sense of the LOs: who takes calls vs. voicemail, who transfers immediately, who wants a nurture sequence.
- Review your first-day EOD with the team lead. Discuss what felt easy and what felt hard.
- Read Chapter 16 (How the Algorithm Works). This is the most important chapter for understanding why your list looks the way it does.

Week 1, Day 4 (Thursday)

- Full calling day.
- Ask your team lead to walk through your morning list with you: for each LO, why did they rank where they did? This is the exercise that builds algorithm fluency.
- Read Chapter 17 (Daily Generation).

Week 1, Day 5 (Friday)

- Full calling day.
- Review the week's EODs with the team lead. Identify patterns — which LOs you connected with, which you missed, which fell into what outcome category.
- First NMLS task may have landed this week; if so, walk through the check with the team lead.
- Read Chapter 23 (NMLS Compliance).

Week 2, Day 6 (Monday)

- Full calling day. You should feel noticeably smoother than week 1.
- Attend the team's Monday stand-up (15 minutes, typically at 9:00). Introduce yourself if not already.
- Read Chapter 11 (LO Roster). Understanding the roster as a data source is the foundation for understanding why the algorithm picks what it picks.

Week 2, Day 7 (Tuesday)

- Full calling day.
- Pair with a different veteran CLR for half an hour. Different CLRs have different styles; exposure to more than one builds your toolkit.
- Read Chapters 18 and 24 if you will eventually become an Admin-CLR. Otherwise, skim them for awareness.

Week 2, Day 8 (Wednesday)

- Full calling day.
- Check in with the team lead on progress. Any friction points? Anything about the platform you'd change?
- Write down your first observation about the team's rhythm — something you've noticed that isn't obvious from the manual. Share it at the next stand-up.

Week 2, Day 9 (Thursday)

- Full calling day.
- Your first NMLS check if not already completed. The team lead should be hands-off; you have the manual and the platform guides.
- Read Chapter 33 (Common Issues Reference). Familiarity with the troubleshooting chapter means you can self-resolve minor issues rather than escalating.

Week 2, Day 10 (Friday)

- Full calling day. End of training period.
- Team lead debriefs with you: what went well, what you'd change, what your goals are for next week.
- Celebrate — two weeks in, you are a fully functioning CLR.

Milestones and expectations

Milestone	Target timing	Signal it is working
First login	Day 1	Welcome email received, password chosen
First call	Day 2	Confident dial, calm tone on the phone
First EOD submission	Day 2 evening	Every field filled thoughtfully
First appointment booked	Days 3-5	Details captured in Bonzo and platform
First transfer	Days 3-10	Warm handoff, LO accepts lead
First NMLS check	Within week 2	Form filled, screenshot attached
First solo-run day	Day 6	Full day without lead intervention
First stand-up contribution	Day 6-10	Observation or question shared

These targets are aggressive but realistic. Most CLRs hit them within a day or two of the targets. A few take a little longer; that is also fine. What matters is the trajectory — is the CLR getting more fluent, more confident, more independent each day?

Common first-two-week pitfalls

- **Over-using the team lead** — every small question gets escalated. Encourage self-service with the manual.
- **Under-using the team lead** — the new CLR is too proud to ask, gets stuck, wastes time. Explicitly invite questions.
- **Fighting the algorithm** — 'the list is wrong; I want to call different LOs.' The algorithm is right unless proven otherwise; trust it for two weeks, then form an opinion.
- **Skipping EOD** — 'I was too busy; I'll do it tomorrow.' The habit forms in week 1; skipping once leads to skipping often.
- **Not reading the manual** — 'I'll figure it out as I go.' Reading the core chapters saves weeks of friction.

Incident Response Guide

An incident is any event that materially affects the platform's availability, correctness, or trust. Outages are the obvious example; less obvious are data corruption, security compromises, and reputational damage from a misfired communication. This section walks through how we classify incidents, how we respond to each class, and how we conduct the post-mortem.

Incident severity levels

Level	Definition	Example
SEV-1	Platform unavailable or data corrupted	Full outage; DB file lost
SEV-2	Major feature broken or data inconsistent	Algorithm mis-assigns list
SEV-3	Non-critical feature broken	Team chat lagging
SEV-4	Minor defect	Typo on a settings label

SEV-1 response

1. Immediately post in team chat: 'SEV-1 incident — [short description]. Investigating.' This tells the team to stop depending on the platform.
2. Assemble: Chris and Ethan in a shared chat or call. Anyone else who is relevant (the affected team lead, for instance).
3. Triage: what is wrong, what is the scope, what is the probable cause?
4. Stabilize: restore service. Rollback a bad deploy, restore a database backup, rotate a compromised secret. The goal is to stop the bleeding, not to fix the root cause yet.
5. Communicate: post updates every 15 minutes until resolved, even if the update is 'still investigating.'
6. Resolve: confirm the platform is back to normal. Ask two people to independently verify.
7. Post-mortem: within 48 hours, write a post-mortem document and share it.

SEV-2 response

1. Post in team chat: 'SEV-2 issue — [description]. Team may see degraded behavior.'
2. Engage the relevant owner (Chris for operational, Ethan for code).
3. Fix within 24 hours. Roll back if the cause is a recent deploy; patch forward if not.
4. Post-mortem optional but encouraged for SEV-2s that reveal a new class of issue.

SEV-3 and SEV-4 response

Treat as regular work. Log in the issue tracker, prioritize against other work, fix in the next day or two for SEV-3s and the next week or two for SEV-4s. No-one needs to be paged; no-one needs to post emergency updates. Communication happens in the normal cadence.

The post-mortem format

A good post-mortem is short, honest, and blame-free. The WCL format has five sections:

1. **Summary:** one sentence describing what happened.
2. **Timeline:** bullet list of every event with its timestamp, from the first symptom to the final resolution.
3. **Impact:** who was affected, for how long, and in what way.
4. **Root cause:** the technical or procedural reason the incident occurred. Usually a single sentence or two.
5. **Prevention:** the specific changes (code, process, monitoring, training) that will reduce the likelihood or impact of a recurrence. Each action item has an owner and a date.

A post-mortem that focuses on blame — 'Ethan deployed the bug' — misses the point. The right framing is 'the platform allowed a bug to be deployed without catching it; here is the specific test we will add so that this class of bug is caught next time.' Systems, not people.

After-action review

Every SEV-1 and SEV-2 gets a 30-minute after-action review with the full engineering side of the team (for CLR Center, this is just Chris and Ethan). The review walks the timeline, discusses what went well, what went poorly, and what to change. Attendance is mandatory; the review is not optional.

The review is not the post-mortem. The post-mortem is a written document produced within 48 hours. The review is a conversation that happens within a week. Both are valuable; both are required.

Security incidents

Security incidents — a credential leaked, an unauthorized access, a suspected intrusion — follow the same severity ladder but with one extra step: rotate all related secrets before anything else. This is the only case where stabilization comes before triage. Rotate `SESSION_SECRET`, rotate `RESEND_API_KEY`, rotate the Bonzo password, and — for the really bad cases — force every user to re-authenticate. Then investigate.

Speed beats perfection in SEV-1

When the platform is down, the right action is often the imperfect one that restores service. A rollback to a 2-week-old build that works is better than a perfect fix that takes 4 hours to ship. You can always move forward again once the team is unblocked. The cost of the team being blocked exceeds the cost of a temporary regression.

Scaling the Team and the Platform

CLR Connection Center was built for a team of roughly a dozen CLR's. As WCL grows, the platform will need to grow with it. This section describes how we think about growth — both in terms of people and in terms of load — and what changes at each scale threshold.

Team sizes and what changes

Team size	Platform changes needed	Operational changes needed
1-6 CLR's	None (smaller than our baseline)	One admin doubles as lead
7-15 CLR's	Current baseline	One lead, one backup lead
16-30 CLR's	Revisit default weights	Two leads, structured stand-ups
31-60 CLR's	Algorithm batching; per-region lists	Multiple leads, daily sync
61-100 CLR's	Sharding or Postgres migration	Regional team structure
100+ CLR's	Full rearchitecture likely	Professional ops team

Our current baseline (7-15 CLR's) is comfortable. Growth to 30 is manageable without major platform changes. Growth past 30 starts to stress the assumptions; growth past 60 requires real engineering investment.

Signals that the platform is starting to strain

- Daily generation takes more than 10 seconds.
- Team chat becomes too noisy to follow.
- EOD email exceeds a single viewable screen.
- Team Stats loads in more than 2 seconds.
- NMLS task volume creates scheduling clashes the smoothing cannot resolve.
- Audit log queries take seconds rather than milliseconds.
- Railway's memory usage trends upward without obvious cause.

What grows with the team, what does not

Some platform dimensions scale linearly with team size; others scale super-linearly; others are largely fixed. Understanding the difference helps plan investment.

Dimension	Scaling	Notes
Users table size	Linear with team	Tiny; not a concern
LO roster size	Linear with business	Independent of team
Assignments per day	team × lists/day	Scales linearly
Appointments per day	Linear with team	Scales linearly
Team chat volume	Super-linear	Conversation complexity grows
Algorithm compute	team × LOs	Multiplies; eventual concern
Audit log size	Linear with activity	Grows but cheap
EOD email complexity	Linear with team	Display becomes unwieldy

The dimensions worth watching as the team grows: team chat volume (add channels), algorithm compute (optimize when it exceeds a few seconds), and EOD email complexity (move from email to a dashboard link when the table becomes too big).

When to migrate to Postgres

SQLite serves us well at current scale. The migration to Postgres becomes appropriate when one of the following is true:

- Write contention becomes visible — two users editing at the same time occasionally get a 'database is locked' error. We have not hit this yet.
- The database file size exceeds a few gigabytes, at which point backup and restoration become operationally cumbersome.
- We want true concurrent multi-writer support for a specific feature.
- We want managed backups, replication, or point-in-time recovery that Postgres services provide out of the box.

When the time comes, the Drizzle ORM makes the migration relatively painless. The schema is portable; most queries work identically. The real work is operational — running both databases in parallel during cutover, verifying data equivalence, flipping the connection string. Budget a week.

Feature evolution principles

As the team grows, new feature requests will arrive. A few principles for evaluating them:

- **Prefer existing tools over new features.** If the goal can be met with weights, boost, or availability adjustments, do that first.
- **Measure the pain.** A feature that saves 30 seconds per CLR per day, multiplied by 12 CLRs times 250 workdays, is 25 hours per year. That is a real number. A feature that vaguely 'improves' something unmeasurable is not.
- **Prefer removing features over adding.** The platform's simplicity is its primary asset. Every feature added is a future source of bugs, questions, and onboarding friction.
- **Watch for 'my favorite CLR' bias.** Features proposed by the most articulate CLR can dominate a roadmap and shape the platform toward their needs. Solicit input from the quieter CLRs too.

The long view

CLR Connection Center is designed to last. Not forever — software of this kind typically has a five- to ten-year useful life before deeper rewrites are warranted — but long enough that the team can plan around it rather than around it breaking. Keep the rhythms in this manual, invest modestly in the platform's health each quarter, and it will serve the WCL CLR team well into the next phase of growth, whatever that looks like.

Field Notes & Lessons Learned

This section is a practitioner's notebook. It captures the small decisions, course corrections, and hard-won insights that accumulated during the platform's first year of live operation. None of these items rises to the level of a chapter on its own, but read together they represent the texture of running CLR Connection Center in production. A new operator who absorbs these notes will skip six months of learning curve.

Lesson 1 — Weights drift slower than you expect

Our initial hypothesis was that the algorithm weights would need near-constant tuning: seasonal adjustments, quarterly re-balancing, responses to changing pipeline dynamics. The reality was the opposite. The weights landed on their current distribution — Days Since 30%, Call Frequency 25%, Availability 20%, Boost 10%, Tier 5%, 90-Day Transfers 10% — within the first month of live use, and have not moved meaningfully since.

The reason, in hindsight, is that the weights encode a stable philosophical position ("freshness matters most, effort matters almost as much, tier is a tiebreaker") and philosophical positions do not drift monthly. They drift when the team's mission changes — which is rare. If your weights are drifting every two weeks, the real problem is probably not the weights; it is an underlying data quality issue that is making the algorithm behave unpredictably. Fix the data, not the weights.

Rule of thumb

If you find yourself adjusting weights more than once a quarter, stop and audit the input data instead. Nine times out of ten, the weight you are fighting is compensating for a stale Bonzo field or a mis-configured LO row.

Lesson 2 — Override is a feature, not a workaround

Early in the rollout, we observed a pattern we called "override drift": a CLR would override the algorithm two or three times a day, each for a plausible reason, and within weeks the override was the default behavior. The algorithm was still computing scores, but the CLR was effectively cherry-picking. That defeats the purpose of the platform.

We addressed this in two ways. First, we made override require a typed justification — not a dropdown, not a radio button, but a sentence or more of free text. Second, we surface override frequency in Team Stats, visible to admins. The combination of friction plus visibility was sufficient. Override frequency dropped from an early peak of around eight per CLR per week to a steady-state of one or two. The ones that remain tend to be legitimate: a hot lead, a scheduled callback, an LO who specifically asked for a particular CLR.

The lesson is not "override is bad." It is that any feature that lets the human bypass the algorithm needs to be slightly uncomfortable to use, or it becomes the algorithm.

Lesson 3 — Tier matters less than you think

The Tier field (1 = highest priority, 3 = lowest) was introduced with a 10% weight. We expected it would matter a lot: Tier 1 LOs would cluster at the top, Tier 3 LOs at the bottom, with clean separation. In practice, the other factors (freshness, effort, availability) dominated so completely that Tier barely moved the ranking. A Tier 3 LO who had not been called in ten days still ranked above a Tier 1 LO called yesterday.

We reduced Tier's weight to 5% during the first month, and we have not touched it since. The conclusion: tier is a tiebreaker, not a sorting key. It earns its place in the algorithm by breaking ties between LOs who are otherwise equivalent — but it should never be the deciding factor. If you feel tempted to raise Tier weight to make Tier 1 LOs "always come first," you are asking the platform to do something it was not designed to do. Use Boost instead, and use it sparingly.

Lesson 4 — The 90-day transfer rule is emotional, not mathematical

Of all the algorithm factors, the 90-day transfer rule (penalizing LOs who have recently transferred away from a CLR) has the least rigorous quantitative justification and the most consistent qualitative one. The CLRs asked for it. The math says it should not matter — a transfer is a transfer, and the LO needs the same outreach regardless. The emotional reality is that calling an LO who just left your book feels bad, and it shows in the call quality.

We honored the team's intuition. The 10% weight on 90-day transfers is not there because the algorithm needs it; it is there because the team needs it. That is a legitimate reason. Algorithms serve the humans who use them. When the humans tell you something matters, the algorithm should reflect that, even when the abstract math disagrees.

Lesson 5 — Morning and afternoon are different jobs

We considered generating a single daily assignment list. We built it that way initially. Within two weeks the CLRs asked us to split it into morning and afternoon lists, each with its own availability window and its own re-ranking. The reason, which seems obvious in retrospect: an LO who does not answer at 9 AM is not proof that the LO is unreachable; it is proof that the LO is not at their desk at 9 AM. At 2 PM they might answer on the first ring.

We added the afternoon regeneration, with a fresh read of Availability and Days Since Last Call (which resets any LO called that morning). The morning list surfaces the freshest, highest-availability LOs. The afternoon list surfaces the ones who were missed in the morning, plus anyone who became newly available during the day. Two lists, two chances, same algorithm.

Lesson 6 — Railway blocks SMTP; plan around it

Our first email implementation used standard SMTP. It worked locally; it failed silently on Railway. After a frustrating afternoon of debugging we learned what the Railway documentation does not emphasize: the platform blocks outbound SMTP connections entirely. This is a spam-prevention measure and it is not negotiable. The fix is to use a transactional email API (we chose Resend) that sends over HTTPS, which Railway allows.

The broader lesson is that platform constraints shape architecture in ways that are not obvious until you hit them. When choosing a hosting platform, read the fine print on outbound network restrictions, filesystem persistence, background job support, and scheduled task limits. These footnotes determine which tools you can use, and the cost of discovering them mid-build is high.

Production note

Our Resend-based email pipeline has delivered thousands of daily reports without a missed send. If you are tempted to "fix" it by switching back to SMTP, do not. The current setup is boring, reliable, and free within current volumes.

Lesson 7 — The audit log earned its keep in week three

In week three of live operation, a CLR believed that an admin had secretly adjusted Boost values to favor a specific LO. The admin denied it. Without the audit log, this would have turned into a personnel problem. With the audit log, the dispute was resolved in five minutes: the log showed no Boost changes had been made that week. The feature the admin had actually touched was the email recipient list, which was unrelated to the dispute.

We had built the audit log partly out of principle and partly because it was easy. We did not expect it to matter so quickly, or to matter in that way. The lesson: any feature that writes permanent records of human action is doing two jobs simultaneously. It documents what happened for debugging. It also protects the humans who touched the system from being blamed for things they did not do. The second function is usually the more valuable one.

Lesson 8 — NMLS compliance is easier than licensing suggests

When we first read the NMLS compliance requirements, we were nervous. Sixty-day call cadences, per-state licensing, seven-day escalations — it read like a legal minefield. We over-engineered the first version, with state-by-state exception rules, a license expiration tracker, and regulatory report exports.

The compliance team took one look at it and told us we had built four times what they needed. What they actually cared about was: (1) is every LO on a 60-day cadence; (2) is there proof of the last contact; (3) does the system escalate before the cadence is missed. That was it. We deleted the state-rule engine and the license tracker. The remaining implementation is simple and it satisfies the audit.

The lesson generalizes beyond compliance: when you do not know the exact requirements, your imagination fills in the gaps with worst-case scenarios. Ask the actual stakeholder what the actual requirement is. The answer is usually two or three concrete items, not the encyclopedia you built in your head.

Lesson 9 — One admin is the right number

The platform technically supports multiple admin-level users, but we operate with exactly one true admin (Chris) plus a few admin-CLR hybrids (operators who are also active CLRs). We considered adding more admins for redundancy. We decided against it. The reason: the admin job is mostly tuning and credential rotation, both of which are high-context activities. Splitting that work across multiple admins produces subtle inconsistencies — one admin adjusts Boost for a reason the other admin does not know, and a week later they collide.

A single admin with a deputy who has read-only access plus the ability to act in emergencies is the right shape for a platform of this size. If the team grows to twenty-plus CLRs across multiple shifts, revisit this. For twelve LOs and a handful of CLRs, one hand on the wheel is correct.

Lesson 10 — Boost should decay, and we did not make it decay

Boost is a manual +N adjustment an admin can apply to an LO's score to push them up the list temporarily. We did not build automatic decay. We probably should have. The current behavior is that a Boost stays in place until an admin explicitly removes it, which in practice means boosts accumulate. Every quarter we audit the Boost values and remove the ones that are no longer relevant.

If we rebuilt the feature, Boost would decay by one point per week automatically, reaching zero after N weeks and being logged in the audit trail each time. This would eliminate the quarterly audit and would match how admins mentally model the feature: "push this LO up for a couple of weeks." Future work, documented here so whoever implements it knows why.

Future enhancement

Add automatic weekly decay to Boost values ($-1/\text{week}$). Log each decay in the audit table. Remove at zero. Estimated effort: half a day. Risk: low. Benefit: eliminates the quarterly Boost audit and reduces administrator cognitive load.

Lesson 11 — Daily email beats a dashboard

The platform has a dashboard. It works well. Almost nobody uses it. What stakeholders actually engage with is the daily email summary, delivered to their inbox at a predictable time with the numbers already in it. The reason is not that the dashboard is bad; it is that an inbox is where people already are, and a dashboard is a place people have to remember to visit.

This was a course correction. We built the dashboard first and treated the email as a secondary feature. Usage told us we had it backwards. We now optimize the email first — clear, concise, designed for three-second scanning on a phone — and treat the dashboard as the place you go when the email raises a question.

Lesson 12 — A small team is an engineering advantage

Twelve LOs and a handful of CLRs means the platform operates at human scale. We can fit the entire active LO roster in our heads. We can call any CLR by name. When a feature feels off, we ask three people and get ground truth in an afternoon. That advantage shapes the architecture: we did not need to build abstractions for LO groups, regions, or team hierarchies, because there is one team.

If the platform ever needs to expand — a second WCL office, a partner firm, a different product line — a lot of the simplifications we made will need to become proper abstractions. Role hierarchy instead of two-role system. Group permissions instead of per-user permissions. Multi-tenant data model. These are not bad changes; they are natural responses to scale. But they should not be pre-built before the scale arrives, because pre-built abstractions for scale that never comes become deadweight the team has to carry forever.

Lesson 13 — Fail loudly, recover automatically

Our error handling philosophy converged on a simple rule: every failure should produce a visible artifact (a log entry, an email to the admin, a red indicator on the dashboard) AND the system should attempt to recover without human intervention. Silent failures are the worst category because they erode trust without giving anyone a chance to respond. Loud failures that self-recover are the best category because they demonstrate the system is healthy.

Concrete examples: the Bonzo sync retries three times with backoff before giving up and emailing the admin. The email-send function logs the response code on every attempt, so we can see partial failures. The scoring job, if it crashes, falls back to yesterday's scores rather than leaving the list empty. None of this is fancy; all of it is the difference between a platform that feels reliable and one that occasionally makes the team's day worse.

Lesson 14 — Documentation is a conversation, not a monument

This manual is the third version of the documentation. Version one was a README in the repo. Version two was a Notion workspace. Version three is what you are reading: a structured PDF that lives alongside the code. Each version served its era and got replaced as the platform grew.

We expect a version four eventually. Maybe a searchable web-rendered version of this content, maybe an interactive walkthrough integrated into the product, maybe something we cannot imagine yet. The important thing is that documentation is a conversation between the people who built the thing and the people who will operate it — and conversations need to be refreshed as the participants change.

Lesson 15 — What we would do differently

For the benefit of future builders, a short list of choices we would revisit:

- **Boost decay.** As noted in Lesson 10, we would bake in automatic weekly decay from day one.
- **Bonzo sync granularity.** We sync on a daily schedule. Finer-grained sync (hourly for active LOs) would eliminate a class of freshness complaints.
- **Dashboard-first, not email-first.** We built the dashboard first and retrofitted the email. If we rebuilt, we would design the email as the primary UX and the dashboard as the drill-down.

- **Audit log retention.** Current retention is unlimited. Six months would be sufficient and would simplify the database.
- **Role hierarchy.** The two-role system (CLR vs admin) has served us, but admin-CLR hybrid users required awkward feature flags. A three-role system (viewer, operator, administrator) would be cleaner.
- **Credential rotation reminders.** Automated quarterly email reminders to rotate Bonzo and Resend keys, instead of relying on a calendar entry.
- **Mobile preview of the email.** The email renders correctly on mobile but we have no automated test that enforces it. A screenshot-diff test in CI would catch regressions early.

Closing reflection

Every piece of software is a collection of decisions made under uncertainty. The decisions that turn out well were not necessarily smarter at the moment they were made; they were just the decisions that happened to match the reality that unfolded afterward. The decisions that turn out poorly were usually reasonable-looking at the time. There is no way to tell them apart in advance.

What you can do is work in a way that makes both kinds of decisions easy to revisit. Small commits. Clear log messages. Reversible migrations. Documentation that explains not just what the code does but why. An audit log that remembers who changed what. A team that trusts the system enough to flag problems early. The combination of those practices is what lets a platform keep improving instead of accumulating scar tissue.

CLR Connection Center was built with those practices, and it has rewarded them. The platform is small, legible, and has gotten better every quarter since it launched. That trajectory is the real product. The features in the manual are the visible surface; the working rhythm underneath is what makes the features hold up.

Postscript — how this manual was written

This manual was drafted with Chris reading every chapter for operational accuracy and Ethan reading every chapter for technical fidelity. Where the two authors disagreed, we resolved it by reference to what the platform actually does, not what either of us wished it did. That rule kept the manual honest and is the rule any future maintainer should adopt.

We considered and rejected several structural alternatives. A chronological narrative ("how the platform evolved") would have read more like a memoir than a reference. A strictly alphabetical reference would have been unreadable for anyone not already expert. The current structure — roles first, then walkthroughs, then internals, then troubleshooting, then appendices — is a compromise that works for both the new reader and the returning reference-seeker. No structure is perfect; this one is adequate for both primary audiences.

If you find an error, a gap, or a section that has aged poorly, fix it. The manual lives in the same repository as the code. Commits to the manual are just as welcome as commits to the source, and are arguably more valuable in the long run: code decays when it is unused, but documentation decays when it is unread. Read this document. Argue with it. Improve it.

End of Field Notes. Appendix and closing pages follow.

Frequently Asked Questions

The questions below come up often enough that preempting them here saves everyone time. Answers are short; follow the chapter references for depth.

Q: Why does my list sometimes have the same LO two days in a row?

A: This is rare but legitimate when an LO has an especially strong Days Since score after being ignored for weeks, plus high Boost or Availability. The algorithm is ranking them at the top on successive days because the signals still point there. The cure is either to log an interaction (which resets Days Since) or to Boost a competing LO. See Chapter 16.

Q: How do I 'snooze' an LO for a week?

A: There is no explicit snooze feature. The practical equivalent is to update the LO's availability calendar to mark them unavailable for the week. The Availability factor will push them down the ranking, and no CLR will be assigned to call them. See Chapter 14.

Q: Why can't I delete an appointment I logged by mistake?

A: CLRs can edit their own appointments; deletion is an admin action. This is deliberate — the audit log for deletions is more consequential than for edits, so we route it through admins to ensure the right context is captured. Ask your team lead; they will delete for you.

Q: Does the algorithm learn over time?

A: Not in the machine-learning sense. The algorithm's six weights do not change automatically based on outcomes; an admin must explicitly tune them. This is deliberate — opaque auto-tuning would make the algorithm harder to reason about and harder to trust. The algorithm does, however, respond to the data it sees: an LO who stops being called eventually rises in rank because of Days Since.

Q: Can I see which LOs I have called, across all CLRs?

A: Yes — the LO Roster page, when clicking into an LO, shows the call history across the team, not just your own. This is useful for 'has anyone talked to Bill recently' investigations.

Q: Why does the EOD email always arrive at 9:10 PM instead of 9:00 PM?

A: The EOD cron runs at 21:00 local time and takes several minutes to compile the report (it is a full aggregation over the day's data). Delivery to the recipient's inbox adds a little more latency. The arrival time is roughly 9:05-9:15 PM. If you ever see it land before 9:05 PM, something is wrong; the report did not have time to compile.

Q: Can I be in two roles at once?

A: No. A user has exactly one role. The closest approximation is Admin-CLR, which combines the capabilities of CLR and admin. A user who genuinely needed to alternate behavior — being listed in the algorithm some weeks, not others — would toggle the Is CLR flag rather than changing role.

Q: What happens if I boost an LO who is already in my list?

A: The boost takes effect at the next daily generation (tomorrow at 5 AM). On that day, the LO's position in your list will likely rise; with default weights, an otherwise-mid-rank LO typically moves up 2-4 positions. If today's list is already wrong in your judgement, an admin can override to force a regeneration.

Q: How is data privacy handled?

A: We do not share data with any third party except Resend (for email delivery). We do not run any analytics or telemetry. Data is stored in our SQLite database and nowhere else. Backups are encrypted at rest in our cloud storage. Audit entries contain operational metadata but never passwords, API keys, or session tokens. See Chapters 24 and 30 for detail.

Q: Who owns this platform?

A: Legally and operationally, West Capital Lending. It was built by Chris Redoble and Ethan Wood as an internal tool. It is maintained by the engineering team at WCL and does not have external users or customers.

Q: What if I want a feature that does not exist?

A: Write it up — the problem you are trying to solve, not just the feature you want — and share it with Chris. Most requests turn out to be solvable with existing tools or with a small change. The platform evolves; feature ideas are welcome.

Q: Can I use CLR Center on mobile?

A: Yes, it is a Progressive Web App. Install it from the browser on iOS 16.4+ or Chrome/Edge on any platform. The mobile view is a responsive adaptation of the desktop view, not a separate app; everything the desktop has, mobile has. Some tables are horizontally scrollable on narrow screens.

Q: What browsers are supported?

A: Chrome (last 2 major versions), Edge (last 2 major versions), Safari 16+, and Firefox (last 2 major versions). Older browsers may work but are not tested. Internet Explorer is not supported in any version.

Q: How do I change my personal password?

A: From the user menu in the top-right, choose Settings. The Change Password section has fields for current password and new password. Enter both, click Save. You will remain signed in after the change.

Q: Can I set my own notification preferences?

A: Not currently. Notifications are team-wide policy set by admins. Individual customization is a feature idea that has not risen to the top of the list; share it if it matters to you.

Q: Why does the algorithm give more weight to old days since than to recent high frequency?

A: The two factors measure different things. Days Since is a staleness alarm; Call Frequency is a historical pattern. A stale LO with no recent history genuinely needs attention, whereas a frequently-called LO with a recent conversation may not need another call today. The weights (30/25) balance these — staleness gets slightly more weight, but not overwhelmingly so. See Chapter 16.

Q: Does the platform integrate with Bonzo?

A: Not automatically. The integration is informal: we store Bonzo credentials (Chapter 22) so CLRs can copy-paste into Bonzo easily, and CLRs manually log appointments in both platforms. A programmatic integration has been proposed but is not a priority.

Q: Can I see audit history for my own account?

A: Only admins can see the audit log directly. However, any CLR can see their own activity in the Appointments page, EOD history, and Team Chat messages. These are the user-facing slices of what the audit log captures in detail.

Quick Reference Cards for Common Tasks

Pocket-sized procedures for the tasks admins perform most often. Each card is a single numbered list of steps; no explanation, no context. Tear these out and pin them to your monitor.

Card 1: Add a new CLR

1. Admin → Users → + Add User.
2. Fill first name, last name, email.
3. Role: CLR. Is CLR: checked.
4. Create User. Welcome email fires.
5. Confirm with the new user in chat.

Card 2: Promote a CLR to Admin-CLR

1. Admin → Users → click their row.
2. Role: Admin-CLR. Is CLR: unchanged.
3. Save Changes.
4. Notify them. Refresh their browser.

Card 3: Archive a departing user

1. Wait until after their last EOD sends.
2. Admin → Users → click their row.
3. Archive.
4. Confirm: their sessions are invalidated.

Card 4: Reset a password for a stuck user

1. Admin → Users → click their row.
2. Reset password.
3. Notify user: 'new password in your inbox'.
4. They sign in and pick a personal one.

Card 5: Boost an LO

1. LO Roster → click the LO.
2. Boost toggle on.
3. Optional: add a note about why.
4. Save. Effective tomorrow at 5 AM.

Card 6: Change algorithm weights

1. Admin → Algorithm Settings.
2. Adjust sliders. Sum must equal 100%.
3. Check Preview; verify top-10 looks sane.
4. Announce change in team chat.
5. Save Weights. Effective tomorrow.

Card 7: Run an override

1. Admin → Algorithm Settings → Override.
2. Type reason (25+ chars required).
3. Confirm 1 of 3.
4. Confirm 2 of 3.
5. Type your own name. Confirm 3 of 3.
6. Broadcast fires to team.

Card 8: Rotate Bonzo password

1. Log into Bonzo directly.
2. Change password (use password manager).
3. Back in CLR Center: Admin → Bonzo Credentials.
4. Paste new password. Save.
5. Announce in team chat.

Card 9: Send an EOD test

1. Admin → Email Settings.
2. Send test email.
3. Confirm delivery within 2 minutes.
4. If fails: check Resend API key, from-domain DNS.

Card 10: Investigate a missing appointment

1. Admin → History.
2. Filter: actor=, date=yesterday.
3. Look for APPOINTMENT_LOGGED.
4. If found: display bug; hard-refresh.
5. If not: check APPOINTMENT_DELETED.
6. If still not: appointment was never saved; re-log.

Card 11: Manually reassign an NMLS task

1. Admin → NMLS Tasks.
2. Click the task row.
3. Reassign dropdown → pick new CLR.
4. Save. Notification fires to new assignee.

Card 12: Weekly start-of-week review (15 min)

1. History: scan weekend entries.
2. Users: scan last-login column.
3. Algorithm: verify weights unchanged.
4. NMLS Tasks: review upcoming 7-day calendar.
5. Email: send test email.
6. Team Stats: review week-over-week trend.

PART EIGHT

Appendix

Quick reference, weight defaults, permissions, glossary, and contacts.

Quick Reference Card

A single page of the facts you reach for most often.

Platform URL

<https://web-production-b6285.up.railway.app>

Primary contacts

Role	Name	When to contact
Platform owner	Chris Redoble	Operational questions, weight changes
Platform engineer	Ethan Wood	Bugs, code changes, deploy problems
Executive recipient	Scott Petrie	Business-level escalations

Daily cadence

Time	What happens
05:00	Daily list generation runs; each CLR's list appears
09:00	Typical start of CLR calling
17:00	Soft deadline for same-day appointment logging
21:00	EOD email sent to Scott Petrie and Chris Redoble
00:00	NMLS escalation sweep runs

Key numbers

- **60 days** — NMLS check cadence per LO.
- **7 days** — NMLS escalation window for missed checks.
- **14 days** — session cookie expiry.
- **10 failures / 15 minutes** — login lockout threshold.
- **3 days** — incomplete-LO reminder interval.

Common actions cheat sheet

I want to...	Go to...
Log an appointment	Dashboard → Log Appointment
See today's list	Your Calling List
Submit EOD report	EOD Reporting
Add a new CLR	Admin → Users → + Add User
Edit an LO's record	LO Roster → click row → Edit
Change algorithm weights	Admin → Algorithm Settings
Run admin override	Admin → Algorithm Settings → Override
View audit log	Admin → History
Rotate Bonzo password	Admin → Bonzo Credentials
See pending NMLS checks	Admin → NMLS Tasks
Rotate Resend API key	Admin → Email Settings

CHAPTER B

Algorithm Weight Defaults

The six weights, their defaults, and their meanings on one page.

Factor	Default	Rewards LOs who...
Days Since Last Call	30%	have not been called in a long time
Call Frequency	25%	have a consistent engagement history
Availability	20%	are available to take calls now
Boost	10%	a human has flagged as priority
Tier	5%	sit in a higher business tier
90-Day Transfers	10%	have transferred business recently
Total	100%	—

These defaults are the recommended steady-state configuration for a team of roughly a dozen CLRs calling a roster of roughly a hundred LOs. Change them only when a concrete business reason demands it (see Chapter 20).

CHAPTER C

Full Permission Matrix

Every capability, matched against every role.

Capability	CLR	Admin-CLR	Admin-only
View own dashboard	Yes	Yes	No
View own calling list	Yes	Yes	No
Log appointments (own)	Yes	Yes	No
Edit own appointments	Yes	Yes	No
Submit own EOD report	Yes	Yes	No
View team stats	Limited	Full	Full
Post in team chat	Yes	Yes	Yes
View LO roster	Read	Read+Edit	Read+Edit
Add/archive LO	No	Yes	Yes
Edit LO availability	No	Yes	Yes
Boost an LO	No	Yes	Yes
View LO stats	Read	Read	Read
Manage users	No	Yes	Yes
Change own password	Yes	Yes	Yes
Reset another's password	No	Yes	Yes
Edit algorithm weights	No	Yes	Yes
Run admin override	No	Yes	Yes
View audit log	No	Yes	Yes
Edit email settings	No	Yes	Yes
Edit Bonzo credentials	No	Yes	Yes
View NMLS tasks	Assigned only	Full	Full

Capability	CLR	Admin-CLR	Admin-only
Reassign NMLS tasks	No	Yes	Yes
Mark NMLS complete on others' behalf	No	Yes	Yes

This table is authoritative. If a screen in the platform appears to grant a permission inconsistent with this matrix, treat it as a bug and report it.

CHAPTER D

Glossary

The platform's vocabulary, defined.

The platform uses a compact vocabulary of specialized terms. This glossary defines every one in plain language. The entries are alphabetical.

Term	Definition
Admin-CLR	A user role that combines the CLR and admin capabilities. They can work the daily list and also manage the platform.
Admin-only	A user role that has admin capabilities without participating in the daily calling list. Used for managers and analysts.
Appointment	A scheduled interaction between a CLR and an LO. Recorded with a scheduled time, outcome, and notes.
Archived	A soft-delete state for users and LOs. Archived records are preserved but hidden from active views and excluded from the algorithm.
Audit log	The immutable record of state-changing actions. See Chapter 24.
Availability	An LO's self-reported schedule of hours when they can take calls. Used by the algorithm as one of the six weighted factors.
Boost	An admin action that flags an LO as high-priority. Boost contributes to the algorithm's Boost factor (default weight 10%).
Bonzo	The external CRM tool WCL uses. Credentials are stored on the Bonzo Credentials admin page (Chapter 22).
Broadcast	A notification with user_id NULL, shown to every user.
CLR	Connection Lead Representative. The front-line user role that runs the daily calling list.

Term	Definition
Cron	A scheduled job. The platform runs several: daily list at 5 AM, EOD email at 9 PM, NMLS escalation sweep at midnight.
Daily generation	The 5 AM algorithm run that computes each CLR's calling list for the day.
Deprioritize	The opposite of Boost. Marks an LO as lower priority without fully disqualifying them.
EOD	End-of-day. The daily summary each CLR submits and the email that aggregates those summaries to Scott and Chris.
Is CLR	A boolean flag on the user record indicating whether they take daily assignments. Distinct from the role field.
LO	Loan Officer. The external mortgage professionals the CLRs call.
NMLS	Nationwide Multistate Licensing System. The regulatory body that licenses loan officers. See Chapter 23.
Override	An admin action that forces a re-run of the daily algorithm before the next scheduled 5 AM run. See Chapter 18.
PWA	Progressive Web App. The CLR Center is installable as a PWA on Chrome, Edge, and Safari iOS 16.4+.
Railway	The platform-as-a-service that hosts CLR Center's container and volume.
Resend	The transactional email API used because Railway blocks direct SMTP.
Session	A logged-in user's authenticated state, represented by the signed clr_session cookie. 14-day lifetime.
Tier	The business classification of an LO: Standard, Premium, or Top. Used as one of the algorithm's weighted factors (default 5%).
Transfer	An appointment outcome where a CLR hands a lead directly to an LO for immediate follow-up. 90-day transfer count is one algorithm factor.
WCL	West Capital Lending. The company.

CHAPTER E

Contact & Support

Who to reach, how to reach them, and what to expect.

Escalation path

Support flows through two people in order, escalating only when needed:

- 1. **Chris Redoble** — platform owner. Handles operational questions: 'how do I do X,' 'why does Y work this way,' 'can we change Z.' Responds within a business day.
- 2. **Ethan Wood** — platform engineer. Handles technical issues: bugs, deploy failures, schema changes, new features. Responds within a business day for non-urgent requests, same-day for production incidents.

How to open a ticket

We do not run a formal ticketing system at our scale. Reach out through whichever channel is already open between you and the contact — team chat, email, Slack. When opening a ticket, include these five items:

- The URL you were on.
- The exact time (including timezone).
- The signed-in user's email.
- The browser name and version.
- A screenshot of the error or unexpected behavior.

What to expect

Issue type	Response target	Resolution target
Production down	15 minutes	Same day
Data incorrect	1 business day	2 business days
Feature question	1 business day	When answered
Feature request	3 business days	Prioritized in roadmap

These targets are conservative; most questions are answered within an hour or two during business hours. But the table above is what you should plan around when communicating expectations to the rest of the team.

After-hours and weekends

Neither contact is on call. After-hours messages will be seen the next business morning. For true emergencies — production down, data loss — contact both Chris and Ethan directly and expect best-effort response. Such events are rare; in the platform's history so far they have not occurred.

A final word

CLR Connection Center exists because West Capital Lending's Connection Lead Representatives needed a tool that actually matched their work. It was built by Chris Redoble and Ethan Wood in close collaboration with the CLR team, and it is maintained by them with the same care.

If you are reading this manual because you are new to the team — welcome. The platform will become second nature within a week. Ask questions liberally; every question makes the next person's onboarding faster.

If you are reading this manual because you are the next owner — congratulations, and thank you for taking it on. Every chapter here was written to make your job easier. Read it once cover to cover, keep it next to you for the first month, and then let this document live alongside the platform as its permanent second self.

Built by Chris Redoble & Ethan Wood · West Capital Lending · 2026

Closing Notes from the Builders

CLR Connection Center was built over the course of several months of close collaboration between Chris Redoble, who articulated the operational needs of the WCL CLR team in exacting detail, and Ethan Wood, who shaped those needs into the platform you have been reading about. This manual is the written summary of everything we learned together. We hope it serves you.

A note on simplicity

If you have read through the entire manual, you may have noticed a recurring theme: the platform is simpler than it could be, on purpose. We could have built a machine-learning-tuned algorithm. We could have integrated with Bonzo directly. We could have added mobile-native apps, rich notification preferences, multi-tenant support. We chose not to.

Each piece we did not build is a piece that does not need to be maintained, does not break, does not need documentation, does not need onboarding. The platform is what it is — no more, no less — and that is its value. A CLR can learn the whole thing in two weeks. A new admin can understand every button in a week. An engineer can read the entire codebase in a day. That compactness is not a limitation; it is the product.

A note on trust

The platform's daily job — deciding which LOs each CLR calls — is a decision that used to be made by team leads subjectively. Moving that decision into an algorithm is only valuable if the team trusts the algorithm. We worked hard to earn that trust: a simple, inspectable scoring function; a score breakdown visible for every assignment; an override protocol that is deliberately painful; an audit log that records every tuning move.

Trust is not given; it is accumulated, call by call, week by week. The platform has been trusted by the WCL CLR team for long enough that it is now part of the fabric of the job. Maintain that trust — by using override sparingly, by communicating weight changes, by keeping the data clean — and the platform continues to earn its keep.

A note on continuous improvement

This manual is a snapshot. The platform will change over time; the manual must change with it. When you make a change to the platform — a new feature, a new setting, a new runbook — update the relevant chapter here. When you resolve a support incident that took more than half an hour, add an entry to Chapter 33. Treat the manual as a living document, not a frozen artifact.

The best predictor of whether a platform continues to serve its team well is whether the documentation keeps pace with the code. If a year passes without this manual being touched, the manual is probably wrong in ways that matter. Schedule a quarterly documentation review alongside your quarterly credential rotation. Both are small investments that compound.

A note on the team

The platform exists to serve the people who use it. The WCL CLR team — Bill, Khashi, Sean, Dan, Aaron, Ian, Cole, James, Derek, Gary, Kurt, and Sean R. at the time of writing — do the work that the platform supports. Every feature in CLR Connection Center was shaped by their feedback, their complaints, their moments of friction, their suggestions. It is their platform more than it is anyone else's.

If you are a future owner reading this after the team has changed — as it inevitably will — remember that the platform's purpose is always to serve the current team, not to preserve decisions made by the previous one. When the team's needs change, the platform should change with them. The rhythm of the platform is the rhythm of the people who use it.

A note on pride

Chris and Ethan are proud of this platform, and we are prouder still of the team that shaped it. CLR Connection Center is not a commercial product, not a reusable framework, not a general-purpose tool. It is one team's answer to one team's specific operational question. Answers like that do not scale to every team in every industry — but for the team that asked the question, the answer is exactly right. That specificity is its craft.

West Capital Lending · CLR Connection Center · 2026

Built by Chris Redoble & Ethan Wood.